# Assignment 1

## Robotics and Embodied Intelligence

Compiled Chapter Reports

Zihao Liu
Student ID: 220261979
213222647@seu.edu.cn

October 2026

| Chapter | Topic |
|---|---|
| Chapter 3 | Acrobots, Cart-Poles, and Quadrotors |
| Chapter 4 | Simple Models of Walking and Running |
| Chapter 5 | Highly-articulated Legged Robots |
| Chapter 1 | Introduction: Manipulation, Teleoperation and Drake |

This document compiles the four chapter reports for Assignment 1. Each chapter has its own title page, contents, figures, code excerpts and autograder results. The accompanying code is in `Assignment_1_Code.zip`.

# Chapter 3 Assignment

## Acrobots, Cart-Poles, and Quadrotors

Zihao Liu
Student ID: 220261979
213222647@seu.edu.cn

October 2026

**Abstract**

This report covers the Chapter 3 assignment. Part I comments on the four textbook examples (Examples 3.1, 3.2, 3.5 and 3.6): the supplied code was run, the numerical results are interpreted using the theory of underactuated robotics, and sensitivity studies on controller weights, initial conditions, actuator limits and physical parameters are reported. Part II solves all exercises of Section 3.8: cart-pole linearisation and balancing, URDF modelling of single/double cart-poles with LQR, controllability of discrete and continuous LTI systems, and the comparison between nonlinear stability and its linearisation. The most important code is reproduced in the report; the complete Python code and notebooks accompany the submission.

## Contents

**1 Overview and Reproducibility** **3**

**2 Example 3.1: Dynamics of the Acrobot** **4**
2.1 Model and equilibrium . . . . . 4
2.2 What was run . . . . . 5
2.3 Linearisation and controllability . . . . . 5
2.4 LQR design and closed-loop behaviour . . . . . 5
2.5 Sensitivity study . . . . . 6

**3 Example 3.2: Dynamics of the Cart-Pole System** **8**
3.1 Model and linearisation . . . . . 8
3.2 LQR results and sensitivity . . . . . 9

**4 Example 3.5: LQR for the Acrobot and Cart-Pole** **11**
4.1 Theory and design . . . . . 11
4.2 Comparison of the two systems . . . . . 11
4.3 Saturation and sensitivity . . . . . 13

**5 Example 3.6: LQR for Quadrotors** **13**
5.1 Planar quadrotor . . . . . 13
5.2 3D quadrotor . . . . . 15

**6 Section 3.8: Exercise 3.1** **17**
6.1 Nonlinear dynamics and linearisation . . . 17
6.2 Linearisation error . . . 17
6.3 LQR and recovery . . . 19

**7 Section 3.8: Exercise 3.2** **19**
7.1 URDF construction . . . 19
7.2 Double pendulum . . . 20
7.3 LQR and results . . . 21

**8 Section 3.8: Exercise 3.3** **23**
8.1 Discrete double integrator . . . 23
8.2 Continuous LTI controllability . . . 24

**9 Section 3.8: Exercise 3.4** **25**
9.1 Model and linearisation . . . 25
9.2 Stability table . . . 27
9.3 Discussion . . . 27

**10 Conclusions** **27**

**11 Code and Reproduction** **30**

**A Key Code Snippets** **31**
A.1 Cart-pole linearisation and LQR . . . 31
A.2 URDF pendulum link and joint . . . 31
A.3 Controllability and two-step reachability . . . 31
A.4 Quadrotor LQR . . . 32

# 1 Overview and Reproducibility

The assignment consists of two parts:

- **Examples** (code supplied, run and comment on the results): Examples 3.1, 3.2, 3.5 and 3.6.
- **Section 3.8** (all exercises to be solved): Exercises 3.1–3.4.

All simulations use Drake 1.58.0 and the `underactuated` package 2026.9.8 inside the Python environment `/Users/leo/venvs/drake/bin/python` (NumPy 2.5.4, SciPy 1.18.1, Matplotlib 3.11.2). The repository layout is:

| Contents | Location |
|---|---|
| Example notebooks and analyses | `Ch-3/Ch.3-examples/` |
| Section 3.8 notebooks | `Ch-3/Ch.3 Exercises/` |
| Section 3.8 analysis scripts | `Ch-3/Ch.3 Exercises/exercises_analysis/` |
| This report | `Ch-3/Ch3_Assignment_Report/` |

The autograder results are:

| Notebook | Score |
|---|---|
| `cartpole_balancing.ipynb` | 13/13 |
| `cartpoles_urdf.ipynb` | 12/12 |

The examples are run with the commands listed in Section 11. All random studies use fixed seeds, so the tables are reproducible.

### How to read this report

The report contains three layers: (i) *models and equations*, (ii) *numerical experiments*, and (iii) *interpretation*. If a formula looks dense, read the **Intuition** boxes first: they give the physical picture without algebra. The **Recipe** boxes summarise a procedure you can reuse, and the **Common pitfall** boxes point out mistakes that are easy to make.

| Part | What is done | What to look for |
|---|---|---|
| Example 3.1 | Acrobot dynamics, linearisation, LQR | underactuation is not the same as uncontrollability; weak directions; saturation |
| Example 3.2 | Cart-pole dynamics, linearisation, LQR | zero eigenvalues from the cart; right-half-plane zero; slow cart mode |
| Example 3.5 | LQR comparison of acrobot and cart-pole | condition number, non-minimum phase, basin of attraction, actuator limits |
| Example 3.6 | Planar and 3D quadrotor LQR | linearisation at *nonzero* thrust; $Q/R$ trade-off; wind offset |
| Exercises 3.1–3.4 | Section 3.8 | linearisation error, URDF frames, reachability proof, stability table |

**The big picture in five sentences**

All systems in this chapter are *underactuated*: there are fewer actuators than degrees of freedom. Linearisation replaces the nonlinear system by its tangent at an equilibrium; LQR stabilises that tangent, so the guarantee is *local*. Two effects limit the usable region: the system may be *non-minimum phase* (it first moves the wrong way) and the actuator may saturate. Controllability answers "can we reach any state?"; it does not answer "how hard is it?". The quadrotor is special only in that the linearisation is taken around a hover point with non-zero thrust.

| Symbol | Meaning |
|---|---|
| $\mathbf{x} = [q, \dot{q}]^\top$ | state (positions and velocities) |
| $\mathbf{x}^\star$ | equilibrium / operating point |
| $A = \partial f/\partial \mathbf{x}$, $B = \partial f/\partial u$ | linearisation at $\mathbf{x}^\star$ |
| $\mathcal{C} = [B, AB, \ldots, A^{n-1}B]$ | controllability matrix |
| $Q \succeq 0$, $R \succ 0$ | LQR state and input weights |
| $K$ or $D$ | state-feedback gain ($u = -K(\mathbf{x} - \mathbf{x}^\star)$ or $u = D(\mathbf{x} - \mathbf{x}^\star)$) |
| RHP zero | right-half-plane zero (non-minimum phase) |

# 2 Example 3.1: Dynamics of the Acrobot

## 2.1 Model and equilibrium

The acrobot is a two-link planar robot in a vertical plane. The shoulder joint is unactuated and the elbow joint is driven by a torque $u = \tau$. Its manipulator equations are

$$M(q)\ddot{q} + C(q, \dot{q})\dot{q} + G(q) = \begin{bmatrix} 0 \\ u \end{bmatrix}, \qquad q = \begin{bmatrix} \theta_1 \\ \theta_2 \end{bmatrix}, \qquad \mathbf{x} = [\theta_1, \theta_2, \dot{\theta}_1, \dot{\theta}_2]^\top. \tag{1}$$

The upright equilibrium is $\mathbf{x}^\star = [\pi, 0, 0, 0]^\top$. The default Drake parameters are $m_1 = m_2 = 1$ kg, $l_1 = 1$ m, $l_2 = 2$ m, $l_{c1} = 0.5$ m, $l_{c2} = 1$ m, $I_{c1} = 0.083$ kg m$^2$, $I_{c2} = 0.33$ kg m$^2$ and $b_1 = b_2 = 0.1$.

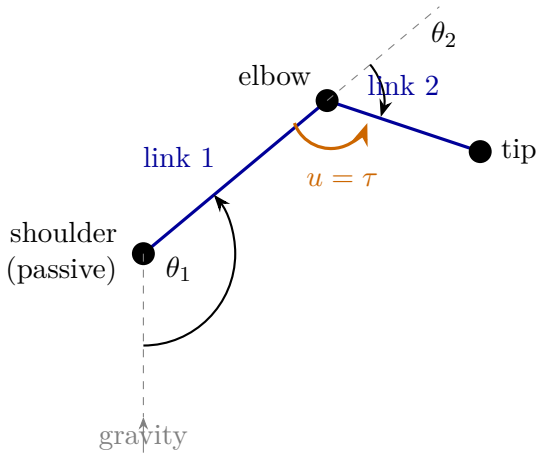



Figure 1: Acrobot schematic. The shoulder is not actuated; the only input is the elbow torque $u = \tau$. $\theta_1$ is measured from the downward vertical and $\theta_2$ from the extension of link 1. Because the shoulder has no motor, the only way to move $\theta_1$ is through the inertial coupling produced by the elbow motion.

**Why can a one-motor two-link robot still be controllable?**

Imagine holding a chain by one end and moving it with your wrist. The far end is not directly actuated, but it moves because the links are coupled by their mass and inertia. The acrobot works the same way: the elbow torque accelerates link 2, and the reaction on link 1 changes $\theta_1$. This is why the linearised pair $(A, B)$ can have rank 4 even though the shoulder has no motor. The coupling is not equally strong in all directions, however; that is the meaning of "weak controllability" below.

### 2.2 What was run

The supplied notebook `Exp3'1-acrobot.ipynb` was run. It (i) simulates the response to a constant elbow torque, (ii) linearises the system at $\mathbf{x}^\star$ and checks controllability, and (iii) closes an LQR loop around $\mathbf{x}^\star$. The numerical analysis in `exp1_analysis/exp1_analysis.py` repeats the linearisation and runs 100-trial Monte-Carlo studies with and without the $\pm 10$ N m actuator limit.

### 2.3 Linearisation and controllability

At $\mathbf{x}^\star$,

$$\mathbf{A} = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 12.629 & -12.693 & -0.172 & 0.3015 \\ -14.749 & 29.612 & 0.3015 & -0.6033 \end{bmatrix}, \qquad \mathbf{B} = \begin{bmatrix} 0 \\ 0 \\ -3.0147 \\ 6.0332 \end{bmatrix}. \tag{2}$$

The open-loop eigenvalues are

$$\lambda(\mathbf{A}) = \{-6.492,\ -2.249,\ +2.231,\ +5.734\}, \tag{3}$$

so the upright position is unstable (two eigenvalues with positive real part). The controllability matrix $\mathcal{C} = [\mathbf{B}, \mathbf{AB}, \mathbf{A}^2\mathbf{B}, \mathbf{A}^3\mathbf{B}]$ has

$$\sigma(\mathcal{C}) = [511.7,\ 123.1,\ 0.0807,\ 0.0715], \qquad \operatorname{rank}\mathcal{C} = 4, \qquad \operatorname{cond}\mathcal{C} \approx 7.15 \times 10^3. \tag{4}$$

**Comment.** The acrobot is underactuated (one actuator, two degrees of freedom), yet the linearised system is completely controllable. This is the central lesson of Chapter 3: underactuation does not imply loss of controllability; the passive joint is driven through the inertial coupling. The very small last two singular values show, however, that some directions are *weakly* controllable. The slow modes have input participation factors of about 0.048 compared with 3.6 for the fast modes, so driving them requires large inputs or long manoeuvres.

### 2.4 LQR design and closed-loop behaviour

With $Q = \operatorname{diag}(10, 10, 1, 1)$ and $R = 1$, the solved Riccati equation gives the controller $u = D(\mathbf{x} - \mathbf{x}^\star)$ with

$$D = [\,278.44,\ 112.29,\ 119.72,\ 56.83\,], \tag{5}$$

and closed-loop poles

$$\lambda_{\text{cl}} = \{-10.126,\ -4.238,\ -2.242 \pm 0.039j\}. \tag{6}$$

All poles are in the left half-plane, hence the nonlinear closed loop is locally exponentially stable at $\mathbf{x}^\star$.

The 100-trial Monte-Carlo study (initial state $\mathbf{x}_0 = \mathbf{x}^\star + 0.05\,\mathcal{N}(0, I_4)$, horizon 4 s) gives:

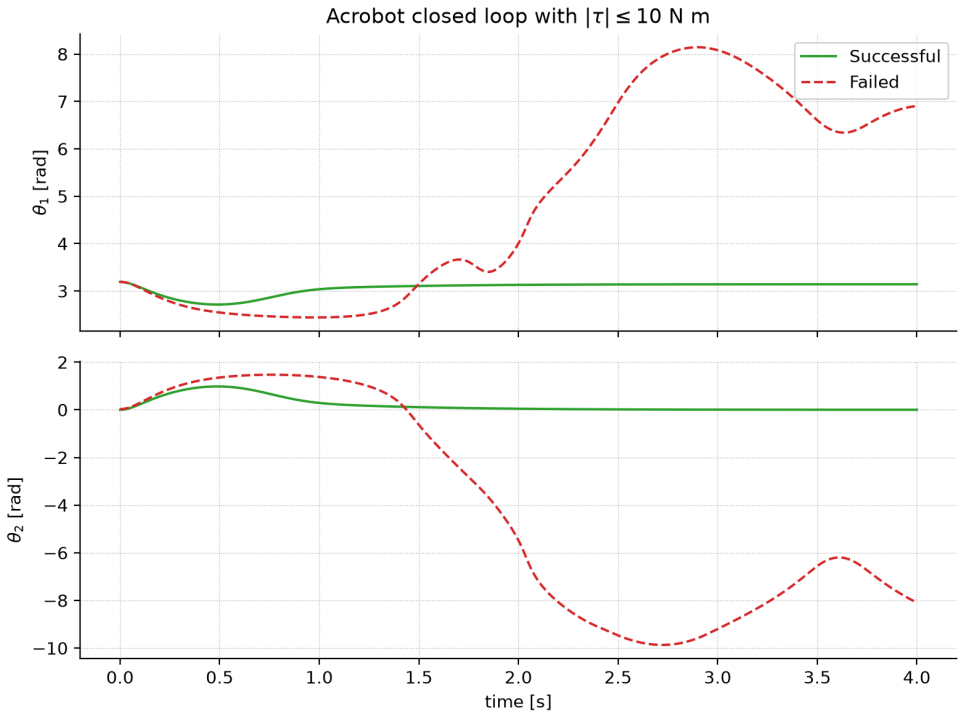



Figure 2: Acrobot: a successful (green) and a failed (red) closed-loop trajectory with the $\pm 10$ N m saturation. The successful trajectory shows the non-minimum-phase undershoot: the elbow first moves far away from zero before returning.

| Condition | Success rate | Peak torque (median) |
|---|---|---|
| No saturation | 98% | 11.2 N m |
| $\lvert\tau\rvert \le 10$ N m | 63% | 11.2 N m |

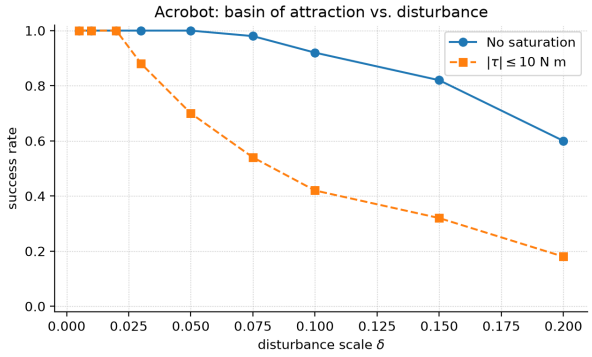
The successful trials have a median peak angular deviation of 0.855 rad and a median settling time of 1.63 s.

**Comment.** Two effects explain the failures. First, the linear LQR guarantee is local; the nonlinear system has a finite basin of attraction. Second, the saturated actuator turns the problem into a constrained nonlinear one, for which the unconstrained LQR optimality no longer applies. The linearised system is also non-minimum phase: the right-half-plane zeros of $u \to \theta_1$ are $\pm 2.052$ and of $u \to \theta_2$ are $+2.283, -2.304$. State feedback does not move zeros, so the closed loop must first move the state away from the origin before returning; this is visible in Fig. 2.

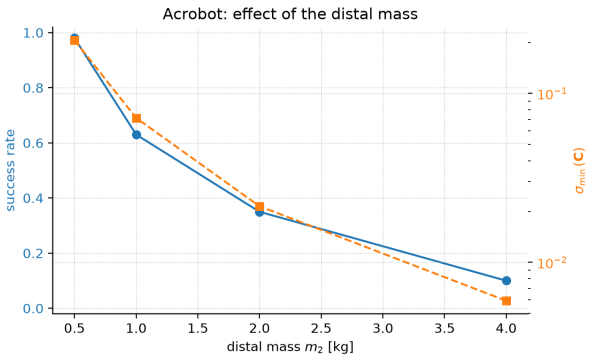
## 2.5 Sensitivity study

**Initial disturbance.**

| $\delta$ | 0.005 | 0.02 | 0.05 | 0.075 | 0.10 | 0.15 | 0.20 |
|---|---|---|---|---|---|---|---|
| No saturation | 100% | 100% | 100% | 98% | 92% | 82% | 60% |
| $\lvert\tau\rvert \le 10$ N m | 100% | 100% | 70% | 54% | 42% | 32% | 18% |




(a) Success rate versus disturbance scale, with and without saturation.




(b) Effect of the distal mass $m_2$.

Figure 3: Acrobot sensitivity studies. Saturation shrinks the basin of attraction substantially; increasing the distal mass makes the system much harder to control.

**Distal mass $m_2$.**

| $m_2$ [kg] | 0.5 | 1.0 | 2.0 | 4.0 |
|---|---|---|---|---|
| $\sigma_{\min}(\mathcal{C})$ | 0.207 | 0.0715 | 0.0215 | 0.0059 |
| $\|D\|$ | 153.2 | 328.2 | 864.3 | 2655.7 |
| Success rate | 98% | 63% | 35% | 10% |
| Median peak torque [N m] | 5.1 | 11.2 | 47.8 | 317.0 |

**Control weight $R$.**

| $R$ | 0.1 | 1 | 10 | 100 |
|---|---|---|---|---|
| $\|D\|$ | 577.1 | 328.2 | 289.7 | 285.5 |
| Success rate | 65% | 63% | 62% | 62% |
| Median peak torque [N m] | 19.8 | 11.2 | 9.8 | 9.7 |

**Comment.** The success rate decreases monotonically with the disturbance size, confirming the finite basin of attraction. The distal mass is the most sensitive physical parameter because it increases the effective inertia and weakens the input leverage; the minimum singular value drops by a factor of 35 and the success rate collapses from 98% to 10%. Increasing $R$ reduces the peak torque only mildly: in the "expensive control" limit the closed-loop poles tend to the mirrors of the stable open-loop poles and the unstable poles, so the gain cannot go to zero for an unstable system.

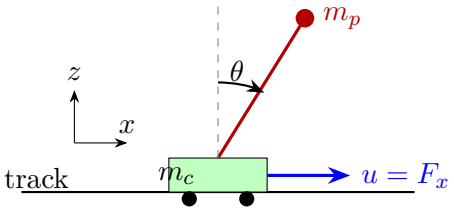



Figure 4: Cart-pole schematic. The cart of mass $m_c$ slides along $x$; the pole of mass $m_p$ is attached by a passive hinge. The only input is the horizontal force $u = F_x$. $\theta$ is measured from the upward vertical ($\theta = \pi$ is upright in the model used in this report).

# 3 Example 3.2: Dynamics of the Cart-Pole System

## 3.1 Model and linearisation

**What the state and the input mean**

$x$ is where the cart is, $\theta$ is how far the pole is from upright, $\dot{x}$ and $\dot{\theta}$ are their velocities, and $u$ is the only force we can apply. There is no motor at the pole hinge, so the pole angle can only be changed by moving the cart. This is the whole difficulty of the cart-pole: we must use one actuator to control two coordinates.

The cart-pole has state $\mathbf{x} = [x, \theta, \dot{x}, \dot{\theta}]^\top$ and input $u = F_x$. The model used by the notebook (`underactuated/models/cartpole.urdf`) has $m_c = 10$ kg, $m_p = 1$ kg, $l_c = 0.5$ m and no joint damping. The upright equilibrium is $\mathbf{x}^\star = [0, \pi, 0, 0]^\top$. Linearisation gives

$$\mathbf{A} = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0.981 & 0 & 0 \\ 0 & 21.582 & 0 & 0 \end{bmatrix}, \qquad \mathbf{B} = \begin{bmatrix} 0 \\ 0 \\ 0.1 \\ 0.2 \end{bmatrix}. \tag{7}$$

The open-loop eigenvalues are $0, 0, \pm 4.6456$; the two zeros come from the cart position integrator. The controllability singular values are

$$\sigma(\mathcal{C}) = [4.326,\ 4.326,\ 0.0907,\ 0.0907], \qquad \operatorname{cond} \mathcal{C} \approx 47.7. \tag{8}$$

**Comment.** The pair is controllable, but the two small singular values correspond to the cart-position direction, which is only weakly controllable. The transfer function from $u$ to $x$ has a right-half-plane zero at $+4.43$: the cart must first move in the opposite direction to bring the pole back. This non-minimum-phase behaviour is visible as a cart undershoot of about 0.3 m for a 0.1 disturbance.

**What a right-half-plane zero really means**

A zero at $+4.43$ does not mean that the system is unstable. It means that the cart's response to a force starts in the *opposite* direction. Physically: to bring the pole back to the left, the cart first accelerates to the right so that the pole starts falling left; then the cart reverses to catch it. The net initial displacement is therefore opposite to the final one. This "undershoot" is not a controller bug: state feedback cannot move a zero, so LQR keeps it. It also explains why the cart moves about 0.3 m in the wrong direction for a 0.1 disturbance, and why aggressive

saturation can destroy the recovery.

### 3.2 LQR results and sensitivity

With $Q = \mathrm{diag}(10, 10, 1, 1)$ and $R = 1$, the gain is

$$D = [\,3.1623,\ -254.3925,\ 9.7692,\ -55.1774\,], \qquad u = D(\mathbf{x} - \mathbf{x}^\star), \tag{9}$$

with closed-loop poles

$$-4.7206, \quad -4.5739, \quad -0.3820 \pm 0.3761j. \tag{10}$$

The two fast poles bring the pole back; the slow complex pair is the cart returning to the origin (time constant $\approx 2.6$ s). Over a 5 s horizon the cart often has not fully returned even when the pole is balanced.

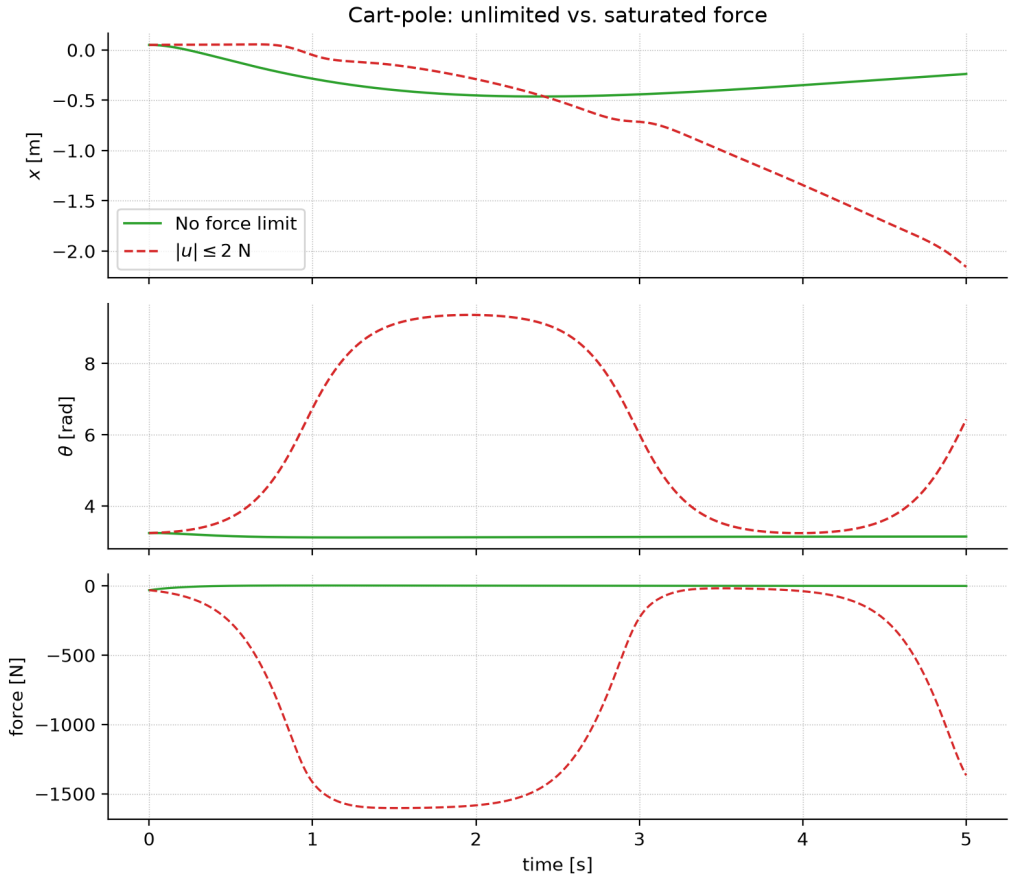
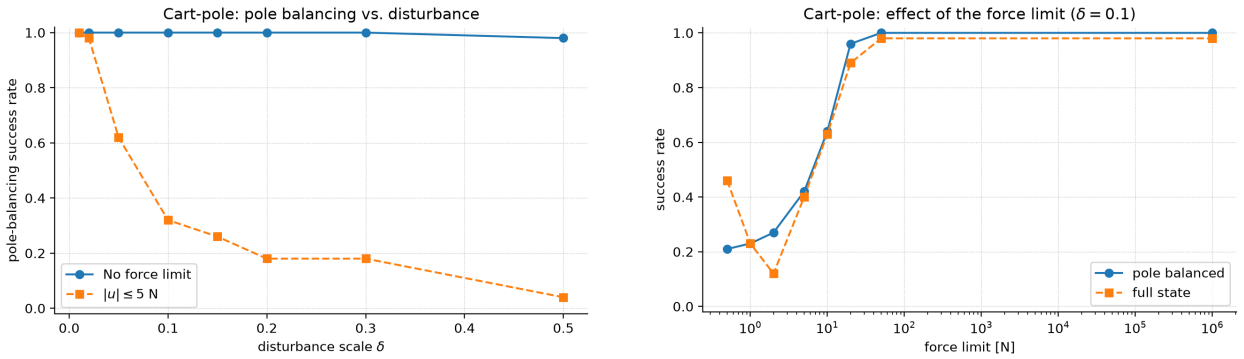



Figure 5: Cart-pole: the same initial condition without a force limit (green) and with a $\pm 2$ N limit (red). The unlimited controller balances the pole, whereas the saturated one loses it.

The 100-trial study with $\mathbf{x}_0 = \mathbf{x}^\star + 0.1\,\mathcal{N}(0, I_4)$ gives:

| Condition | Pole balanced | Full state | Peak force (median) |
|---|---|---|---|
| No limit | 100% | 98% | 17.2 N |
| $\lvert u\rvert \le 20$ N | 90% | 82% | – |
| $\lvert u\rvert \le 5$ N | 36% | 33% | – |




(a) Disturbance-size sweep.

(b) Effect of the force limit at $\delta = 0.1$.

Figure 6: Cart-pole sensitivity: the pole itself is robust without saturation, but a realistic force limit quickly reduces the basin of attraction.

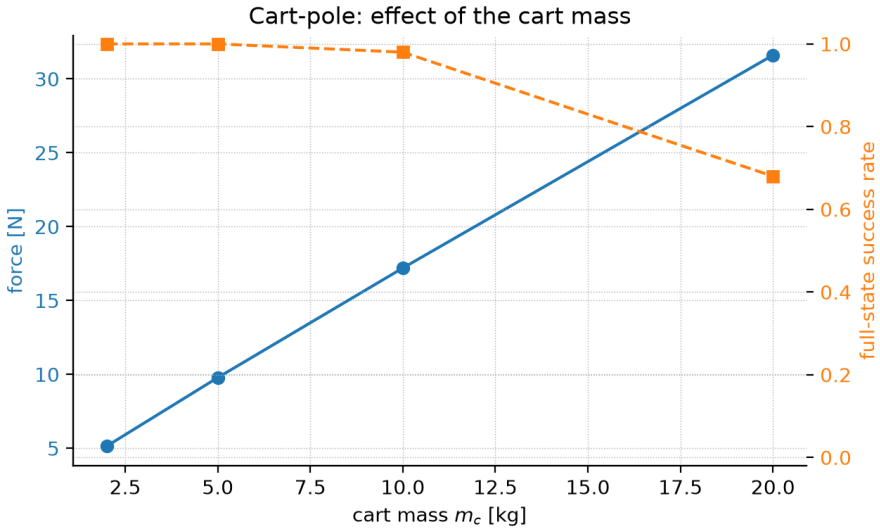



Figure 7: Effect of the cart mass $m_c$. A heavier cart requires more force and slows down the cart-return mode.

**Comment.** The original notebook has no force saturation, so the pole almost always balances. The real constraint is the actuator: the median peak force is 17.2 N and the 90th percentile is 40 N. When the limit is reduced to 5 N, saturation destroys the non-minimum-phase undershoot and the pole falls. The cart mass is the dominant physical parameter: from 2 to 20 kg the required force grows from about 5 N to 32 N and the full-state success rate falls from 100% to 68%, while the pole itself remains balanceable without saturation. Increasing $R$ saves force but makes the slow cart mode slower; for $R = 100$ the slow pole is about $-0.12$, so the cart cannot return within 5 s.

# 4 Example 3.5: LQR for the Acrobot and Cart-Pole

## 4.1 Theory and design

For $\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}u$ and the cost

$$J = \int_0^\infty \left( \mathbf{x}^\top \mathbf{Q} \mathbf{x} + u^\top \mathbf{R} u \right) dt, \tag{11}$$

the optimal state feedback is $u = -\mathbf{R}^{-1}\mathbf{B}^\top P \mathbf{x}$ with $P$ solving the continuous-time algebraic Riccati equation

$$\mathbf{A}^\top P + P\mathbf{A} - P\mathbf{B}\mathbf{R}^{-1}\mathbf{B}^\top P + \mathbf{Q} = 0. \tag{12}$$

If the linearised pair is stabilisable, LQR returns a stabilising gain for any $\mathbf{Q} \succeq 0$, $\mathbf{R} \succ 0$. The closed-loop eigenvalues determine local exponential stability of the nonlinear system; they say nothing about the size of the basin of attraction or about input saturation.

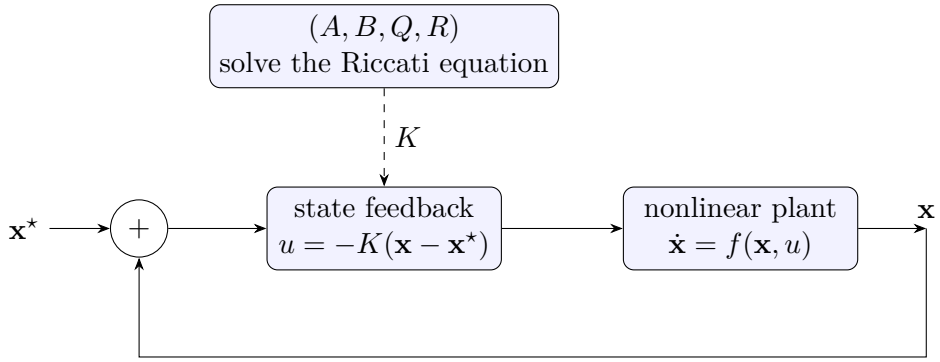



Figure 8: LQR at a glance. The design block solves the algebraic Riccati equation once, offline. The online controller is just a linear state feedback, so it is cheap to run. The nonlinear simulation is what tells us how far from $\mathbf{x}^\star$ the linear guarantee remains useful.

**Reading $Q$ and $R$**

Think of the cost as a negotiation between two goals. $Q$ says "keep the state close to the equilibrium"; $R$ says "do not use too much actuator effort". Increasing $Q$ (or decreasing $R$) makes the controller more aggressive: it corrects faster but uses larger inputs. Increasing $R$ makes it gentler: smaller inputs, slower response, and often a larger steady-state error under a disturbance. The Riccati equation finds the best compromise for the *linear* model; it knows nothing about saturation, friction or the size of the nonlinear basin. That is why the simulations in this report are essential.

## 4.2 Comparison of the two systems

**Comment.** Both systems are underactuated and both are controllable. The acrobot is much more weakly controllable (condition number about 150 times larger), because the passive shoulder is driven only through the elbow coupling. The cart-pole has two zero eigenvalues from the cart integrator, which produce a slow closed-loop mode. Both have right-half-plane zeros, so the closed loop necessarily exhibits undershoot.

Table 1: Linearisation and LQR comparison (Drake default parameters).

| | Acrobot | Cart-pole |
|---|---|---|
| Controllability rank | 4 | 4 |
| Condition number of $\mathcal{C}$ | $7.15 \times 10^3$ | 47.7 |
| Right-half-plane zeros | $+2.05,\ +2.28$ | $+4.43$ |
| Gain $D$ ($u = D(\mathbf{x} - \mathbf{x}^\star)$) | $[278.4, 112.3, 119.7, 56.8]$ | $[3.16, -254.4, 9.77, -55.2]$ |
| Closed-loop poles | $-10.13, -4.24, -2.24 \pm 0.04j$ | $-4.72, -4.57, -0.38 \pm 0.38j$ |

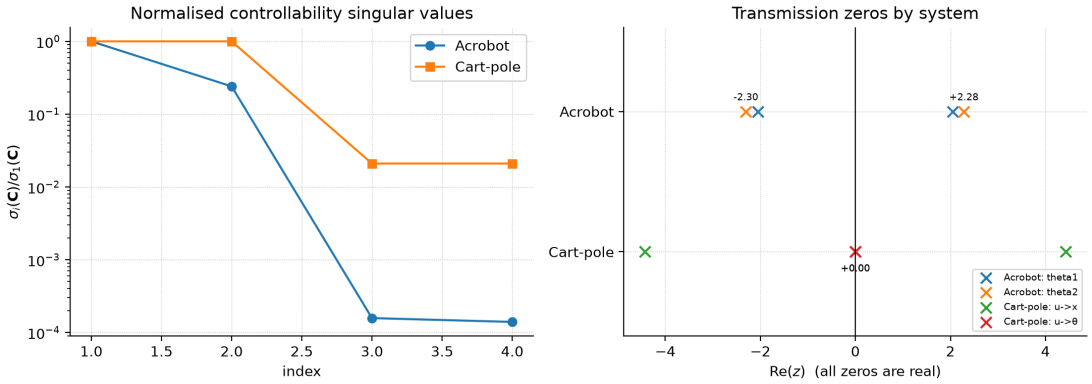



Figure 9: Normalised controllability singular values and right-half-plane zeros. Both systems are controllable, but the acrobot is much more weakly controllable; both are non-minimum phase.

Table 2: Baseline success rates for 100 random disturbances.

| System | Input constraint | Success | Peak control |
|---|---|---|---|
| Acrobot | none | 98% | median 11.2 N m |
| Acrobot | $\lvert\tau\rvert \le 10$ N m | 63% | – |
| Cart-pole | none | 100% (pole), 99% (full) | median 18.4 N |
| Cart-pole | $\lvert u\rvert \le 20$ N | 90% (pole), 82% (full) | – |
| Cart-pole | $\lvert u\rvert \le 5$ N | 36% (pole), 33% (full) | – |

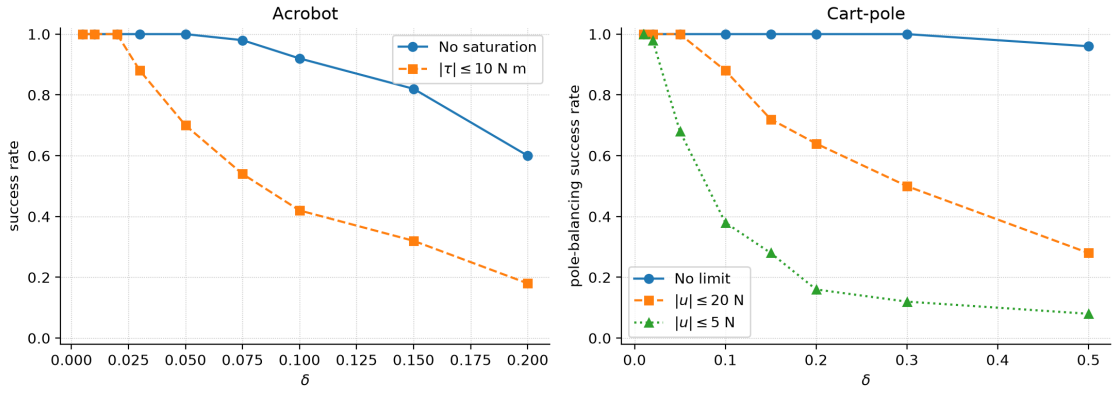
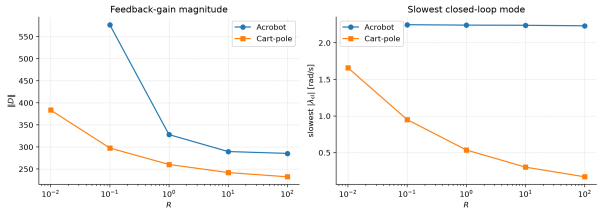
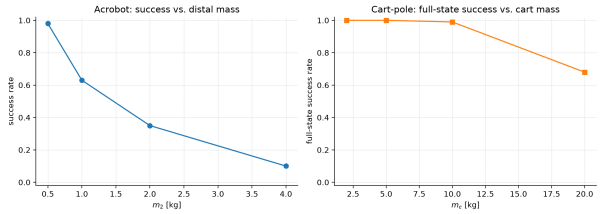



Figure 10: Success rate versus disturbance scale for the acrobot with and without torque saturation (left) and the cart-pole with different force limits (right).




(a) Effect of $R$ on the gain and the slow pole.




(b) Physical-parameter sensitivity.

Figure 11: Weight and parameter sensitivity. For the cart-pole, increasing $R$ slows the cart-return mode; for the acrobot, the distal mass is the critical parameter.

### 4.3 Saturation and sensitivity

**Comment.** The acrobot needs a finite minimum torque even in the "expensive control" limit, because its linearisation has unstable poles. The cart-pole's slow pole moves from 1.66 to 0.17 rad/s as $R$ grows from 0.01 to 100, so the full-state success rate falls from 100% to 27% even though the pole remains balanced. The physical-parameter studies show two different failure modes: the acrobot becomes genuinely uncontrollable in practice when $m_2$ is large, whereas the cart-pole mainly needs more force and a longer time for the cart to return.

## 5 Example 3.6: LQR for Quadrotors

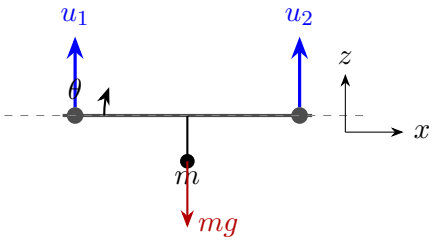



Figure 12: Planar quadrotor schematic. The two propeller thrusts are the only inputs. At hover, $u_1 + u_2 = mg$ and $\theta = 0$. The vehicle can translate by tilting and then using the horizontal component of the total thrust.

### 5.1 Planar quadrotor

The planar quadrotor has state

$$\mathbf{x} = [x, z, \theta, \dot{x}, \dot{z}, \dot{\theta}]^\top \tag{13}$$

and inputs $u = [u_1, u_2]^\top$ (two propeller thrusts). With mass $m$, arm length $l$, inertia $I$ and gravity $g$, the dynamics are

$$\ddot{x} = \frac{1}{m}\left(w_x - (u_1 + u_2)\sin\theta\right), \tag{14}$$

$$\ddot{z} = \frac{1}{m}\left(w_y + (u_1 + u_2)\cos\theta\right) - g, \tag{15}$$

$$\ddot{\theta} = \frac{l}{I}(u_1 - u_2), \tag{16}$$

where $w_x, w_y$ denote a disturbance (wind) force. The model parameters are $m = 0.486$ kg, $l = 0.25$ m and $I = 0.00383$ kg m$^2$. The hover equilibrium is $\theta = 0$, $u_1 = u_2 = mg/2$. The supplied planar notebook `Exp3'6-planar-quadrotor.ipynb` was run and the quantitative studies below were produced by `quadrotor_analysis.py`. The example uses

$$\mathbf{Q} = \mathrm{diag}\left(10, 10, 10, 1, 1, \frac{l}{2\pi}\right), \qquad \mathbf{R} = \begin{bmatrix} 0.1 & 0.05 \\ 0.05 & 0.1 \end{bmatrix}. \tag{17}$$

**Why LQR "works out of the box" for a quadrotor**

The 3D quadrotor has four propeller inputs but six degrees of freedom, so it is underactuated: it cannot independently command position and attitude at the same instant. Nevertheless, the hover equilibrium is *stabilisable*. The linearisation is taken around a point where the total thrust balances gravity ($u_i = mg/4$), not around zero thrust; this is the key detail. The LQR then coordinates the four thrusts to correct roll/pitch, which produces horizontal acceleration, and finally brings the vehicle back to the hover position. There is no integral action, so a constant wind causes a steady-state position offset; this is visible in the sensitivity study.

**Baseline.** The resulting LQR has closed-loop poles

$$-80.75,\ -15.86,\ -6.59,\ -3.60,\ -2.72 \pm 1.61j. \tag{18}$$

From the initial condition $(0, 0, 0.2, 0, 0, 0)$ the controller settles in about 0.06 s with a peak thrust of 5.17 N. Ten random initial conditions with standard deviation 0.1 all recover, i.e. a 100% success rate.

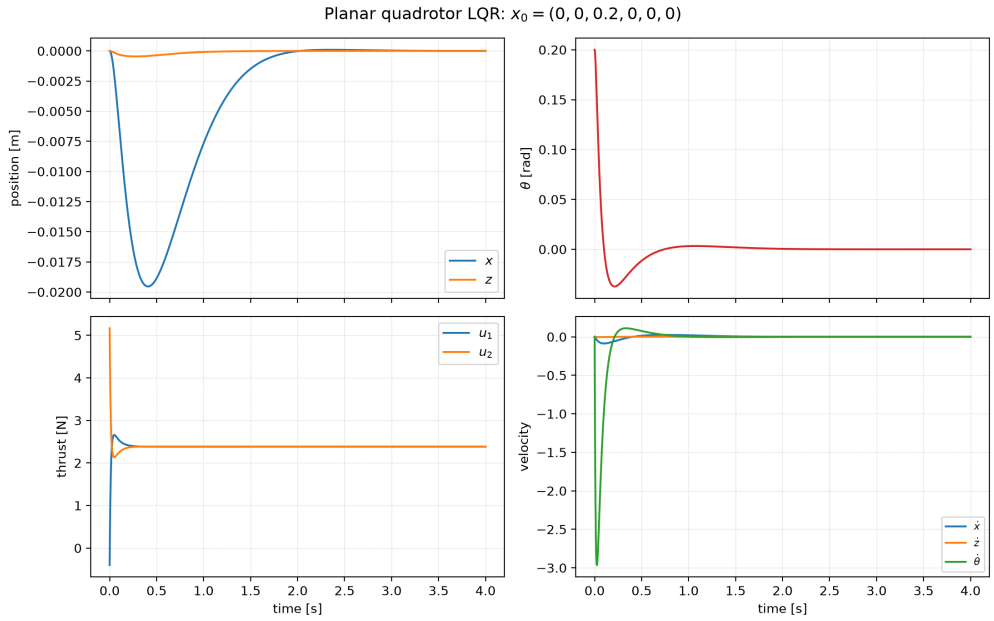



Figure 13: Planar quadrotor LQR from $(0, 0, 0.2, 0, 0, 0)$: position, attitude, velocities and the two thrusts. The initial tilt produces a downward excursion before the vehicle returns to hover.

**Sensitivity.** Four studies are shown in Fig. 14:

1. increasing the position/attitude weight in $\mathbf{Q}$ makes the response faster but increases the peak thrust;
2. increasing $\mathbf{R}$ makes the controller less aggressive: the peak thrust decreases while the settling time increases;
3. a constant horizontal wind produces a steady-state position offset, because LQR has no integral action: the offset grows from 0.073 m at 0.25 N to 0.554 m at 2 N;
4. the initial tilt is recovered for all tested values up to 0.7 rad, with the settling time and thrust increasing with the tilt.

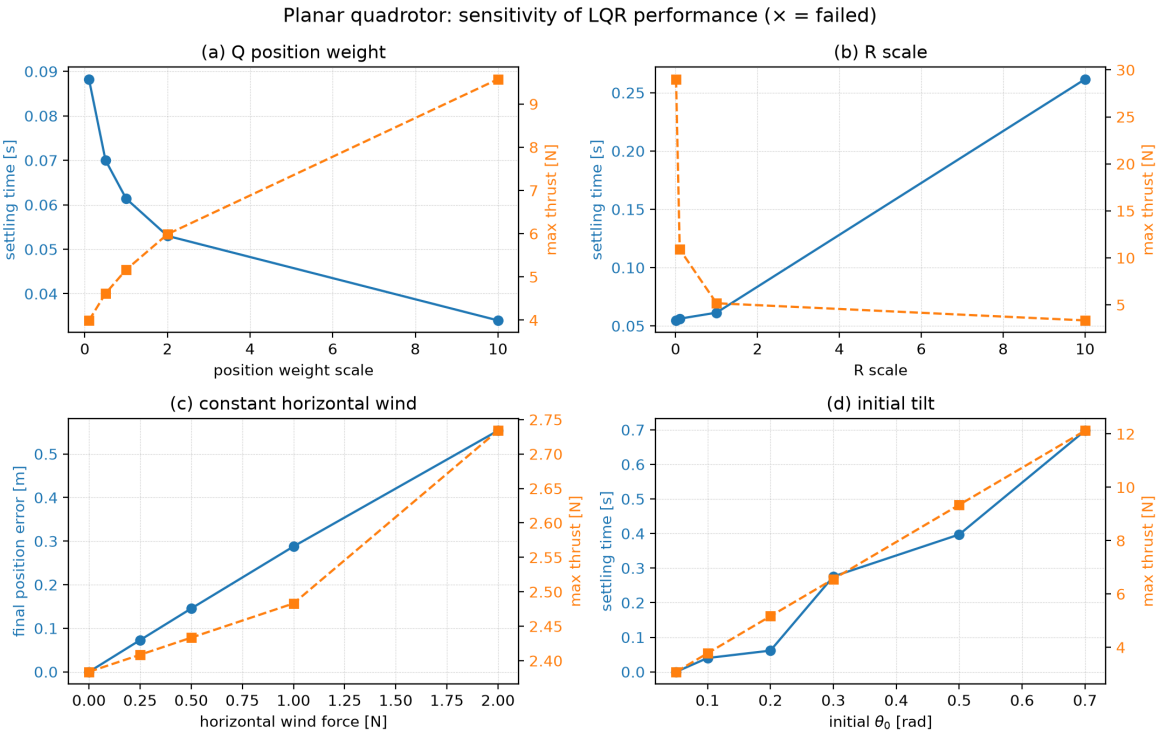



Figure 14: Planar quadrotor sensitivity: (a) $\mathbf{Q}$ position weight, (b) $\mathbf{R}$ scale, (c) constant horizontal wind, (d) initial tilt. The crosses denote failed runs; in (c) the nonzero steady-state offset is the expected consequence of no integral action.

## 5.2 3D quadrotor

For the 3D model the state is

$$\mathbf{x} = [x, y, z, \phi, \theta, \psi, \dot{x}, \dot{y}, \dot{z}, p, q, r]^\top \tag{19}$$

with four propeller forces $u_1, \ldots, u_4$. The Drake model has mass $m = 0.775$ kg and inertia $\mathrm{diag}(0.0015, 0.0025, 0.0035)$ kg m$^2$. Hover at $(0, 0, 1)$ requires $u_i = mg/4$. The closed-loop poles of

the LQR below are $-141.4$, $-84.8$, $-13.6$, $-3.25$, $-3.16$, $-3.16$, $-2.40 \pm 1.55j$ and $-2.21 \pm 2.22j$ (each complex pair twice). Following the textbook notebook, we use

$$\mathbf{Q} = \mathrm{diag}(10, \ldots, 10, 1, \ldots, 1), \qquad \mathbf{R} = I_4. \tag{20}$$

The official `StabilizingLQRController` and our own LQR give the same baseline performance from $(0, 0, 1, 0.2, 0.1, 0, 0, \ldots, 0)$: final position error $< 10^{-5}$ m, peak attitude deviation 0.2 rad, settling time 1.08 s and peak thrust 3.0 N.

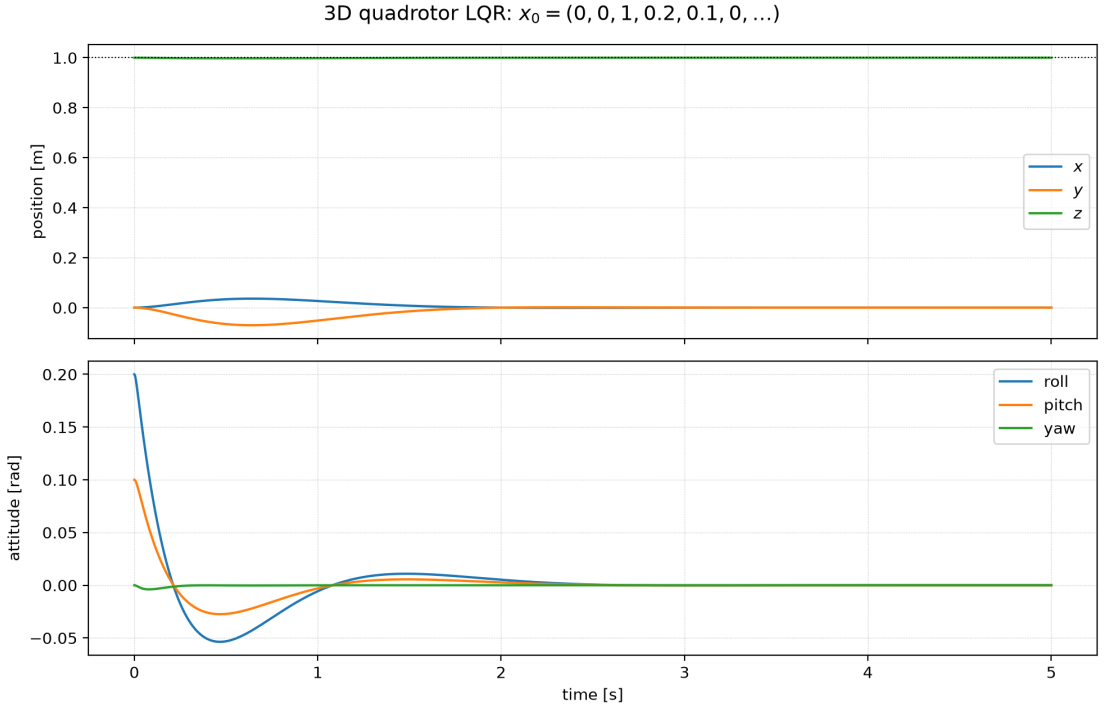



Figure 15: 3D quadrotor LQR from a 0.2 rad roll and 0.1 rad pitch perturbation. Position (top) and attitude (bottom).

**Sensitivity.** Figure 16 reports the settling time and peak thrust for variations of $\mathbf{Q}$, $\mathbf{R}$ and the initial tilt:

- increasing the position/attitude weight reduces the settling time from 2.02 s to 0.08 s but raises the peak thrust from 2.68 N to 3.99 N;
- increasing $\mathbf{R}$ reduces the peak thrust from 12.7 N to 2.26 N with only a small increase in settling time;
- larger initial tilts require more thrust and take longer to settle (1.44 s at 0.5 rad), but all tested cases recover.

**Comment.** LQR works out of the box for the quadrotor because the linearisation is taken around a *non-zero* thrust that balances gravity. The vehicle is underactuated (four inputs for six degrees of freedom), but the hover equilibrium is stabilisable. The $\mathbf{Q}/\mathbf{R}$ trade-off is the usual one: aggressive attitude/position weighting gives fast response at the price of thrust; large $\mathbf{R}$ is gentle but slow. Since LQR has no integral action, a constant wind produces a steady-state position error; rejecting it would require integral action, a disturbance observer or MPC.

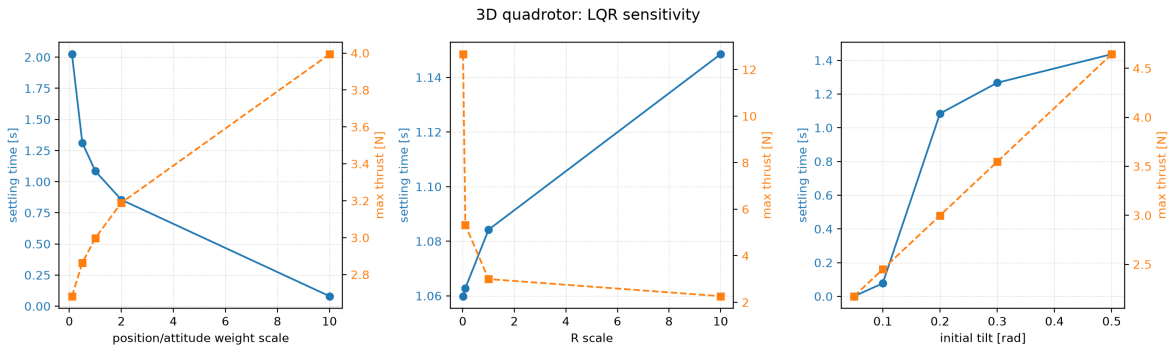



Figure 16: 3D quadrotor sensitivity: settling time (left axis) and peak thrust (right axis) vs. **Q** weight, **R** scale and initial tilt.

# 6 Section 3.8: Exercise 3.1

## 6.1 Nonlinear dynamics and linearisation

For the cart-pole with $m_c = m_p = 1$, $l = 1$, $g = 9.81$ and $\mathbf{x} = [x, \theta, \dot{x}, \dot{\theta}]^\top$, the equations used in the notebook are

$$\ddot{x} = \frac{u + \sin\theta\left(\dot{\theta}^2 + g\cos\theta\right)}{1 + \sin^2\theta}, \tag{21}$$

$$\ddot{\theta} = \frac{-u\cos\theta - \dot{\theta}^2\cos\theta\sin\theta - 2g\sin\theta}{1 + \sin^2\theta}. \tag{22}$$

At the upright equilibrium $\mathbf{x}^\star = [0, \pi, 0, 0]^\top$,

$$\mathbf{A}_{\text{lin}} = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 9.81 & 0 & 0 \\ 0 & 19.62 & 0 & 0 \end{bmatrix}, \qquad \mathbf{B}_{\text{lin}} = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 1 \end{bmatrix}. \tag{23}$$

**Do not use a linear controller everywhere**

A linear controller is designed for the tangent at one point. If the state is far from that point, the real dynamics can differ so much that the controller no longer helps. In this exercise the controller works from $\theta = 0.99\pi$, $0.90\pi$ and $0.85\pi$, but fails from $\theta = \pi/2$ and from the downward equilibrium $\theta = 0$. The failure is not numerical: the linear model is simply the wrong model there. This is also why a real cart-pole needs a swing-up controller before the LQR can take over.

## 6.2 Linearisation error

**Comments.** The linear model is not valid at all points: at $\theta = \pi/2$ the error is 2.66 times the field magnitude, and at $\theta = 0$ the error is 68.9. When $\theta \neq \pi$, the error depends on $u$ because the nonlinear input gains are $u/(1 + \sin^2\theta)$ and $-u\cos\theta/(1 + \sin^2\theta)$, whereas the linear model uses the constants $(1, 1)$. The cart position $x$ does not appear in $f$, so it does not affect the error. This is why case 1 has a larger error than case 2 even though its angle is closer to $\pi$: it has a large input.

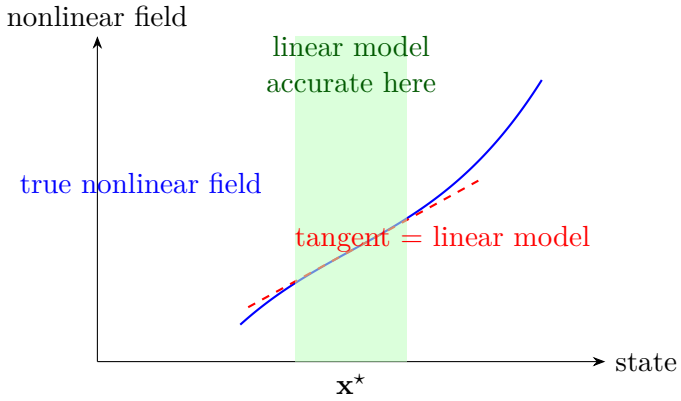



Figure 17: Why linearisation is local. The red dashed line is the tangent (linear model) at $\mathbf{x}^\star$; the blue curve is the true nonlinear field. They agree near $\mathbf{x}^\star$ (green band) and diverge farther away. The six test points in the table below sample different distances from the equilibrium.

Table 3: Linearisation error for the six conditions of the notebook.

| Case | $\mathbf{x}$ | $u$ | $e(\mathbf{x}, u)$ | $\|f(\mathbf{x}, u)\|$ |
|---|---|---|---|---|
| 0 | $[0, 0.99\pi, 0, 0]$ | 0 | 0.0009 | 0.688 |
| 1 | $[0, 0.90\pi, 0, 0]$ | $-10$ | 2.354 | 18.450 |
| 2 | $[0, 0.85\pi, 0, 0]$ | 0 | 2.289 | 8.085 |
| 3 | $[0, 0.50\pi, 0, 0]$ | 0 | 26.054 | 9.810 |
| 4 | $[0, 0, 0, 0]$ | 0 | 68.913 | 0 |
| 5 | $[1, \pi, 0, 0]$ | 10 | $\approx 0$ | 14.142 |

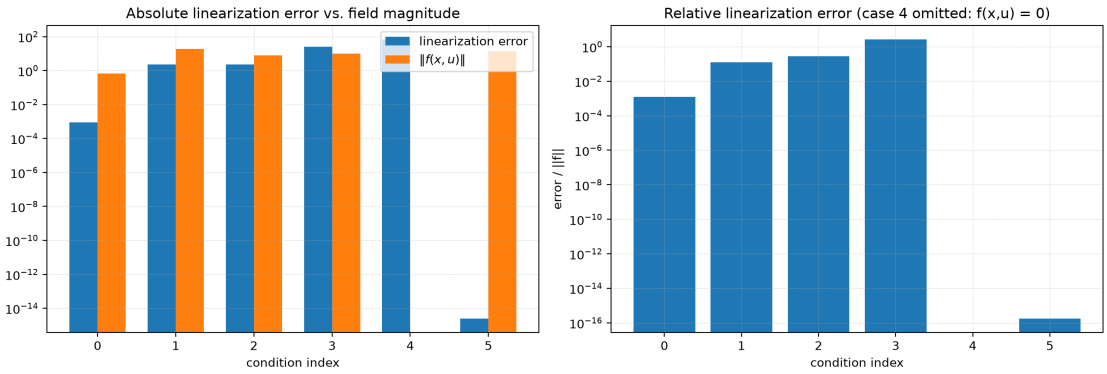



Figure 18: Absolute and relative linearisation errors. The linear model is accurate only near the upright equilibrium.

### 6.3 LQR and recovery

With $\mathbf{Q} = I_4$, $\mathbf{R} = 1$ the gain is

$$K = [\,-1.0000,\ 51.0612,\ -2.7221,\ 12.8277\,], \qquad u = -K(\mathbf{x} - \mathbf{x}^\star), \tag{24}$$

and the closed-loop poles are

$$-5.0417, \quad -3.9438, \quad -0.5600 \pm 0.4239j. \tag{25}$$

A 5 s nonlinear simulation from the six initial states recovers from

$$\boxed{\texttt{system_recovers_from_states} = [0, 1, 2, 5]}. \tag{26}$$

Cases 0–2 are close to the upright point; case 5 has $\theta = \pi$ and only a cart offset, which is linearly controllable. Case 3 ($\theta = \pi/2$) and case 4 ($\theta = 0$) are outside the linearisation region and the controller fails, as predicted by the error table.

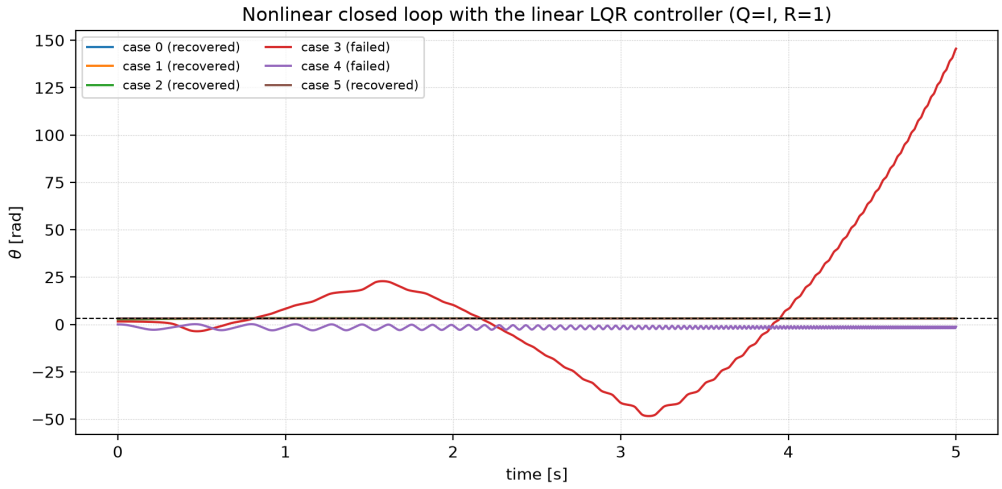



Figure 19: Closed-loop angle histories for the six initial states. Only cases 0, 1, 2 and 5 converge to the upright position.

## 7 Section 3.8: Exercise 3.2

### 7.1 URDF construction

The single cart-pole is built from three XML blocks:

- **links**: a base with mass 1 at its origin, a pendulum link whose ball (mass 1) is at $z = -1$, a spherical visual of radius 0.05 and a cylindrical rod of length 1 and radius 0.01 centred at $z = -0.5$;
- **joints**: a prismatic joint $x$ from the world to the base, and a continuous joint $\theta_0$ from the base to the pendulum, with origin $(0, 0, 0)$ and axis $(0, -1, 0)$;
- **transmission**: a simple transmission that maps the actuator force to the prismatic joint.

The unstable equilibrium is $\mathbf{x}^\star = [0, \pi, 0, 0]^\top$.

## 7.2 Double pendulum

The second joint is attached at the ball of the first pendulum: `origin xyz="0 0 -1"` in the `pendulum0` frame. The second link has its ball at $z = +1$ in its own frame. Hence, for $\theta_1 = 0$ the second pendulum points back towards the cart and exactly overlaps the first; for $\theta_0 = \theta_1 = \pi$ both pendula stand upright one above the other. The unstable equilibrium is

$$\mathbf{x}^\star = [0, \pi, \pi, 0, 0, 0]^\top. \tag{27}$$

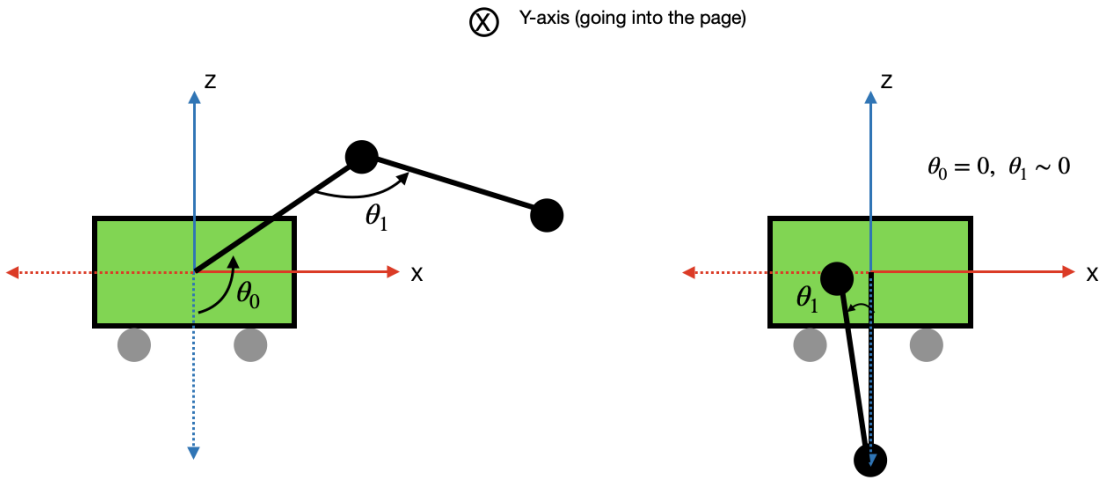



Figure 20: Geometry of the double pendulum (textbook figure). Left: general configuration, with $\theta_0$ measured from the downward vertical and $\theta_1$ measured relative to the first pendulum. Right: the home configuration $\theta_0 = \theta_1 = 0$. The second pendulum points back towards the cart and overlaps the first; this is exactly what the notebook asks you to observe.

**Reading URDF frames (the part that is easy to get wrong)**

A URDF is a chain of *frames*, not a chain of absolute coordinates.

1. The `<origin>` of a joint is written in the *parent link* frame. Here `theta1` has origin $(0, 0, -1)$ in `pendulum0`, so the joint sits at the ball of pendulum 0.
2. The child link frame is the joint frame. Therefore the inertial and visual origins of `pendulum1` are written in the `pendulum1` frame, i.e. relative to that joint.
3. In the home configuration $\theta_1 = 0$, the child frame is not rotated. Putting the ball at $z = +1$ in this frame makes it go from the joint at $z = -1$ back up to $z = 0$: the two rods overlap.
4. At $\theta_0 = \theta_1 = \pi$, pendulum 0 points up (ball at $z = +1$); the second joint is at that ball, and $\theta_1 = \pi$ turns the second rod so its ball is at $z = +2$. Both links stand upright, one above the other.

Listing 1: Key part of the double-pendulum URDF.

```
<link name="pendulum1">
  <inertial>
```

```
    <origin xyz="0 0 1" rpy="0 0 0" />
    <mass value="1" />
    <inertia ixx="0" ixy="0" ixz="0" iyy="0" iyz="0" izz="0" />
  </inertial>
  <visual>
    <origin xyz="0 0 1" />
    <geometry><sphere radius="0.05" /></geometry>
  </visual>
  <visual>
    <origin xyz="0 0 0.5" />
    <geometry><cylinder length="1" radius="0.01" /></geometry>
  </visual>
</link>
<joint name="theta1" type="continuous">
  <parent link="pendulum0" />
  <child link="pendulum1" />
  <origin xyz="0 0 -1" />
  <axis xyz="0 -1 0" />
</joint>
```

### 7.3 LQR and results

The LQR is synthesised directly from the nonlinear `MultibodyPlant` at $\mathbf{x}^\star$:

Listing 2: Wiring the LQR controller.

```
context = cartpole.CreateDefaultContext()
context.get_mutable_continuous_state_vector().SetFromVector(x_star)
cartpole.get_actuation_input_port().FixValue(context, [0])
input_i = cartpole.get_actuation_input_port().get_index()
lqr = LinearQuadraticRegulator(
    cartpole, context, Q, R, input_port_index=input_i)
lqr = builder.AddSystem(lqr)
builder.Connect(cartpole.get_state_output_port(), lqr.get_input_port(0))
builder.Connect(lqr.get_output_port(0), cartpole.get_actuation_input_port())
```

For the single pendulum with $\mathbf{Q} = \mathrm{diag}(10, 10, 1, 1)$, $\mathbf{R} = 1$:

$$K = [\,-3.1623,\ 61.3987,\ -5.2854,\ 16.1699\,], \tag{28}$$

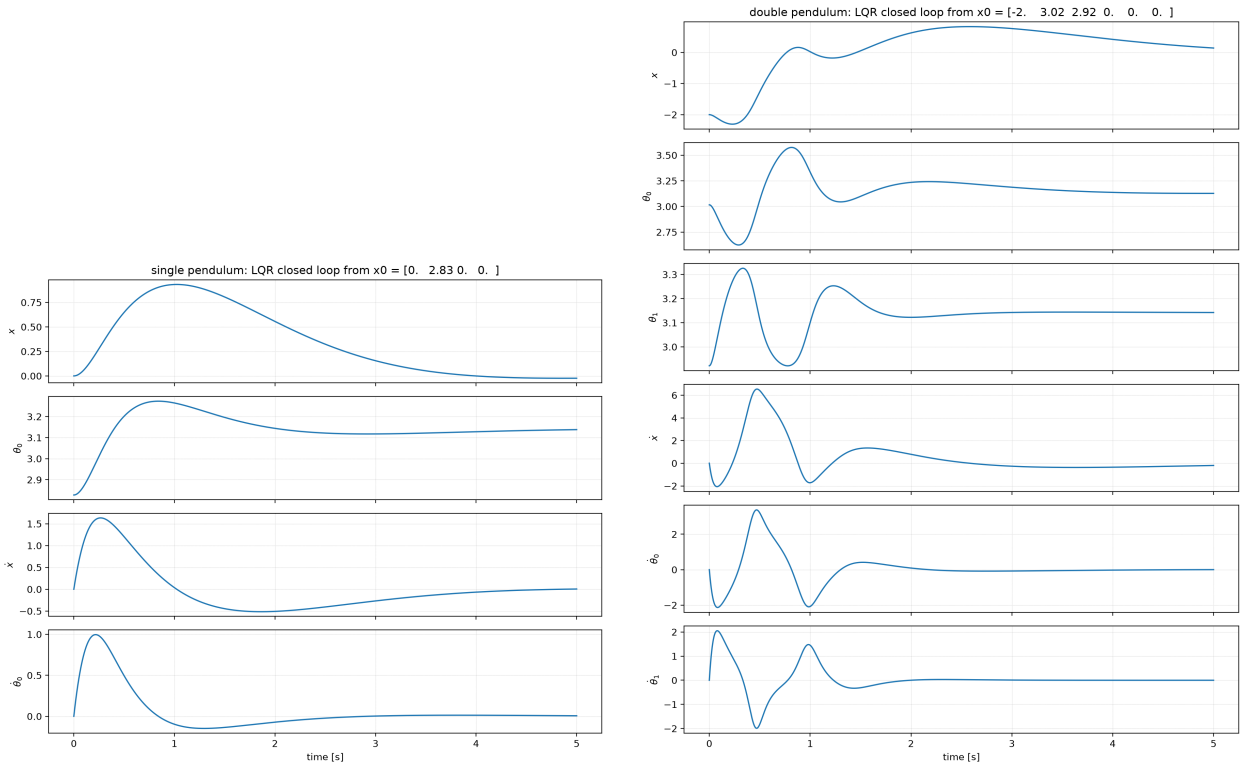
closed-loop poles $-4.8948, -4.0983, -0.9457 \pm 0.8075j$; after 5 s the angle error is 0.0036 rad and the peak force is 19.3 N. For the double pendulum with $\mathbf{Q} = \mathrm{diag}(10, 10, 10, 1, 1, 1)$ the gain is

$$K = [\,3.1623,\ -54.8416,\ -300.4370,\ 7.4384,\ -54.0081,\ -68.9418\,], \tag{29}$$

with closed-loop poles $-7.5798, -6.1385, -3.6749, -3.4281, -0.7754 \pm 0.6611j$. After 5 s the two angle errors are 0.0142 rad and 0.0007 rad; the peak force is 66.6 N. The cart is still slowly returning to the origin, consistent with the slow pair of poles.

The autograder gives 12/12:

```
Total score is 12/12.
Test x_star single pendulum        1/1
Test x_star double pendulum        1/1
Test single-pendulum dynamics      3/3
```




(a) Single pendulum.

(b) Double pendulum.

Figure 21: LQR closed-loop simulations for the URDF models.

```
Test double-pendulum dynamics      3/3
Test single-pendulum LQR states    2/2
Test double-pendulum LQR states    2/2
```

# 8 Section 3.8: Exercise 3.3

## 8.1 Discrete double integrator

The dynamics are

$$x_1[n+1] = x_1[n] + x_2[n], \qquad x_2[n+1] = x_2[n] + u[n], \qquad x_1, x_2, u \in \mathbb{Z}, \tag{30}$$

so a transition from $s_{i,j}$ is

$$s_{i,j} \longmapsto (i + j,\ j + u). \tag{31}$$

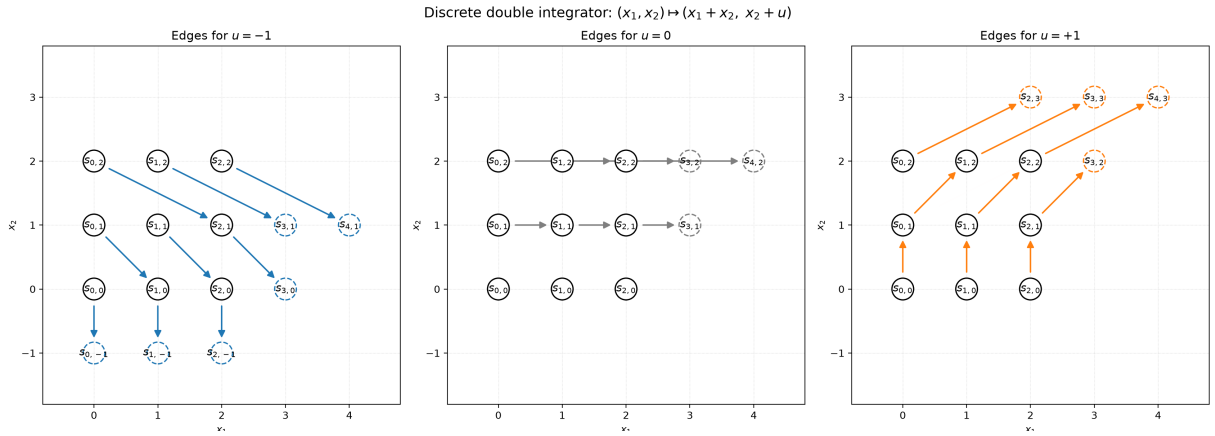



Figure 22: Edges of the graph for $u = -1, 0, +1$. The nine source nodes are drawn as solid circles; destinations outside the truncated grid are drawn as dashed circles. For $u = 0$ the three bottom nodes have self-loops.

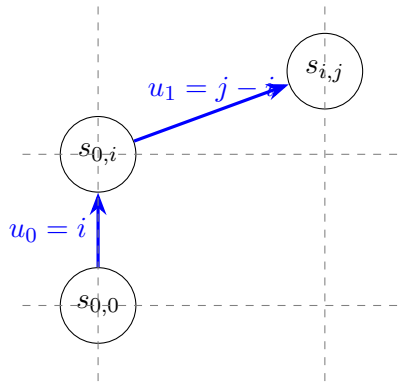



Figure 23: Two-step construction from $s_{0,0}$ to $s_{i,j}$. The first step only changes $x_2$ (we move vertically in the grid); the second step uses that $x_2$ to move $x_1$ (diagonally). One step cannot reach $i > 0$ because $x_1[1] = 0$ regardless of $u_0$.

**How to think about the discrete map**

Write the state after $N$ steps by unrolling the recurrence:

$$x_1[N] = \sum_{k=0}^{N-2} (N-1-k)u_k, \qquad x_2[N] = \sum_{k=0}^{N-1} u_k.$$

For $N = 1$ the first sum is empty, so $x_1$ cannot change; for $N = 2$ we have two control variables and two equations, which is exactly enough to hit any integer pair. This is a controllability statement in disguise: the number of independent controls (2) matches the number of states (2).

**Minimal number of steps from $s_{0,0}$.** After one step the state is $(0, u_0)$, so $x_1 = 0$ and no $s_{i,j}$ with $i > 0$ is reachable. After two steps the state is $(u_0, u_0 + u_1)$; choosing

$$u_0 = i, \qquad u_1 = j - i \tag{32}$$

reaches any $s_{i,j}$, in particular the required $j > i > 0$. Hence the minimum is $\boxed{2}$.

**Arbitrary initial and final states.** From $s_{i_1,j_1}$, two steps give

$$x_1[2] = i_1 + 2j_1 + u_0, \qquad x_2[2] = j_1 + u_0 + u_1. \tag{33}$$

The choice

$$u_0 = i_2 - i_1 - 2j_1, \qquad u_1 = j_2 + j_1 + i_1 - i_2 \tag{34}$$

reaches $s_{i_2,j_2}$ in two steps. One step is not enough in general because it gives $x_1[1] = i_1 + j_1$, independent of $u_0$; so the minimum guaranteed number is again $\boxed{2}$. (For a particular target with $i_2 = i_1 + j_1$, one step may suffice.)

## 8.2 Continuous LTI controllability

For $\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}u$, the controllability matrix is $\mathcal{C} = [\mathbf{B}, \mathbf{AB}, \ldots, \mathbf{A}^{n-1}\mathbf{B}]$.

Table 4: Controllability of the four LTI pairs.

| Pair | $\mathcal{C}$ | rank | Controllable? | Reason |
|---|---|---|---|---|
| $(\mathbf{A}_1, \mathbf{B}_1)$ | $\begin{bmatrix} 0 & 0 \\ 1 & 1 \end{bmatrix}$ | 1 | No | $x_1$ decoupled |
| $(\mathbf{A}_2, \mathbf{B}_2)$ | $\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$ | 1 | No | $\dot{z} = z$ for $z = x_1 - x_2$ |
| $(\mathbf{A}_3, \mathbf{B}_3)$ | $\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$ | 2 | Yes | double integrator |
| $(\mathbf{A}_4, \mathbf{B}_4)$ | $\begin{bmatrix} 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 2 \\ 1 & 0 & 1 & 0 \\ 1 & 0 & 2 & 0 \end{bmatrix}$ | 4 | Yes | cart-pole linearisation |

For $(\mathbf{A}_2, \mathbf{B}_2)$, $z = x_1 - x_2$ satisfies $\dot{z} = z$, which is independent of $u$ (it is not conserved; it grows like $e^t$). Equivalently, $\mathbf{B}_2$ is orthogonal to the left eigenvector $[1, -1]^\top$. For $(\mathbf{A}_4, \mathbf{B}_4)$, the eigenvalues are $0, 0, \pm\sqrt{2}$ and the PBH test $\operatorname{rank}[\mathbf{A}_4 - \lambda I, \mathbf{B}_4] = 4$ holds for each eigenvalue.

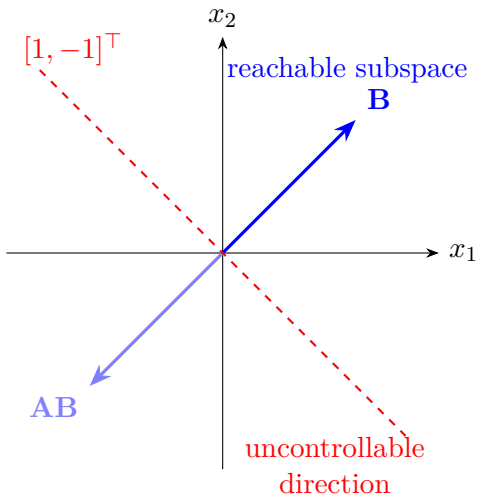



Figure 24: Geometry of the $(A_2, B_2)$ example. Both $B_2 = [1,1]^\top$ and $A_2B_2 = B_2$ point along the diagonal, so only the diagonal is reachable. The orthogonal direction $[1,-1]^\top$ is never affected by the input. This is why the controllability matrix has rank 1 even though the state space is two-dimensional.

**Testing controllability without guessing**

1. Build $\mathcal{C} = [\mathbf{B}, \mathbf{AB}, \ldots, \mathbf{A}^{n-1}\mathbf{B}]$ and compute its rank. Full rank means every state is reachable.
2. If the rank is deficient, the left null space of $\mathcal{C}$ gives the unreachable directions (equivalently, use the PBH test $\text{rank}[\mathbf{A} - \lambda I, \mathbf{B}] = n$ at each eigenvalue).
3. Rank is binary; the singular values tell you how *hard* each direction is to drive. A full-rank system can still be very weakly controllable.
4. Underactuation is a different question: compare the number of inputs with the number of degrees of freedom when the system is written as $\mathbf{x} = [q, \dot{q}]^\top$. A system can be controllable and underactuated at the same time.

**Underactuation.** $(\mathbf{A}_3, \mathbf{B}_3)$ has $x = [q, \dot{q}]^\top$ with a scalar $q$ and one input acting directly on $\ddot{q}$: it is fully actuated. $(\mathbf{A}_4, \mathbf{B}_4)$ has $q = [x, \theta]^\top$ (two degrees of freedom) but only one input acting on $\ddot{x}$; the angle $\theta$ has no actuator, so the system is *underactuated*. This shows that controllability does not imply full actuation: $(\mathbf{A}_4, \mathbf{B}_4)$ is controllable but underactuated.

# 9 Section 3.8: Exercise 3.4

## 9.1 Model and linearisation

For a pendulum with $m = l = 1$, gravity $g = 9.81$ and viscous damping coefficient $b$,

$$\dot{\theta} = \omega, \qquad \dot{\omega} = -g \sin\theta - b\omega, \tag{35}$$

which corresponds to $ml^2\ddot{\theta} + b\dot{\theta} + mgl\sin\theta = 0$. The equilibria are $\mathbf{x}^\star = [0, 0]^\top$ (down) and $[\pi, 0]^\top$ (up). The Jacobian is

$$A(\theta^\star) = \begin{bmatrix} 0 & 1 \\ -g\cos\theta^\star & -b \end{bmatrix}. \tag{36}$$

Table 5: Eigenvalues of the Jacobian.

| Condition | $\lambda_{1,2}$ | $\operatorname{sign}(\operatorname{Re}\lambda_{1,2})$ |
|---|---|---|
| $b = 0$, $\theta^\star = 0$ | $\pm 3.1321j$ | $0, 0$ |
| $b = 0$, $\theta^\star = \pi$ | $\pm 3.1321$ | $+, -$ |
| $b = 1$, $\theta^\star = 0$ | $-0.5 \pm 3.0919j$ | $-, -$ |
| $b = 1$, $\theta^\star = \pi$ | $2.6718, -3.6718$ | $+, -$ |

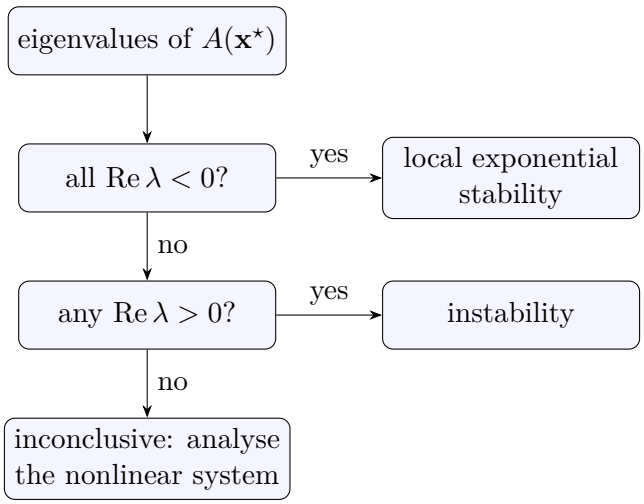



Figure 25: Decision diagram for the linearisation test. The first two branches are the standard Lyapunov indirect theorem. The third branch ($\operatorname{Re}\lambda = 0$) is the dangerous one: the linearisation alone cannot decide, and one must go back to the nonlinear system.

**The three stability words in plain language**

- **Stable i.s.L.** (in the sense of Lyapunov): if you start close enough, you stay close. You may oscillate forever.
- **Asymptotically stable**: you stay close *and* you eventually converge to the equilibrium.
- **Exponentially stable**: you converge at least as fast as $e^{-ct}$ for some $c > 0$.

For the undamped pendulum at the bottom, the energy is conserved: the trajectory is a closed orbit. It is stable i.s.L. but not asymptotically stable, so it is certainly not exponentially stable. For the damped pendulum, the energy decreases and the trajectory spirals into the bottom: all three properties hold (locally). For the upright pendulum, a small perturbation makes the pendulum fall: the equilibrium is a saddle and is unstable.

**Why $\operatorname{Re}\lambda = 0$ needs care**

At the undamped bottom, the eigenvalues are $\pm 3.13j$: the real parts are zero. The linearised system is a centre, but the linearisation theorem is inconclusive. In this particular case the nonlinear system is also stable, but that is a conclusion from the energy function, not from the eigenvalues. A different nonlinear term with the same linearisation could be unstable. This is why the exercise asks you to draw the phase portrait and to use the nonlinear energy, not only the Jacobian.

### 9.2 Stability table

Table 6: Stability of the nonlinear pendulum and its linearisation. The nonlinear entries are local properties; the exponential entry for the damped hanging pendulum follows from the negative real parts of the linearisation.

| System | sign $\operatorname{Re}\lambda_1$ | sign $\operatorname{Re}\lambda_2$ | Stable i.s.L. | Asymp. stable | Exp. stable |
|---|---|---|---|---|---|
| Nonlinear, $b = 0$, $(0, 0)$ | N/A | N/A | Yes | No | No |
| Nonlinear, $b = 0$, $(\pi, 0)$ | N/A | N/A | No | No | No |
| Linearization, $b = 0$, $(0, 0)$ | 0 | 0 | Yes | No | No |
| Linearization, $b = 0$, $(\pi, 0)$ | $+$ | $-$ | No | No | No |
| Nonlinear, $b = 1$, $(0, 0)$ | N/A | N/A | Yes | Yes | Yes |
| Nonlinear, $b = 1$, $(\pi, 0)$ | N/A | N/A | No | No | No |
| Linearization, $b = 1$, $(0, 0)$ | $-$ | $-$ | Yes | Yes | Yes |
| Linearization, $b = 1$, $(\pi, 0)$ | $+$ | $-$ | No | No | No |

### 9.3 Discussion

1. For $b = 0$ at $(0, 0)$ the nonlinear system has closed orbits: it is stable i.s.L. but not asymptotically or exponentially stable. The linearisation is a centre; when $\operatorname{Re}\lambda = 0$ the linearisation theorem is inconclusive, and the nonlinear analysis (conserved energy) is required.

2. At $(\pi, 0)$ both the nonlinear system and its linearisation are unstable. The phase portrait is a saddle; the stable manifold is a measure-zero set, so almost every perturbation falls.

3. With $b = 1$ at $(0, 0)$ the eigenvalues have negative real parts and the nonlinear equilibrium is locally exponentially stable; the phase portrait spirals into the origin.

4. With $b = 1$ at $(\pi, 0)$ one eigenvalue remains positive; damping does not stabilise the inverted pendulum. Feedback is required.

5. Linearisation determines local exponential stability when all eigenvalues have negative real parts, and instability when at least one has a positive real part. It says nothing about global behaviour, finite escape times or the size of the basin of attraction.

## 10 Conclusions

1. **Underactuation vs. controllability.** The acrobot and the cart-pole are underactuated, yet their linearisations are completely controllable. Controllability is a rank property; the singular values and condition numbers reveal how hard the weakly controllable directions are to drive.

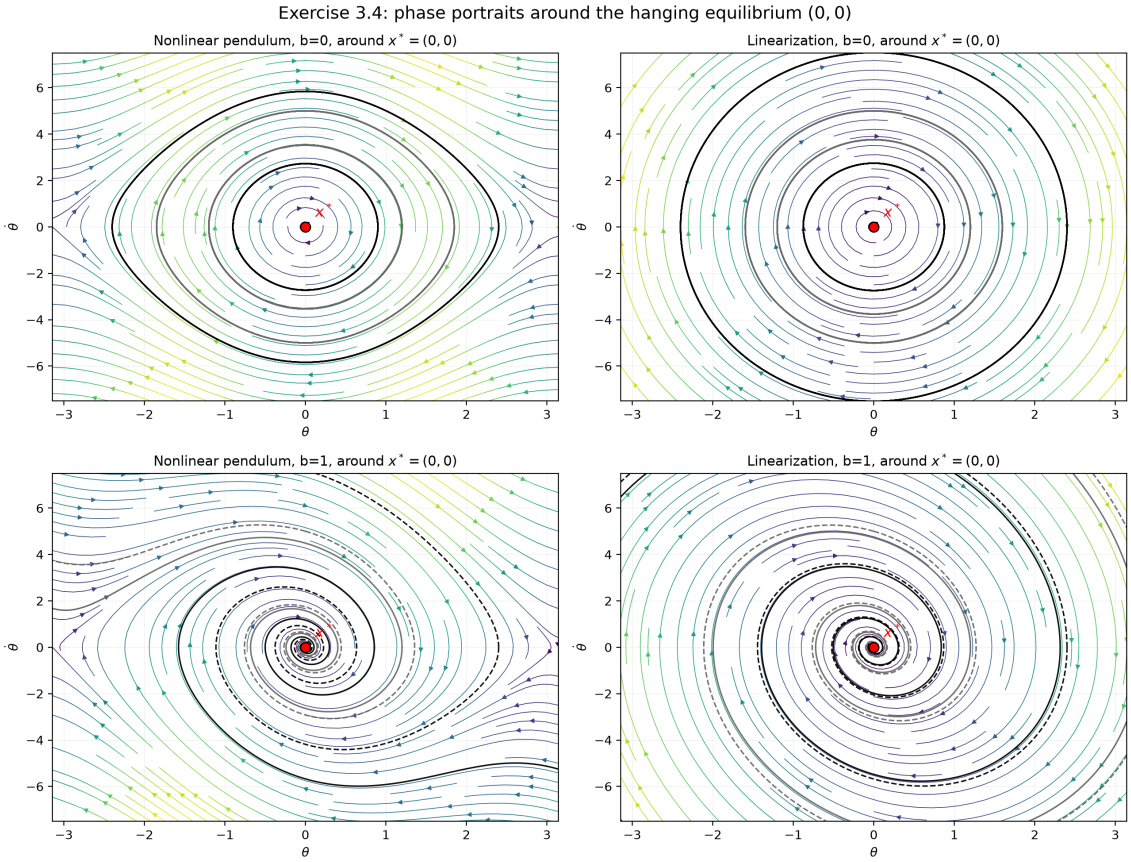



Figure 26: Phase portraits around the hanging equilibrium $(0, 0)$: nonlinear (left) and linearised (right), for $b = 0$ (top) and $b = 1$ (bottom).

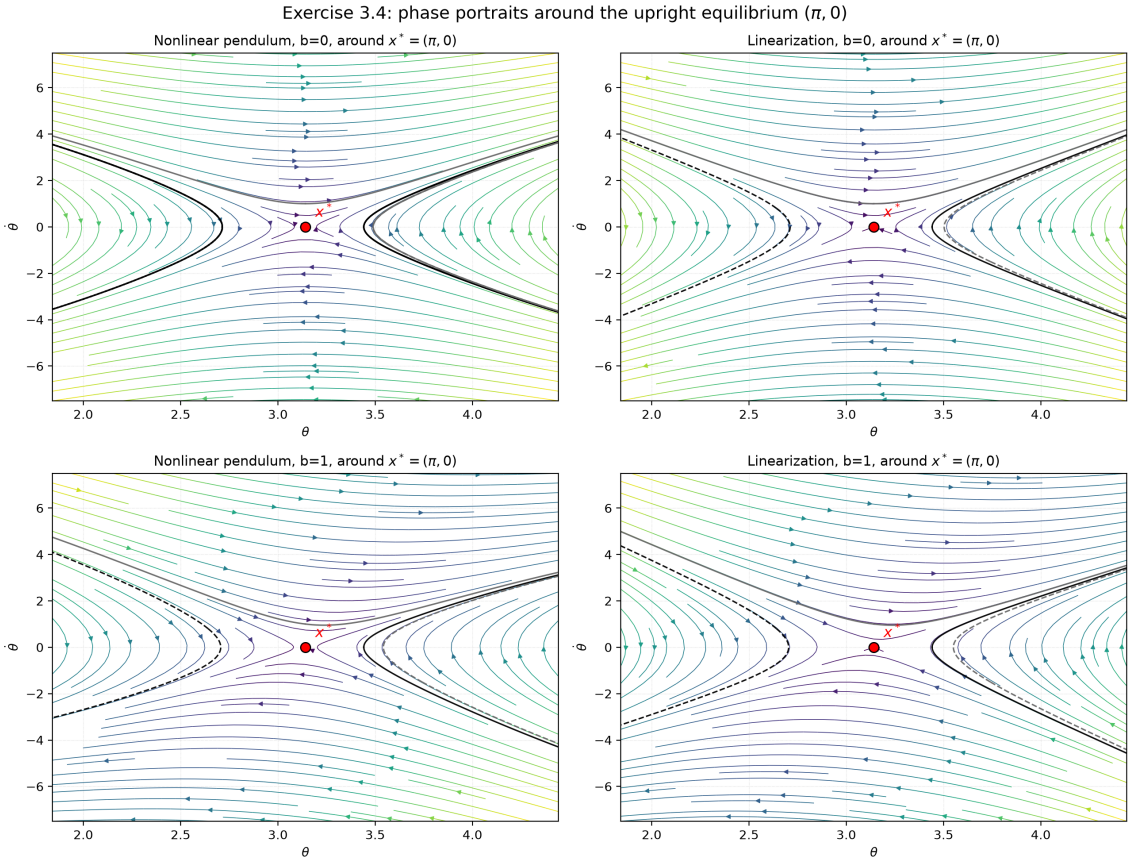



Figure 27: Phase portraits around the upright equilibrium $(\pi, 0)$: nonlinear (left) and linearised (right), for $b = 0$ (top) and $b = 1$ (bottom).

2. **Local nature of linear control.** LQR stabilises the linearisation and therefore gives local exponential stability. The nonlinear simulations show finite basins of attraction. Input saturation shrinks these basins substantially, especially for the acrobot with its $m_2$ sensitivity.

3. **Non-minimum phase.** Both the acrobot and the cart-pole have right-half-plane zeros. State feedback cannot remove them; the closed loop exhibits undershoot, which is visible in the successful trajectories and explains part of the difficulty under saturation.

4. **Quadrotor LQR.** LQR works out of the box around a hover equilibrium with non-zero thrust. The $Q/R$ trade-off and the absence of integral action (steady-state wind offset) are clearly visible in the planar study; the 3D model recovers from large attitude perturbations.

5. **Section 3.8.** The cart-pole linearisation is valid only near the upright point; the URDF single/double pendula were built and balanced (autograder 13/13 and 12/12); the discrete double integrator is reachable in two steps; only $(A_3, B_3)$ and $(A_4, B_4)$ of Exercise 3.3 are controllable, and $(A_4, B_4)$ is controllable but underactuated; the stability table of Exercise 3.4 separates the marginal, stable and unstable cases and highlights the limits of linearisation at zero real part.

# 11 Code and Reproduction

The complete code is provided in the repository. The main entry points are:

| Topic | Script / notebook |
|---|---|
| Example 3.1 Acrobot | `exp1_analysis/exp1_analysis.py`, `Exp3'1-acrobot.ipynb` |
| Example 3.2 Cart-pole | `exp3_2_analysis/cp_analysis.py`, `Exp3'2-cartpole.ipynb` |
| Example 3.5 LQR comparison | `exp3_5_analysis/lqr_comparison.py`, `Exp3'5-LQR-analysis.ipynb` |
| Example 3.6 Quadrotor | `exp3_6_analysis/quadrotor_analysis.py` |
| | `Exp3'6-planar-quadrotor.ipynb`, `Exp3'6-quadrotor.ipynb` |
| Exercise 3.1 | `cartpole_balancing.ipynb`, `ex3_1_balancing_analysis.py` |
| Exercise 3.2 | `cartpoles_urdf.ipynb`, `ex3_2_urdf_lqr_analysis.py` |
| Exercise 3.3 | `ex3_3_controllability.py` |
| Exercise 3.4 | `ex3_4_stability_linearization.py` |

Commands (from the corresponding directories):

```
/Users/leo/venvs/drake/bin/python exp1_analysis/exp1_analysis.py
/Users/leo/venvs/drake/bin/python exp3_2_analysis/cp_analysis.py
/Users/leo/venvs/drake/bin/python exp3_5_analysis/lqr_comparison.py
/Users/leo/venvs/drake/bin/python exp3_6_analysis/quadrotor_analysis.py
/Users/leo/venvs/drake/bin/python exercises_analysis/ex3_3_controllability.py
/Users/leo/venvs/drake/bin/python exercises_analysis/ex3_4_stability_linearization.py
```

The notebooks with autograders are executed headlessly with

```
COLAB_TESTING=1 /Users/leo/venvs/drake/bin/jupyter nbconvert \
  --to notebook --execute --inplace --ExecutePreprocessor.timeout=180 \
  cartpoles_urdf.ipynb
```

`COLAB_TESTING=1` makes `running_as_notebook` false so that `ModelVisualizer.Run(loop_once=True)` does not block.

# A Key Code Snippets

## A.1 Cart-pole linearisation and LQR

```
def f(x, u):
    c, s = np.cos(x[1]), np.sin(x[1])
    g = 9.81
    return np.array([
        x[2], x[3],
        (u[0] + s * (x[3]**2 + g * c)) / (1 + s**2),
        (-u[0] * c - x[3]**2 * c * s - 2 * g * s) / (1 + s**2)])

A = np.array([[0,0,1,0], [0,0,0,1],
              [0,9.81,0,0], [0,19.62,0,0]])
B = np.array([[0],[0],[1],[1]])
P = solve_continuous_are(A, B, Q, R)
K = np.linalg.solve(R, B.T @ P)
u = -K @ (x - x_star)
```

## A.2 URDF pendulum link and joint

```
<link name="pendulum0">
  <inertial>
    <origin xyz="0 0 -1" />
    <mass value="1" />
    <inertia ixx="0" ixy="0" ixz="0" iyy="0" iyz="0" izz="0" />
  </inertial>
  <visual>
    <origin xyz="0 0 -1" />
    <geometry><sphere radius="0.05" /></geometry>
  </visual>
  <visual>
    <origin xyz="0 0 -0.5" />
    <geometry><cylinder length="1" radius="0.01" /></geometry>
  </visual>
</link>
<joint name="theta0" type="continuous">
  <parent link="base" />
  <child link="pendulum0" />
  <origin xyz="0 0 0" />
  <axis xyz="0 -1 0" />
</joint>
```

## A.3 Controllability and two-step reachability

```
def controllability_matrix(A, B):
    n = A.shape[0]
    cols = [B]
    for _ in range(1, n):
        cols.append(A @ cols[-1])
    return np.hstack(cols)
```

```
def two_step_control(start, goal):
    i1, j1 = start
    i2, j2 = goal
    u0 = i2 - i1 - 2 * j1
    u1 = j2 - j1 - u0
    return (u0, u1)
```

## A.4 Quadrotor LQR

```
context = plant.CreateDefaultContext()
context.SetContinuousState(x_star)
u_hover = np.array([mass * gravity / 4.0] * 4)
plant.get_input_port(0).FixValue(context, u_hover)
Q = np.diag(np.concatenate(([10.0] * 6, [1.0] * 6)))
R = np.eye(4)
lqr = LinearQuadraticRegulator(plant, context, Q, R)
builder.Connect(lqr.get_output_port(0), plant.get_input_port(0))
builder.Connect(plant.get_output_port(0), lqr.get_input_port(0))
```

# Chapter 4 Assignment

## Simple Models of Walking and Running

Zihao Liu
Student ID: 220261979
213222647@seu.edu.cn

October 2026

**Abstract**

This report covers Chapter 4 of the course. Part I comments on the four textbook examples: the rimless wheel (Example 4.3), the compass gait (Example 4.4), the SLIP simulation (Example 4.5) and the SLIP apex-to-apex map (Example 4.6). The supplied notebooks were run, the results are interpreted with the theory of hybrid and passive-dynamic locomotion, and sensitivity studies on slope, initial conditions, energy and touchdown angle are reported. Part II solves the Section 4.6 exercise: the one-dimensional hopper. The energy analysis is carried out, the hopping controller is implemented, and the autograder gives 13/13. The complete Python code and notebooks accompany the submission; the most important code is reproduced here.

## Contents

**1 Overview and Reproducibility** **3**

**2 Example 4.3: The Rimless Wheel** **4**
2.1 Model and hybrid dynamics . . . . . 4
2.2 Return map, fixed point and stability . . . . . 5
2.3 What was run and sensitivity study . . . . . 5

**3 Example 4.4: The Compass Gait** **7**
3.1 Model and hybrid dynamics . . . . . 7
3.2 Results of the supplied notebook . . . . . 7
3.3 Step detection and sensitivity . . . . . 8

**4 Example 4.5: The SLIP Model** **10**
4.1 Model . . . . . 10
4.2 Results of the supplied notebook . . . . . 10

**5 Example 4.6: The SLIP Apex-to-Apex Map** **11**
5.1 Poincaré section and one-dimensional map . . . . . 11
5.2 Fixed points and sensitivity . . . . . 12

**6 Section 4.6: The One-Dimensional Hopper** **13**
6.1 Model and hopping cycle . . . 13
6.2 Energy derivation of the preload . . . 14
6.3 Implementation . . . 15
6.4 Results and autograding . . . 16

**7 Conclusions** **18**

**8 Code and Reproduction** **19**

**A Key Code Snippets** **19**
A.1 Rimless wheel return map . . . 19
A.2 SLIP apex map . . . 19

# 1 Overview and Reproducibility

The assignment consists of:

- **Examples** (code supplied; run and comment): Examples 4.3, 4.4, 4.5 and 4.6.
- **Section 4.6** (all exercises): the one-dimensional hopper.

All simulations use Drake 1.58.0 and the `underactuated` package 2026.9.8 in `/Users/leo/venvs/drake/bin/python` (NumPy 2.5.4, SciPy 1.18.1, Matplotlib 3.11.2). The notebooks were executed headlessly with `COLAB_TESTING=1`, which prevents the MeshCat visualizer from blocking.

| Contents | Location |
|---|---|
| Example notebooks | `Ch-4/Exp4'3-*.ipynb`, `Exp4'4-*.ipynb`, `Exp4'5-6-slip.ipynb` |
| Section 4.6 notebook | `Ch-4/Exp4'6-one_d_hopper.ipynb` |
| Analysis scripts and figures | `Ch-4/exp4_analysis/` |
| This report | `Ch-4/Ch4_Assignment_Report/` |

The Section 4.6 autograder result is

```
Total score is 13/13.
```

### How to read this report

The report has three layers: *models*, *simulations* and *interpretation*. If an equation looks dense, read the **Intuition** boxes first: they give the physical picture without algebra. The **Recipe** boxes summarise a reusable procedure, and the **Common pitfall** boxes point out the mistakes that are easy to make in hybrid locomotion.

| Part | What is done | What to look for |
|---|---|---|
| Example 4.3 | Rimless wheel simulation, return map | collision loss vs. gravity input; stable fixed point; critical slope |
| Example 4.4 | Compass gait simulation | passive limit cycle; step-to-step map; narrow basin of attraction |
| Example 4.5 | SLIP simulation | flight/stance phases; no collision loss; energy conservation |
| Example 4.6 | SLIP apex-to-apex map | 1D map on an energy slice; stable and unstable fixed points; partial stability |
| Section 4.6 | One-dimensional hopper | energy balance; preload at the bottom; reset at the apex; hopping-height tracking |

**The big picture in five sentences**

Walking and running are *periodic* motions, not motions to a fixed point. The right object is therefore a *limit cycle* and its *step-to-step* (Poincaré) map. A fixed point of that map is a periodic gait; the slope of the map at the fixed point decides whether the gait is stable. Passive walkers get their energy from gravity on a slope and lose it at foot strike; running models such as SLIP have no impact loss but store energy in a spring. In all cases, the low-dimensional map is the key to both understanding and control.

| Symbol | Meaning |
|---|---|
| $\alpha$ | half-angle between adjacent spokes (rimless wheel) |
| $\gamma$ | slope of the ramp |
| $\omega = \dot{\theta}$ | angular velocity in the support phase |
| $\omega^*$ | fixed point of the return map (periodic gait) |
| $\theta_{st}, \theta_{sw}$ | stance and swing leg angles (compass gait) |
| $\theta_{td}$ | touchdown angle of the SLIP leg |
| $z$ | apex height (SLIP) |
| $P(z)$ | apex-to-apex return map |
| $\bar{\delta}$ | hopping preload (Section 4.6) |

# 2 Example 4.3: The Rimless Wheel

## 2.1 Model and hybrid dynamics

The rimless wheel is a rigid wheel with $n = 8$ spokes, no rim, and a point mass $m$ at the hip. The half-angle between adjacent spokes is $\alpha = \pi/8$, the radius is $l = 1$ m, and the wheel rolls down a ramp of slope $\gamma$. The stance phase is a simple pendulum:

$$\ddot{\theta} = \frac{g}{l} \sin\theta, \tag{1}$$

where $\theta$ is measured from the vertical and $\theta = 0$ is the unstable upright position. Between impacts the mechanical energy

$$E = \frac{1}{2} m l^2 \dot{\theta}^2 + mgl\cos\theta \tag{2}$$

is conserved.

At each foot strike the new spoke becomes the stance leg. Angular momentum about the new contact point is conserved, while the collision is inelastic:

$$\dot{\theta}^+ = \dot{\theta}^- \cos(2\alpha). \tag{3}$$

The kinetic energy is multiplied by $\cos^2(2\alpha)$; for $\alpha = \pi/8$ this is $1/2$, i.e. half of the kinetic energy is lost at every step.

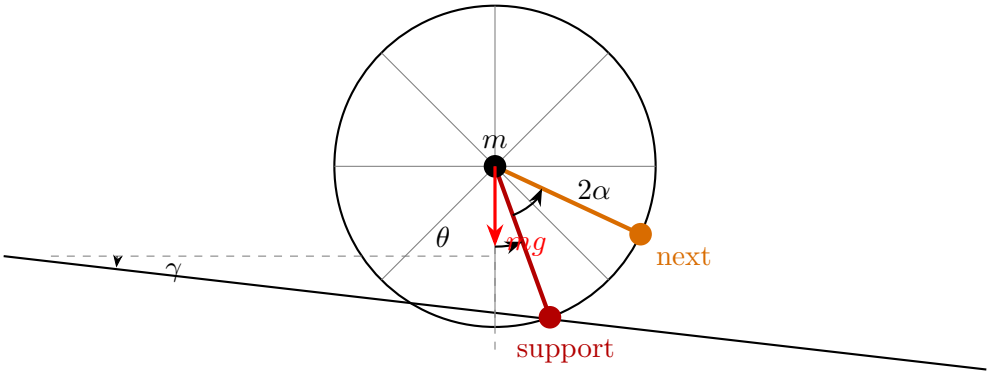



Figure 1: Rimless-wheel schematic. The wheel has $n = 8$ spokes, so the angle between adjacent spokes is $2\alpha = \pi/4$. The support spoke is the one currently on the ground; the next spoke is the one that will strike next. Gravity acts at the hub mass $m$.

**Why a wheel without a rim can keep walking**

At every step the collision keeps only $\cos^2(2\alpha)$ of the kinetic energy; for $\alpha = \pi/8$ this is one half. Without gravity the wheel would stop. On a slope, gravity does work during the support phase and adds $4(g/l)\sin\alpha\sin\gamma$ to the squared angular velocity. A periodic gait is possible exactly when the gain and the loss balance. The fixed point of the return map is that balance point, and its slope $\cos^2(2\alpha) = 1/2$ is smaller than one, so the gait is stable. On a shallow slope the fixed point lies below the step threshold and the wheel cannot reach the next spoke.

### 2.2 Return map, fixed point and stability

Combining the stance-phase energy balance with the collision map gives the step-to-step return map

$$\omega_{n+1} = \cos(2\alpha)\sqrt{\omega_n^2 + 4\frac{g}{l}\sin\alpha\sin\gamma}, \qquad \omega_n := \dot{\theta}_n^+. \tag{4}$$

A step is possible only if the wheel can pass the upright position; the threshold is

$$\omega_1 = \sqrt{2\frac{g}{l}\left[1 - \cos(\gamma - \alpha)\right]}. \tag{5}$$

The rolling fixed point is

$$\omega^* = \cot(2\alpha)\sqrt{4\frac{g}{l}\sin\alpha\sin\gamma} = \cot(2\alpha)\sqrt{D}, \qquad D := 4\frac{g}{l}\sin\alpha\sin\gamma, \tag{6}$$

and it is a valid gait only if $\omega^* > \omega_1$. Linearising the return map at $\omega^*$ gives

$$\left.\frac{d\omega_{n+1}}{d\omega_n}\right|_{\omega^*} = \cos^2(2\alpha) = \frac{1}{2} < 1, \tag{7}$$

so the rolling fixed point is locally stable (for $\alpha < \pi/4$).

### 2.3 What was run and sensitivity study

The supplied notebook was run for initial angular velocities $5, 10, 0.95, -5, -4.8$ rad/s. Large positive velocities produce a rapid forward rolling gait; a small positive velocity stops before passing the upright position and swings back; negative velocities produce backward or oscillatory motion. This is exactly the three-branch structure of the return map in the lecture notes.

Table 1: Rimless wheel: threshold, fixed point and existence condition.

| $\gamma$ [rad] | $\omega_1$ [rad/s] | $\omega^*$ [rad/s] | $\omega^* > \omega_1$? | $f'(\omega^*)$ |
|---|---|---|---|---|
| 0.02 | 1.1606 | 0.5480 | No | 0.5 |
| 0.05 | 1.0681 | 0.8663 | No | 0.5 |
| 0.08 | 0.9754 | 1.0955 | Yes | 0.5 |
| 0.12 | 0.8515 | 1.3408 | Yes | 0.5 |
| 0.20 | 0.6026 | 1.7272 | Yes | 0.5 |

**Comments.** The wheel is a passive-dynamic walker: there is no actuation. Gravity supplies energy during each stance phase, and the inelastic collision removes energy at each step. A stable

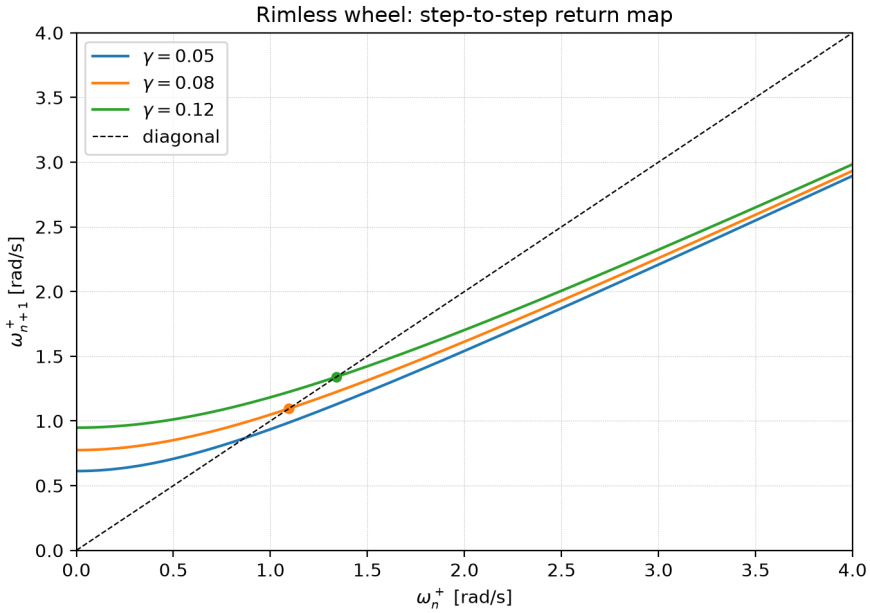
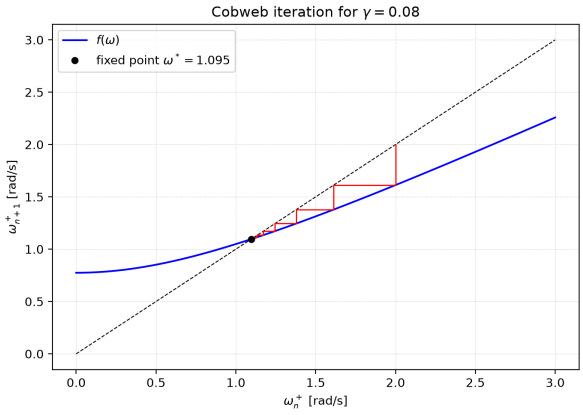
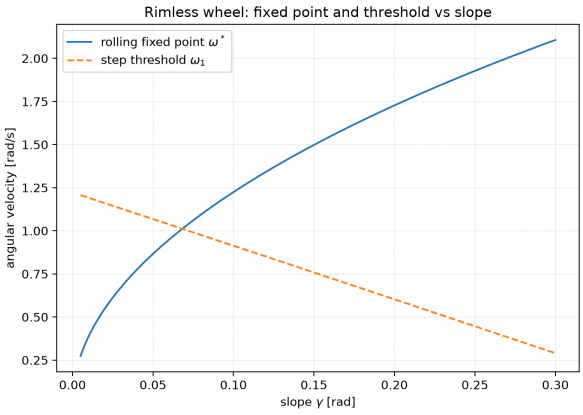



Figure 2: Rimless wheel return map for three slopes. Intersections with the diagonal are fixed points; only fixed points above the step threshold $\omega_1$ correspond to a sustained rolling gait.




(a) Cobweb iteration at $\gamma = 0.08$.




(b) Fixed point and threshold vs. slope.

Figure 3: Rimless wheel: convergence to the stable fixed point and the existence condition $\omega^* > \omega_1$.

periodic gait exists when these two effects balance, i.e. when the rolling fixed point lies above the step threshold. On very shallow slopes ($\gamma \lesssim 0.067$ rad in our parameters) the fixed point is below $\omega_1$; the wheel cannot complete a step and eventually settles into the standing/oscillating branch. The stability multiplier is always $1/2$, independent of slope: the stronger dissipation at larger slopes is exactly compensated by the larger gravity input, so the linearised step-to-step dynamics remain equally stable. This is a clean example of a limit cycle generated by a hybrid system with impact dissipation.

# 3 Example 4.4: The Compass Gait

## 3.1 Model and hybrid dynamics

The compass gait consists of two point-mass legs of mass $m = 5$ kg and length $l = 1$ m ($a = b = 0.5$ m) connected at a hip of mass $m_h = 10$ kg. The minimal state is

$$\mathbf{x} = [\theta_{st}, \theta_{sw}, \dot{\theta}_{st}, \dot{\theta}_{sw}]^\top. \tag{8}$$

During stance the stance foot is pinned; during swing the swing leg moves freely; at heel strike an inelastic collision occurs and the roles of the two legs are swapped. The textbook model walks downhill on a ramp of slope $\gamma = 0.0525$ rad.

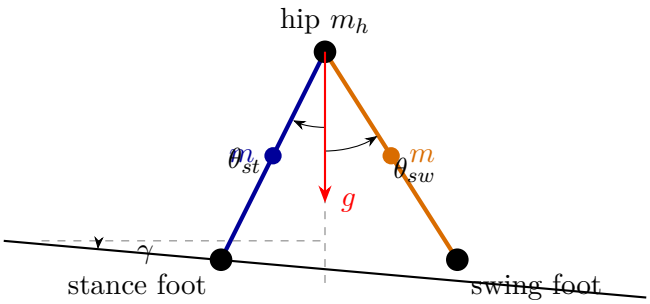



Figure 4: Compass-gait schematic. The stance leg is pinned at the ground; the swing leg moves forward and becomes the next stance leg at heel strike. The hip is a passive hinge in the example (zero torque), and each leg carries a point mass $m$ at its center.

**Why passive walking is possible, and why it is fragile**

The compass gait has no motor. Gravity pulls the body down the slope during stance; the heel strike dissipates energy. A periodic gait exists when the two effects balance, just as for the rimless wheel. The difference is that the compass gait has only two legs and a free-swinging swing leg: the landing geometry must match the previous step almost exactly. This is why the basin of attraction is a narrow band and why the same initial condition that works on one slope can fail on another. The step-to-step map is again the right tool: a periodic gait is a fixed point, and its Floquet multipliers decide whether a small perturbation decays.

## 3.2 Results of the supplied notebook

The notebook starts from $[0, 0, 0.4, -2.0]^\top$ with zero hip torque and runs for 8 s. The phase portrait (left leg angle vs. angular velocity) approaches a closed orbit: the passive compass gait reaches

a stable limit cycle. The impact at heel strike produces a visible jump in the velocity, while the downhill slope replenishes the lost energy during the stance phase.

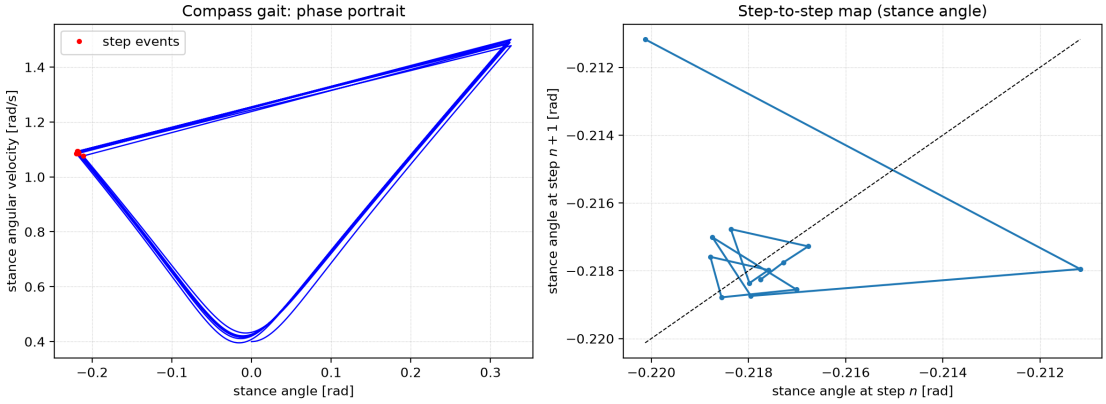



Figure 5: Compass gait: phase portrait (left) and step-to-step map of the stance angle (right). Red dots mark the detected heel strikes; the points converge to a fixed point of the step map.

### 3.3 Step detection and sensitivity

We detect a step by a change of the plant's abstract "left support" state. For the nominal initial condition the gait takes 14 steps in 10 s, with a mean step period of 0.735 s. The post-impact states converge rapidly to a neighbourhood of the limit cycle.

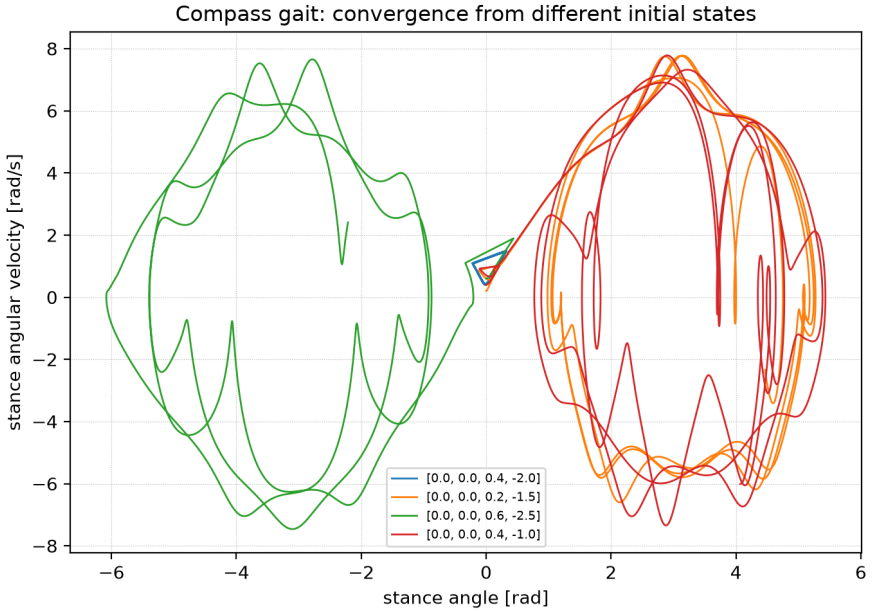



Figure 6: Compass gait from different initial states. Only a limited set of initial conditions reaches the limit cycle; others leave the narrow basin and fall.

Table 2: Compass gait: initial-condition sensitivity (10 s horizon).

| Initial state | Steps | Bounded/sustained? |
|---|---|---|
| $[0, 0, 0.4, -2.0]$ | 14 | Yes |
| $[0, 0, 0.2, -1.5]$ | 7 | No |
| $[0, 0, 0.6, -2.5]$ | 1 | No |
| $[0, 0, 0.4, -1.0]$ | 10 | No |

For the slope sensitivity we used a $3 \times 3$ grid of initial states $\dot{\theta}_{st}(0) \in \{0.2, 0.4, 0.6\}$, $\dot{\theta}_{sw}(0) \in \{-2.5, -2.0, -1.5\}$ and ran 12 s for each slope. A run is classified as sustained when it has at least 10 steps, the last five step periods are consistent, and the state remains bounded.

Table 3: Compass gait: fraction of initial states leading to a sustained gait.

| Slope $\gamma$ [rad] | Successes / trials | Fraction | Mean period [s] |
|---|---|---|---|
| 0.0200 | 3/9 | 0.33 | 0.705 |
| 0.0350 | 3/9 | 0.33 | 0.715 |
| 0.0525 | 3/9 | 0.33 | 0.730 |
| 0.0800 | 0/9 | 0.00 | – |
| 0.1200 | 0/9 | 0.00 | – |

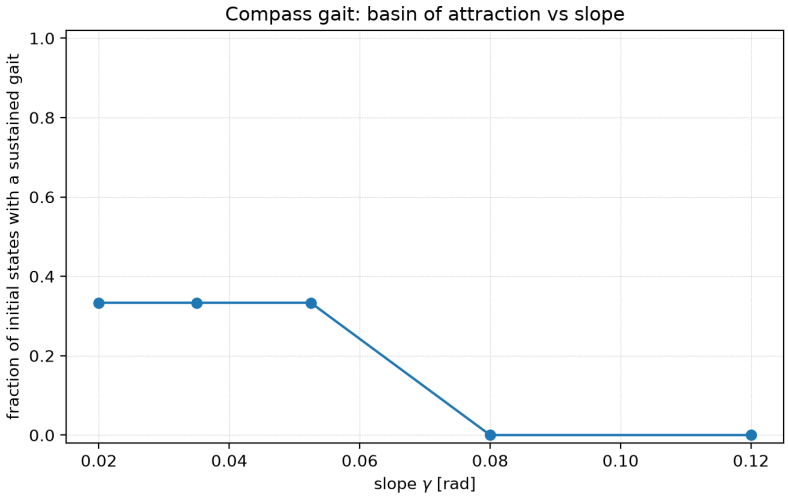



Figure 7: Compass gait: basin of attraction for the coarse initial-state grid as a function of slope.

**Comments.** The compass gait is also passive: no hip torque is used. Its limit cycle is an example of a stable periodic orbit of a hybrid system; the step-to-step map plays the role of the Poincaré map and its linearised eigenvalues (Floquet multipliers) determine orbital stability. In contrast to the rimless wheel, the basin of attraction is much narrower: only a small set of initial conditions, and a limited range of slopes, leads to sustained walking. This is consistent with the lecture notes: with only two legs there is no "extra spoke" to catch the body, so the swing-leg landing must be timed precisely. Real robots therefore need feedback (energy injection, foot placement, posture control) to enlarge the basin.

# 4 Example 4.5: The SLIP Model

## 4.1 Model

The spring-loaded inverted pendulum (SLIP) models running with a point mass $m = 80$ kg, a massless spring leg of rest length $r_0 = 1$ m and stiffness $k = 10.7\,mg/r_0$, and a touchdown angle that can be reset during flight. During flight the dynamics are ballistic:

$$\ddot{x} = 0, \qquad \ddot{z} = -g. \tag{9}$$

During stance, with $r$ the leg length and $\theta$ the leg angle,

$$m\ddot{r} - mr\dot{\theta}^2 + mg\cos\theta - k(r_0 - r) = 0, \tag{10}$$

$$mr^2\ddot{\theta} + 2mr\dot{r}\dot{\theta} - mgr\sin\theta = 0. \tag{11}$$

The total energy

$$E = \frac{1}{2}m\left(\dot{r}^2 + r^2\dot{\theta}^2\right) + mgr\cos\theta + \frac{1}{2}k(r_0 - r)^2 \tag{12}$$

is conserved: there is no impact loss because the leg and foot are massless. This is the key difference from the rimless wheel and compass gait.

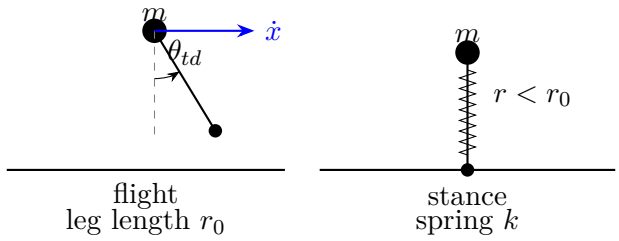



Figure 8: SLIP schematic. In flight the leg is reset to the touchdown angle $\theta_{td}$; in stance the spring compresses ($r < r_0$), stores energy and extends again. The leg and foot are massless, so there is no impulsive energy loss at touchdown.

**Why SLIP is a "conservative" runner**

The leg and foot are massless, so the ground contact does not produce an impulse and the mechanical energy is conserved. That is the main difference from the rimless wheel and the compass gait, which lose energy at every impact. Running is therefore a segmented Hamiltonian motion: ballistic flight, then a spring-mass stance, then flight again. The touchdown angle decides how much horizontal velocity is converted into compression. Because the energy is constant, the apex-to-apex map can be reduced to one dimension on an energy slice, but that also limits what the map can prove.

## 4.2 Results of the supplied notebook

The notebook starts from a flight state with touchdown angle $\theta_{td} = 30°$, dimensionless energy $\tilde{E} = 1.61$ and apex height $z = 0.9$ m, and simulates 2 s. The leg compresses during stance, stores elastic energy, and extends again; the body follows a running-like trajectory with a flight phase between stance phases. The notebook also builds the apex-to-apex map, which is analysed in the next section.

**Comments.** SLIP is a segmented Hamiltonian system. Unlike the walking models, it has no dissipative impact, so the energy is constant to integration accuracy. The stance phase is a spring-mass oscillator with centrifugal and gravity terms; the touchdown angle selects how much of the horizontal velocity is converted into compression. The model captures the essential running mechanism (compression and restitution) while remaining low dimensional.

# 5 Example 4.6: The SLIP Apex-to-Apex Map

## 5.1 Poincaré section and one-dimensional map

At the apex the vertical velocity is zero and the horizontal position is irrelevant because of translation symmetry. Energy conservation makes the horizontal velocity a function of the apex height,

$$\dot{x} = \sqrt{\frac{2}{m}\,(E - mgz)}, \tag{13}$$

so the return map on the constant-energy slice is one-dimensional:

$$z[n+1] = P(z[n]). \tag{14}$$

A fixed point $z^* = P(z^*)$ corresponds to a periodic running gait; its stability is determined by $P'(z^*)$ (stable if $|P'| < 1$).

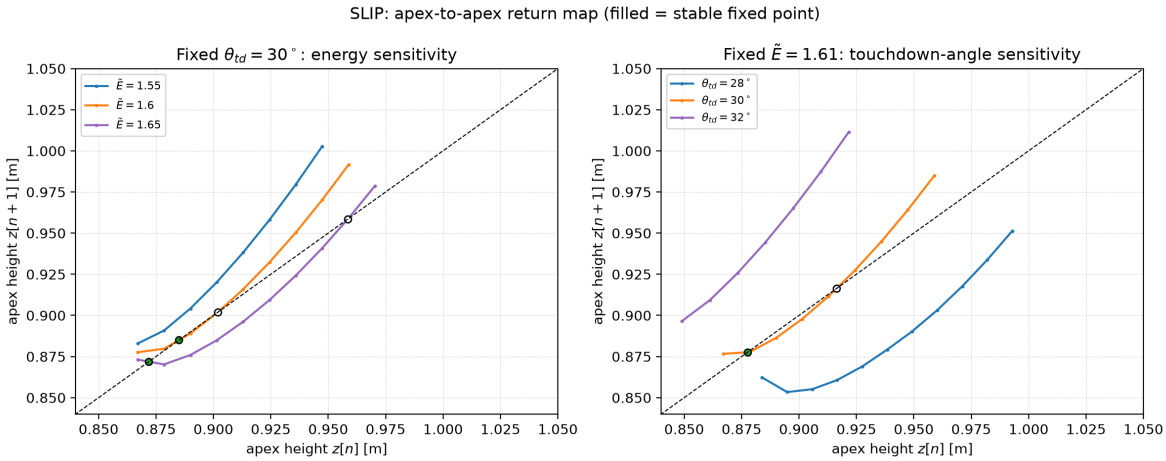



Figure 9: SLIP apex-to-apex map. Left: fixed touchdown angle $\theta_{td} = 30°$, varying energy. Right: fixed energy $\tilde{E} = 1.61$, varying touchdown angle. Filled circles are stable fixed points, open circles are unstable ones.

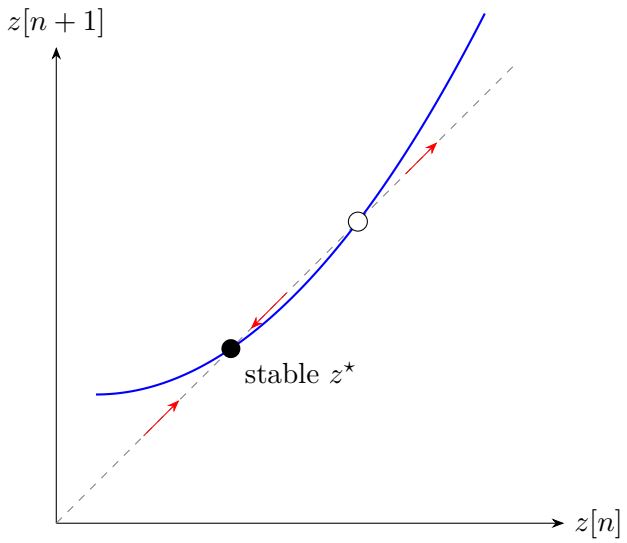



Figure 10: Schematic of a one-dimensional apex map. Intersections of the blue curve with the diagonal are periodic gaits. A stable fixed point has $|P'(z^\star)| < 1$ (nearby iterates move toward it); an unstable one has $|P'(z_u)| > 1$. The red arrows show the direction of the iteration.

**How to build and read a Poincaré map**

1. Choose a section that the periodic orbit crosses once per cycle (e.g. the apex for SLIP, or the heel strike for walking).
2. Simulate from many initial conditions on the section and record the next crossing. This defines the map $z[n+1] = P(z[n])$.
3. Find fixed points by solving $P(z) = z$; the slope $P'(z)$ at the fixed point decides stability ($|P'| < 1$).
4. Remember what the map projects away. If it was obtained by using a conserved quantity, as in SLIP, the stability is only partial.

**A stable apex map is not full stability**

For SLIP the map is computed on a constant-energy slice: the horizontal velocity at the apex is reconstructed from $z$ and the total energy. A perturbation that changes the energy, such as a step in the terrain or a push, is not represented by the one-dimensional map. Thus $|P'| < 1$ proves stability only within that slice. This is why the notes call it *partial* stability, and why a real robot still needs energy regulation.

### 5.2 Fixed points and sensitivity

For the nominal condition $\theta_{td} = 30°$, $\tilde{E} = 1.61$ we find

$$z^*_{\text{stable}} = 0.8776, \quad P'(z^*) = 0.420, \qquad z^*_{\text{unstable}} = 0.9163, \quad P'(z^*) = 1.288. \tag{15}$$

These agree with the textbook values (0.8768 and 0.9168, with derivatives 0.488 and 1.346); the small differences come from the finite grid and linear interpolation.

Table 4: SLIP: fixed points for the tested parameter neighbourhood.

| $\theta_{td}$ [deg] | $\tilde{E}$ | $z^*$ [m] | $P'(z^*)$ |
|---|---|---|---|
| 30 | 1.60 | 0.8850 | 0.500 (stable) |
| 30 | 1.60 | 0.9019 | 1.167 (unstable) |
| 30 | 1.61 | 0.8776 | 0.420 (stable) |
| 30 | 1.61 | 0.9163 | 1.288 (unstable) |
| 30 | 1.65 | 0.8719 | 0.122 (stable) |
| 30 | 1.65 | 0.9586 | 1.513 (unstable) |

**Comments.** The apex map has a stable and an unstable fixed point for the nominal parameters; initial heights between them converge to the lower (stable) gait, while heights above the unstable point leave the map's domain or fail. Changing the energy by $\pm 0.05$ moves the fixed points but leaves a stable gait; changing the touchdown angle to $28^\circ$ or $32^\circ$ removes the fixed point in our scan. The stable fixed point at the nominal condition is therefore a narrow operating point, consistent with the map's limited domain.

**Partial stability.** The one-dimensional map is obtained by projecting the two-dimensional apex state $(z, \dot{x})$ onto $z$ using energy conservation. Consequently, $|P'| < 1$ only proves stability with respect to perturbations that preserve the total energy. It does not prove stability to terrain changes, pushes or other disturbances that change the energy. Since SLIP is conservative, this is a *partial* stability result, not asymptotic stability of the full running motion. This distinction is important for control design: the apex map is useful for one-step-ahead controllers, but a real robot needs additional energy regulation.

# 6 Section 4.6: The One-Dimensional Hopper

## 6.1 Model and hopping cycle

The hopper has a body of mass $m_b$ and a leg of mass $m_l$ connected by an elastic prismatic joint of stiffness $k$. With $z_b$ the body height and $l = z_l - z_b$ the leg compression, the state is $[z_b, l, \dot{z}_b, \dot{l}]^\top$. The spring force is $-k(l + \delta)$, where $\delta \geq 0$ is the preload control.

The cycle has four phases: flight, touchdown (inelastic impact of the leg), the stance phase (leg on the ground, body moving down and up), and lift-off (inelastic impact between body and leg). The controller changes $\delta$ only twice per cycle:

- at the bottom $t_{bt}$: set $\delta(t_{bt}^+) = \bar{\delta}$;
- at the apex $t_{ap}$: reset $\delta(t_{ap}^+) = 0$.

**The controller in one sentence**

The collisions at touchdown and lift-off remove energy. At the bottom of the stance phase we compress the spring by a preload $\bar{\delta}$; this injects exactly enough energy so that the next apex reaches the desired height $h$. At the apex we reset the preload to zero. The whole controller is therefore a scalar energy calculation, updated twice per hop, not a trajectory tracker.

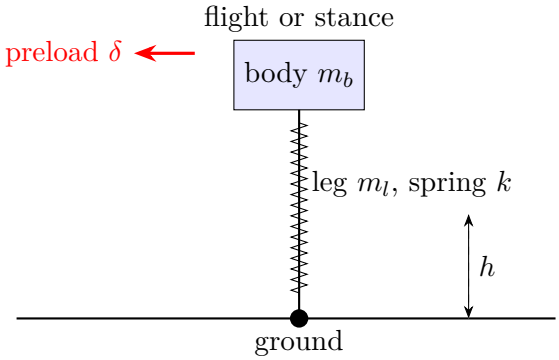



Figure 11: One-dimensional hopper schematic. The body and leg are connected by a spring; $\delta \geq 0$ is the preload. The controller changes $\delta$ only at the bottom (apply $\bar{\delta}$) and at the apex (reset to zero).

### 6.2 Energy derivation of the preload

The mechanical energy is

$$E = \frac{1}{2} m_b \dot{z}_b^2 + \frac{1}{2} m_l (\dot{z}_b + \dot{l})^2 + m_b g z_b + m_l g (z_b + l) + \frac{1}{2} k (l + \delta)^2. \tag{16}$$

Before the preload is applied at the bottom, $l = -z_b$ and $\dot{z}_b = 0$:

$$E(t_{bt}^-) = m_b g z_b + \frac{1}{2} k z_b^2. \tag{17}$$

After applying $\bar{\delta}$,

$$E(t_{bt}^+) = m_b g z_b + \frac{1}{2} k (\bar{\delta} - z_b)^2. \tag{18}$$

At lift-off, the body is at the hard stop and the leg is on the ground. Water energy conservation from bottom to lift-off gives the body kinetic energy before impact:

$$\frac{1}{2} m_b \dot{z}_b^2 (t_{lo}^-) = m_b g z_b + \frac{1}{2} k z_b^2 - k \bar{\delta} z_b. \tag{19}$$

Momentum conservation for the inelastic lift-off impact gives

$$E(t_{lo}^+) = \frac{m_b}{m_b + m_l} \left[ m_b g z_b + \frac{1}{2} k z_b^2 - k \bar{\delta} z_b \right] + \frac{1}{2} k \bar{\delta}^2. \tag{20}$$

At the apex both body and leg are at rest at height $h$:

$$E(t_{ap}^-) = (m_b + m_l) g h + \frac{1}{2} k \bar{\delta}^2. \tag{21}$$

Since energy is conserved during flight, $E(t_{lo}^+) = E(t_{ap}^-)$, which gives the linear equation for $\bar{\delta}$:

$$\boxed{\bar{\delta} = \frac{1}{2} z_b + \frac{m_b g}{k} - \frac{(m_b + m_l)^2}{k m_b z_b} g h.} \tag{22}$$

If this value is negative, the system already has too much energy; the controller then uses $\bar{\delta} = 0$ and lets the impacts remove energy.

### 6.3 Implementation

Listing 1: Mechanical energy of the hopper.

```
def mechanical_energy(state, delta):
    k, mb, ml, g = get_hopper_parameters()
    zb, l, zb_dot, l_dot = state
    zl_dot = zb_dot + l_dot
    E = (0.5 * mb * zb_dot**2 + 0.5 * ml * zl_dot**2
         + mb * g * zb + ml * g * (zb + l)
         + 0.5 * k * (l + delta)**2)
    return np.asarray(E)
```

Listing 2: Preload controller.

```
class PreloadController(LeafSystem):
    def __init__(self, h):
        LeafSystem.__init__(self)
        self.DeclareVectorInputPort("state", 4)
        self.DeclareVectorOutputPort("delta", 1, self.DoCalcVectorOutput)
        self.h = h
        self.hopper_parameters = get_hopper_parameters()
        self.last_delta = 0.0
        self.last_state = np.full(4, np.nan)

    def is_bottom_or_apex(self, state):
        if np.any(np.isnan(self.last_state)):
            return None
        zb, l, zb_dot, l_dot = state
        last_zb_dot = self.last_state[2]
        if last_zb_dot <= 0.0 and zb_dot > 0.0:
            return "bottom" if zb < 0.0 else "apex"
        if last_zb_dot >= 0.0 and zb_dot < 0.0:
            return "apex" if zb > 0.0 else "bottom"
        return None

    def get_delta_bar(self, state):
        k, mb, ml, g = self.hopper_parameters
        zb = state[0]
        return (0.5 * zb + mb * g / k
                - ((mb + ml) ** 2) * g * self.h / (k * mb * zb))

    def DoCalcVectorOutput(self, context, delta):
        state = self.get_input_port(0).Eval(context)
        event = self.is_bottom_or_apex(state)
        if event == "bottom":
            delta.SetFromVector([max(self.get_delta_bar(state), 0.0)])
        elif event == "apex":
            delta.SetFromVector([0.0])
        else:
            delta.SetFromVector([self.last_delta])
        self.last_delta = delta[0]
        self.last_state = np.array(state)
```

### 6.4 Results and autograding

The notebook was executed end-to-end. The autograder reports

**Total score is 13/13: mechanical energy 3/3, preload controller 10/10.**

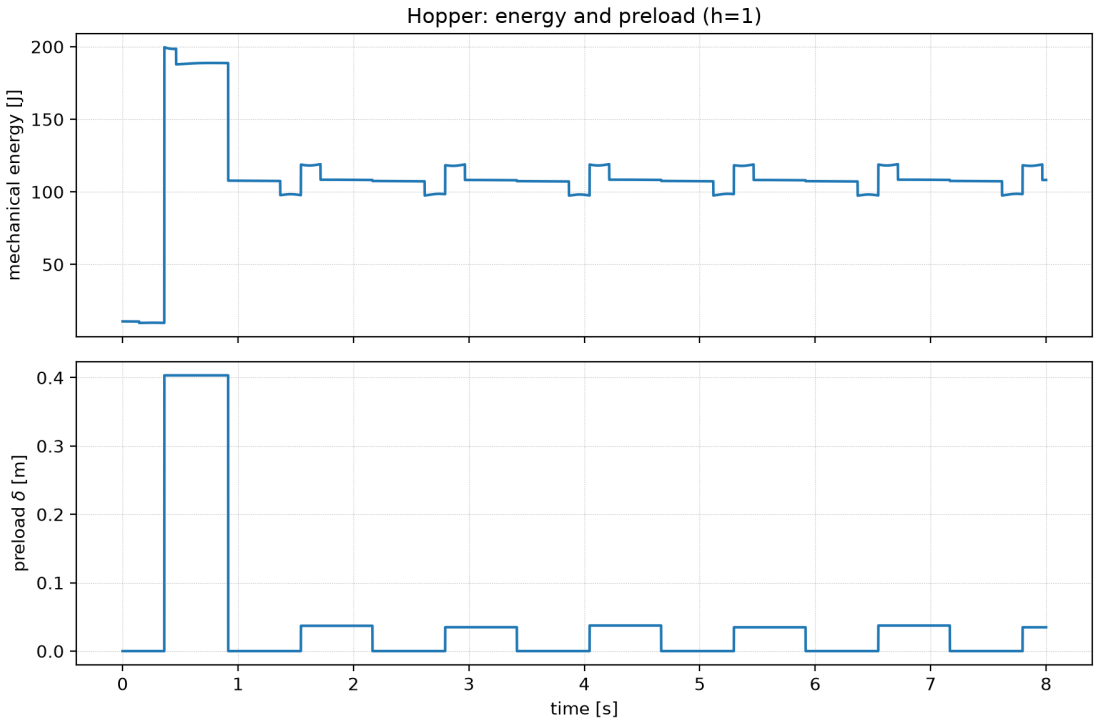



Figure 12: Hopper energy and preload for $h = 1$ m. The preload is applied at the bottom and reset at the apex; the energy drops at the two impacts.

Table 5: Hopper: target-height sensitivity (final 2 s of an 8 s run).

| Target $h$ [m] | Max body height [m] | Error [m] | Max preload $\delta$ [m] |
|---|---|---|---|
| 0.50 | 0.4985 | −0.0015 | 0.024 |
| 0.75 | 0.7477 | −0.0023 | 0.093 |
| 1.00 | 0.9966 | −0.0034 | 0.163 |
| 1.25 | 1.2444 | −0.0056 | 0.232 |
| 1.50 | 1.4946 | −0.0054 | 0.302 |

Table 6: Hopper: initial-height sensitivity for $h = 1$ m.

| Initial $z_b(0)$ [m] | Max height (last 2 s) [m] | Error [m] | Max preload [m] |
|---|---|---|---|
| 0.1 | 0.9966 | −0.0034 | 0.403 |
| 0.3 | 0.9966 | −0.0034 | 0.247 |
| 0.5 | 0.9966 | −0.0034 | 0.163 |
| 0.8 | 0.9966 | −0.0034 | 0.078 |
| 1.0 | 0.9966 | −0.0034 | 0.037 |

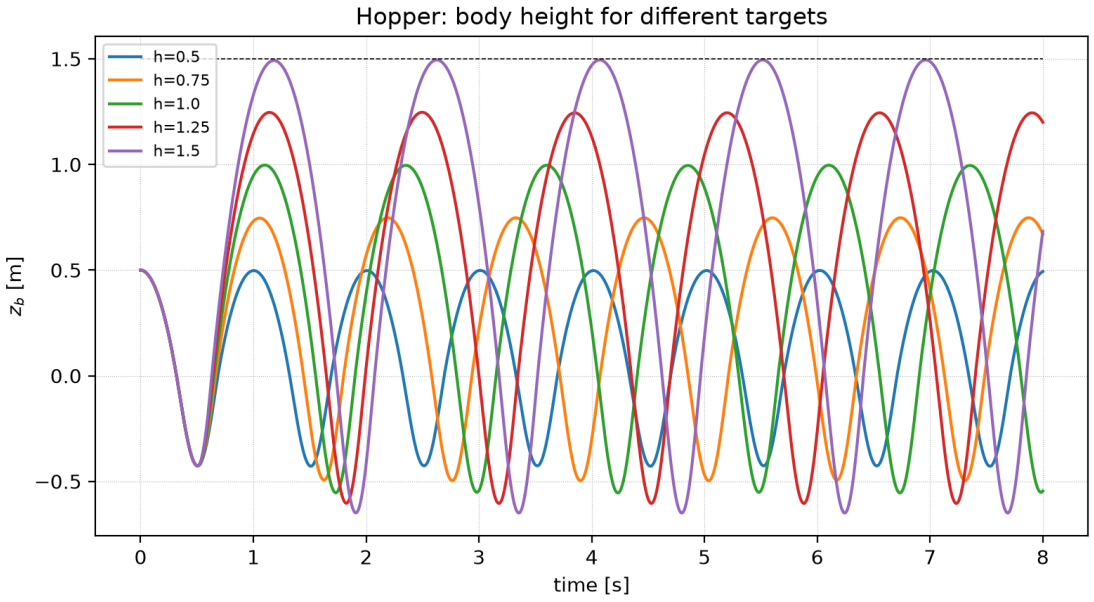



Figure 13: Hopper: body height for different target heights. After a short transient all runs settle into a periodic hop at the requested height.

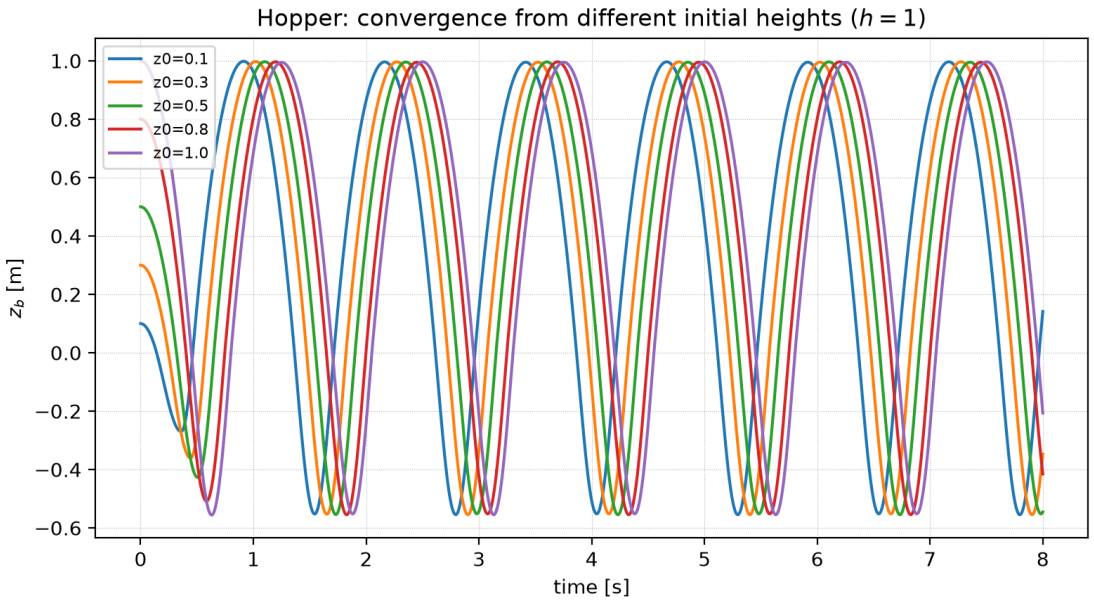



Figure 14: Hopper: convergence from different initial body heights at $h = 1$ m. The energy controller rejects the initial-height error.

**Comments.** The controller is an energy-based one-step controller: it does not track a reference trajectory, but chooses the preload exactly so that the next apex has the desired mechanical energy. The target-height error is below 6 mm for all tested heights, and the controller is insensitive to the initial body height. The maximum preload grows roughly linearly with the target height: a higher hop needs more energy, and the spring stores it as $\frac{1}{2}k\bar{\delta}^2$. The initial transient is larger when the hopper starts low, because more energy must be injected at the first bottom; after that the cycle is periodic. This is a clean example of using a Poincaré/energy argument to reduce a trajectory-tracking problem to a scalar control decision once per hopping cycle.

# 7 Conclusions

1. **Passive dynamic walking.** The rimless wheel and the compass gait show that a stable periodic gait can arise without actuation: gravity supplies energy during stance and collisions remove it at foot strike. The step-to-step return map is the natural tool; its fixed point is the gait and its slope/Floquet multipliers determine orbital stability.

2. **Rimless wheel.** The return map is one-dimensional and the fixed point is stable with multiplier $\cos^2(2\alpha) = 1/2$. The main sensitivity is the slope: below a critical slope the fixed point lies below the step threshold and no rolling gait exists.

3. **Compass gait.** The limit cycle is much more delicate: only a narrow set of initial conditions and slopes leads to sustained walking. The coarse sensitivity grid confirms the small basin of attraction; feedback is needed for robustness.

4. **SLIP.** The model is conservative and captures running as a sequence of flight and spring stance phases. The apex map has a stable and an unstable fixed point for the nominal parameters, but the one-dimensional stability is only partial because it relies on energy conservation. The touchdown angle and energy strongly affect whether a fixed point exists.

5. **Hopper.** The energy analysis gives a closed-form preload $\bar{\delta}(z_b, h)$; the controller applies it at the bottom and resets at the apex. The autograder passes 13/13, the hopping height error is below 6 mm, and the controller is robust to the initial body height.

# 8 Code and Reproduction

The complete code is in the repository. Main entry points:

| Topic | Script / notebook |
|---|---|
| Example 4.3 | `Exp4'3-rimless-wheel.ipynb`, `rimless_wheel_analysis.py` |
| Example 4.4 | `Exp4'4-compass-gait.ipynb`, `compass_gait_analysis.py` |
| Examples 4.5/4.6 | `Exp4'5-6-slip.ipynb`, `slip_analysis.py` |
| Section 4.6 | `Exp4'6-one_d_hopper.ipynb`, `one_d_hopper_analysis.py` |

Commands (from `Ch-4/exp4_analysis`):

```
/Users/leo/venvs/drake/bin/python rimless_wheel_analysis.py
/Users/leo/venvs/drake/bin/python compass_gait_analysis.py
/Users/leo/venvs/drake/bin/python slip_analysis.py
/Users/leo/venvs/drake/bin/python one_d_hopper_analysis.py
```

Headless notebook execution:

```
COLAB_TESTING=1 /Users/leo/venvs/drake/bin/jupyter nbconvert \
  --to notebook --execute --inplace --ExecutePreprocessor.timeout=300 \
  "Exp4'6-one_d_hopper.ipynb"
```

The report is compiled with

```
pdflatex Ch4_Report.tex
pdflatex Ch4_Report.tex
```

# A Key Code Snippets

## A.1 Rimless wheel return map

```python
def f_map(omega, gamma):
    return np.cos(2 * ALPHA) * np.sqrt(
        np.maximum(omega**2 + D(gamma), 0.0))

def fixed_point(gamma):
    return (1.0 / np.tan(2 * ALPHA)) * np.sqrt(D(gamma))
```

## A.2 SLIP apex map

```python
def apex_map(touchdown_angle, Etilde, n=30, zmax=1.2):
    plant = SpringLoadedInvertedPendulum()
    s = SLIPState(np.zeros(8))
    s.theta = touchdown_angle
    s.r = 1.0
    simulator = Simulator(plant)
```

```
    context = simulator.get_mutable_context()
    plant.get_input_port(0).FixValue(context, [touchdown_angle])
    context.SetAccuracy(1e-5)
    zs = np.linspace(np.cos(touchdown_angle) + 0.001, zmax, n)
    zns = np.full(n, np.nan)
    for i, z in enumerate(zs):
        s.z = z
        s.xdot = plant.apex_velocity_from_dimensionless_system_energy(Etilde, z)
        context.SetTime(0.0)
        context.get_mutable_continuous_state_vector().SetFromVector(s[:])
        simulator.Initialize()
        simulator.AdvanceTo(0.6)
        if plant.last_apex is not None:
            zns[i] = plant.last_apex
        plant.last_apex = None
    return zs, zns
```

# Chapter 5 Assignment

## Highly-articulated Legged Robots

Zihao Liu
Student ID: 220261979
213222647@seu.edu.cn

October 2026

**Abstract**

This report covers Chapter 5 of the course. Part I comments on the two textbook examples: the ZMP planner (Example 5.1) and the LittleDog gait optimization (Example 5.2). The supplied notebooks were executed, the results are interpreted with the centroidal-dynamics and ZMP theory of the chapter, and the sensitivity to footstep timing, disturbances and gait parameters is studied. Part II solves all exercises of Section 5.8: footstep planning via mixed-integer quadratic programming (MIQP), the conceptual exercise on the zero-moment point, and footstep planning via a graph of convex sets (GCS). Both programming exercises pass their autograders with 16/16 and 16/16. The complete Python code and the executed notebooks accompany the submission; the most important code is reproduced in the appendix.

## Contents

**1 Overview and Reproducibility** **2**

**2 Example 5.1: The ZMP Planner** **3**
2.1 Model: the linear inverted pendulum and the ZMP . . . 3
2.2 Nominal plan and timing sensitivity . . . 4
2.3 The feedback policy rejects disturbances . . . 6

**3 Example 5.2: LittleDog Gait Optimization** **7**
3.1 Model and decision variables . . . 7
3.2 Constraints and costs . . . 7
3.3 Results . . . 8

**4 Section 5.8(a): Footstep Planning via MIQP** **11**
4.1 Terrain and decision variables . . . 11
4.2 The four constraints and the objective . . . 11
4.3 Results on the two benchmark terrains . . . 12
4.4 Step-span sensitivity and route switching . . . 13

**5 Section 5.8(b): Understanding the Zero-Moment Point** **14**
5.1 Free-body diagrams . . . 14
5.2 (c) The ZMP without an ankle actuator . . . 15

5.3 (e)–(f) With an ideal ankle actuator . . . . . 16
5.4 (g)–(h) Foot design . . . . . 16

**6 Section 5.8(c): Footstep Planning via GCS** **17**
6.1 Transcription . . . . . 17
6.2 Results . . . . . 18
6.3 MIQP versus GCS . . . . . 19

**7 Conclusions** **20**

**8 Code and Reproduction** **20**

**A Key Code Snippets** **21**
A.1 ZMP planner evaluation . . . . . 21
A.2 MIQP constraints . . . . . 22
A.3 GCS edges . . . . . 22
A.4 LittleDog centroidal constraints (sketch) . . . . . 23

# 1 Overview and Reproducibility

The assignment consists of:

- **Examples** (code supplied; run and comment): Example 5.1 (`ZmpPlanner`) and Example 5.2 (LittleDog gait optimization).
- **Section 5.8** (all exercises): MIQP footstep planning, the zero-moment-point exercise, and GCS footstep planning.

All simulations use Drake 1.58.0 and the `underactuated` package 2026.9.8 in `/Users/leo/venvs/drake/bin/python` (NumPy 2.5.4, SciPy 1.18.1, Matplotlib 3.11.2). SNOPT is available in this Drake build and is used both for the LittleDog trajectory optimization and for the GCS shortest-path convex relaxation. The exercise notebooks were executed headlessly with `COLAB_TESTING=1`, which prevents interactive visualizers from blocking. The LittleDog notebook was executed *without* that flag, because `running_as_notebook` must be true for SNOPT to be allowed its full iteration budget; the complete optimization takes 8.3 s.

| Contents | Location |
|---|---|
| Example notebooks | `Ch-5/Ch.5-examples/Exp5'1-*.ipynb`, `Exp5'2-*.ipynb` |
| Section 5.8 notebooks | `Exp5'8-footstep-planning.ipynb`, `Exp5'8-footstep-planning-gcs.ipynb` |
| Analysis scripts and figures | `Ch-5/exp5_analysis/` |
| This report | `Ch-5/Ch5_Assignment_Report/` |

The autograder results are

```
MIQP footstep planning:  16/16.    GCS footstep planning:  16/16.
```

### How to read this report

The report has three layers: *models*, *optimization* and *interpretation*. If an equation looks dense, read the **Intuition** boxes first: they give the physical picture without algebra. The **Recipe** boxes summarise a reusable procedure, and the **Common pitfall** boxes point out the mistakes that are easy to make.

| Part | What is done | What to look for |
|---|---|---|
| Example 5.1 | LIPM + `ZmpPlanner` | CoM lags the ZMP; timing changes sway; feedback rejects pushes |
| Example 5.2 | LittleDog trajectory optimization | contact schedule; CoM and angular-momentum constraints; gait snapshots |
| Section 5.8 (MIQP) | Big-M footstep planning | one stone per foot; step-span limit; lateral detour |
| Section 5.8 (ZMP) | Foot/leg/bob analysis | ZMP line of action; heel lift; actuated posture range |
| Section 5.8 (GCS) | Shortest path in a graph of convex sets | copies of stones; edge constraints; LP relaxation |

**The big picture in five sentences**

A highly articulated robot that stands on flat ground can be planned from the outside in: first choose footsteps, then choose a CoM trajectory whose zero-moment point stays inside the support polygon, and finally realize that CoM trajectory with the whole body. The CoM equations are exact for the full multibody system; the only approximation is the set of contact forces the robot can produce. Footstep planning is a combinatorial problem, solved either by branch and bound over binaries (MIQP) or by a shortest path in a graph of convex sets (GCS). The ZMP is the bridge between the two layers: it turns a contact constraint into a constraint on CoM acceleration.

| Symbol | Meaning |
|---|---|
| $\mathbf{p}_{cm} = (x, y, z)$ | center of mass position |
| $\mathbf{p}_{cp}$, $\mathbf{p}_{zmp}$ | center of pressure / zero-moment point |
| $m$ | total mass |
| $z$ | constant CoM height in the LIPM |
| $\mathbf{f}_i$ | contact force at contact point $i$ |
| $\mu$ | friction coefficient |
| $d$ | step span (side of the reachable square) |
| $\delta_{t,i}$ | binary: foot at step $t$ is on stone $i$ |
| $N, L_1, L_2, h$ | exercise parameters (leg length, toe/heel offsets, ankle height) |

# 2 Example 5.1: The ZMP Planner

## 2.1 Model: the linear inverted pendulum and the ZMP

For a point mass at constant height $z$, the horizontal CoM dynamics are the linear inverted pendulum model (LIPM)

$$\ddot{\mathbf{c}} = \frac{g}{z}\left(\mathbf{c} - \mathbf{y}\right), \tag{1}$$

where $\mathbf{c} = (x, y)$ is the CoM position and $\mathbf{y}$ is the zero-moment point. On flat ground the ZMP coincides with the center of pressure (CoP), i.e. with the point where the resultant ground reaction acts. Equation (1) is another way of writing the moment balance

$$\mathbf{y} = \mathbf{c} - \frac{z}{g}\ddot{\mathbf{c}}. \tag{2}$$

The ZMP must lie in the support polygon; for a single flat foot this is the segment between heel and toe.

Drake's `ZmpPlanner` takes a desired ZMP trajectory $\mathbf{y}_d(t)$ (a piecewise polynomial), an initial CoM state $\mathbf{x}_0 = [\mathbf{c}_0; \dot{\mathbf{c}}_0]$ and the height $z$, and solves

$$\min_{\mathbf{y},\ddot{\mathbf{c}}} \int_0^{t_f} \left[ \|\mathbf{y}(t) - \mathbf{y}_d(t)\|^2_{\mathbf{Q}_y} + \|\ddot{\mathbf{c}}(t)\|^2_{\mathbf{R}} \right] dt + \bar{\mathbf{x}}(t_f)^\top \mathbf{S}\, \bar{\mathbf{x}}(t_f), \tag{3}$$

subject to (2), where $\bar{\mathbf{x}}$ is the state relative to the final ZMP and $\mathbf{S}$ is the infinite-horizon LQR cost-to-go of the same dynamics. The solution is a nominal CoM trajectory together with a time-varying linear feedback policy

$$\ddot{\mathbf{c}}^* = \mathbf{K}(t)\left(\mathbf{x}(t) - \mathbf{x}_{\text{nom}}(t)\right) + \ddot{\mathbf{c}}_{\text{nom}}(t). \tag{4}$$

This is the important practical point: the planner does not only return a trajectory, it returns a policy that rejects state perturbations.

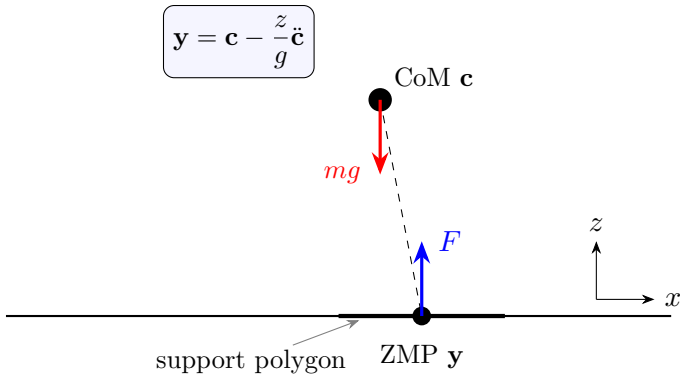



Figure 1: The LIPM and the ZMP. The ground reaction at the ZMP and gravity at the CoM form a couple that produces the CoM acceleration. Keeping the ZMP inside the support polygon (the black segment under the foot) keeps the foot flat.

**Building a ZMP plan from footsteps**

1. Choose a footstep sequence $\mathbf{p}_0, \mathbf{p}_1, \ldots$ and a timing (single-support and double-support durations).
2. Build the desired ZMP as a piecewise-linear path between the foot centers: hold at each foot during single support, interpolate during double support.
3. Call `ZmpPlanner.Plan(zmp_traj, x0, height)`.
4. Evaluate `get_nominal_com`, `get_nominal_comd` and `ComputeOptimalCoMdd`; use the latter as the feedback policy.
5. Check that the realized ZMP $\mathbf{c} - (z/g)\ddot{\mathbf{c}}$ stays inside the support polygon and close to the desired one.

## 2.2 Nominal plan and timing sensitivity

The supplied notebook was completed with a plotting and evaluation block. The footsteps are

$$\text{footsteps} = [(0, -0.05), (0.5, 0.05), (1, -0.05), (1.5, 0.05), (2, -0.05), (2.5, 0.05)],$$

i.e. a 0.5 m stride with a 0.1 m lateral sway, and the CoM height is $z = 1$ m. The nominal timing is a 0.5 s double support and a 1.0 s single support.

Table 1: Example 5.1: nominal plan metrics.

| Quantity | Value |
|---|---|
| Plan duration | 8.50 s |
| Peak lateral CoM excursion $\max_t \lvert\text{CoM}_y\rvert$ | 46.3 mm |
| Maximum ZMP tracking error | 0.158 m |
| RMS ZMP tracking error | 0.078 m |
| Final CoM $x$ | 2.490 m (last footstep at 2.500 m) |

The CoM sway is only 4.6 cm although the footsteps alternate by 10 cm; the finite Q/R weighting

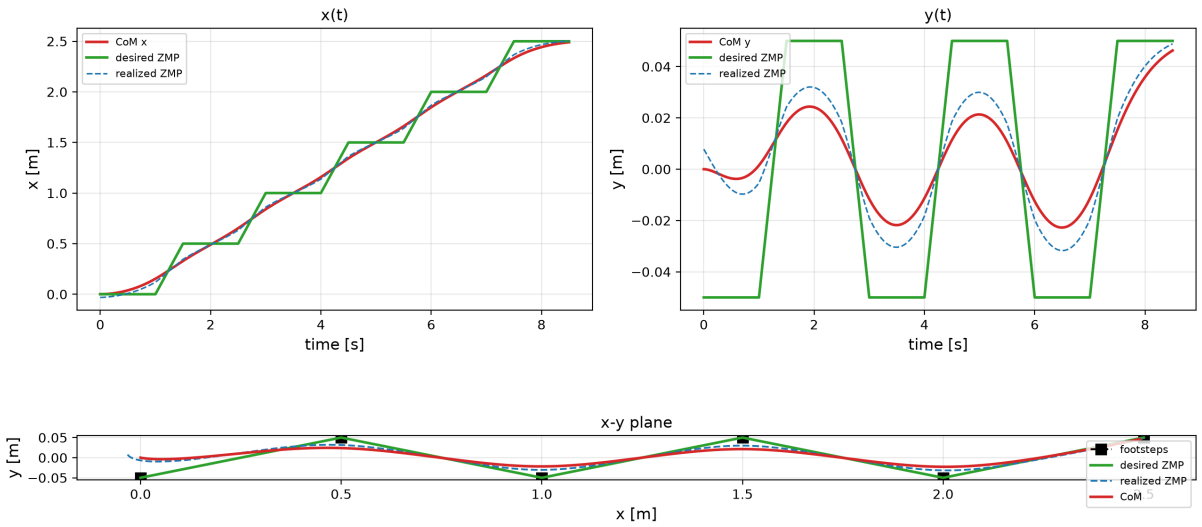



Figure 2: Example 5.1, nominal plan. Top: CoM and realized/desired ZMP in $x(t)$ and $y(t)$. Bottom: top view. The realized ZMP is smoother than the commanded staircase because the quadratic cost (3) trades ZMP tracking against CoM acceleration.

makes the CoM cut the corners of the ZMP staircase. The ZMP tracking error is not zero: with $\mathbf{Q}_y = I$ and $\mathbf{R} = 0.1I$ the planner prefers a smooth CoM acceleration and accepts a lag of up to 15.8 cm during double support. This is a useful reminder that a "ZMP plan" is an optimal-trade-off plan, not a hard equality constraint.

Table 2: Example 5.1: footstep-timing sensitivity (DS = double support, SS = single support).

| DS [s] | SS [s] | duration [s] | peak $\lvert\mathrm{CoM}_y\rvert$ [m] | max ZMP error [m] |
|---|---|---|---|---|
| 0.2 | 1.4 | 9.40 | 0.0490 | 0.204 |
| 0.5 | 1.0 | 8.50 | 0.0463 | 0.158 |
| 1.0 | 0.5 | 8.00 | 0.0411 | 0.095 |
| 1.0 | 1.0 | 11.00 | 0.0485 | 0.101 |

**Comments.** The single-support duration is the time the ZMP is held at one foot; the double-support duration is the time it travels between feet. A long single support leaves the CoM on its own for longer, so the pendulum falls farther laterally and the planner must use a larger sway. A long double support smooths the ZMP motion and reduces the tracking error, but it makes the plan longer. The fastest plan in the table (DS = 1.0, SS = 0.5) is also the one with the smallest tracking error, because the ZMP moves quickly between feet and then rests, which is easy for the LIPM to follow.

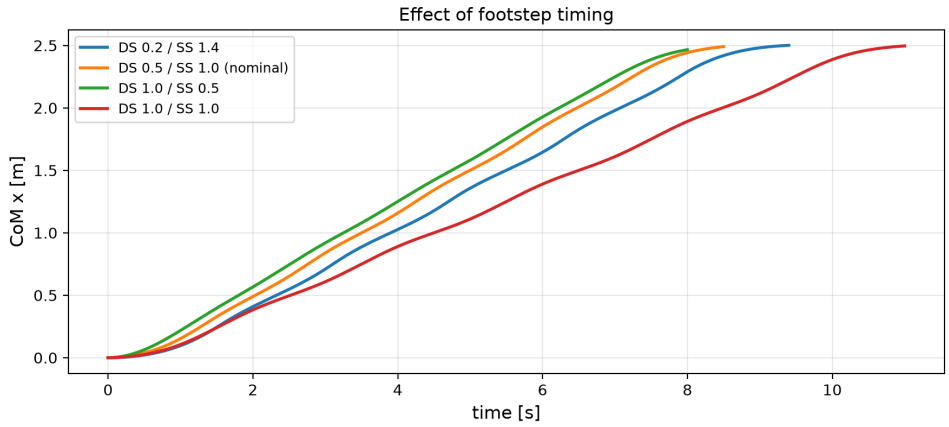



Figure 3: Example 5.1: CoM $x(t)$ for four footstep timings. Longer single support means a longer plan and larger lateral sway; a long double support with a short single support gives the fastest walk in this set.

### 2.3 The feedback policy rejects disturbances

The planner's value function gives the policy (4). We simulated the LIPM in closed loop from four initial states: the nominal state, a +0.30 m/s and a −0.30 m/s horizontal velocity perturbation, and a $(5, 3)$ cm CoM position offset.

Table 3: Example 5.1: closed-loop response to initial-state perturbations.

| Case | max $x$ deviation [m] | final $x$ deviation [m] | final $y$ deviation [m] |
|---|---|---|---|
| nominal | 0 | 0 | 0 |
| push +0.30 m/s in $x$ | 0.0709 | $1.2 \times 10^{-6}$ | 0 |
| push −0.30 m/s in $x$ | 0.0709 | $-1.2 \times 10^{-6}$ | 0 |
| CoM offset $(5, 3)$ cm | 0.0500 | $8.5 \times 10^{-8}$ | $5.1 \times 10^{-8}$ |

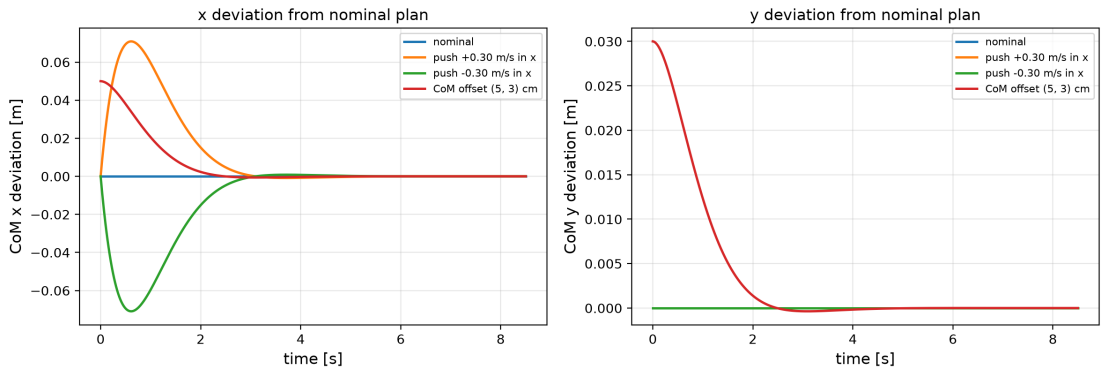



Figure 4: Example 5.1: deviation from the nominal plan under initial-state perturbations. The feedback policy removes a 0.30 m/s push within the 8.5 s plan; the residual at the final time is at the level of integration error.

**Why a ZMP planner is a feedback controller**

The LIPM is an unstable inverted pendulum: a small velocity error grows like $\exp(t\sqrt{g/z})$. The ZMP plan alone would diverge. The value function of the LQR problem, however, defines the optimal cost-to-go for every state, and its gradient gives a linear feedback law. Pushing the CoM forward makes the policy move the ZMP forward, which decelerates the CoM. In this sense the ZMP trajectory is a feedforward term and the policy is the feedback term.

**The ZMP is not the CoM**

A common mistake is to draw the CoM path and call it the ZMP, or to assume the ZMP must equal the CoM projection. They coincide only when the CoM acceleration is zero. During walking the CoM is constantly accelerating, and the ZMP leads or lags it by $(z/g)\ddot{\mathbf{c}}$. In Fig. 2 the desired ZMP is a staircase while the CoM is a smooth curve; the horizontal gap between them is exactly the acceleration term.

**Comments.** The example makes the layering of Chapter 5 concrete: the footstep sequence defines a ZMP trajectory, the ZMP trajectory defines a CoM plan and a feedback policy, and a whole-body controller would then realize the CoM plan with the joints. The LIPM ignores the angular momentum about the CoM; for slow walking this is a good approximation, but fast motions or arm swing require the centroidal-dynamics formulation discussed in the chapter.

# 3 Example 5.2: LittleDog Gait Optimization

## 3.1 Model and decision variables

LittleDog is a small quadruped with a floating base and three joints per leg (hip roll, hip pitch, knee): $n_q = 7 + 12 = 19$ positions and $n_v = 6 + 12 = 18$ velocities. The optimization is a direct transcription with $N = 21$ knot points. The decision variables are

- the $N - 1$ time steps $h_n > 0$;
- the configurations $\mathbf{q}_n \in \mathbb{R}^{19}$ and velocities $\mathbf{v}_n \in \mathbb{R}^{18}$;
- the four contact forces $\mathbf{f}_{i,n} \in \mathbb{R}^3$ at the four feet;
- the CoM position/velocity/acceleration $\mathbf{c}_n, \dot{\mathbf{c}}_n, \ddot{\mathbf{c}}_n$;
- the centroidal angular momentum $\mathbf{H}_n$ and its rate $\dot{\mathbf{H}}_n$.

## 3.2 Constraints and costs

The transcription enforces:

$$\mathbf{v}_{n+1} - \mathbf{v}_n = h_n \dot{\mathbf{v}}_n, \qquad \mathbf{q}_{n+1} - \mathbf{q}_n = h_n\, \mathbf{B}(\mathbf{q}_n)\mathbf{v}_n, \tag{5}$$

$$m\ddot{\mathbf{c}}_n = \sum_{i=1}^{4} \mathbf{f}_{i,n} + m\mathbf{g}, \qquad \dot{\mathbf{H}}_n = \sum_{i=1}^{4} (\mathbf{p}_{i,n} - \mathbf{c}_n) \times \mathbf{f}_{i,n}, \tag{6}$$

$$\mathbf{c}_n = \mathrm{CoM}(\mathbf{q}_n), \qquad \mathbf{H}_n = \mathbf{A}(\mathbf{q}_n, \mathbf{v}_n)\mathbf{v}_n, \tag{7}$$

$$|f^x_{i,n}| \le \mu f^z_{i,n}, \qquad |f^y_{i,n}| \le \mu f^z_{i,n}, \tag{8}$$

$$0 \le f^z_{i,n} \le s_{i,n}\, 4mg, \tag{9}$$

where $s_{i,n} \in \{0, 1\}$ is the prescribed stance schedule. The kinematic constraints force a stance foot to be on the ground ($z = 0$) and not to move between consecutive stance knots, and force a swing foot to have at least 1 cm clearance. Joint position and velocity limits, unit quaternions and a body-orientation constraint are imposed at every knot. Periodic boundary conditions are imposed on the half stride (the gait is laterally symmetric, so a half stride plus a mirror reflection produces a full stride).

The cost is a quadratic running cost

$$\sum_{n=0}^{N-1} \left[ (\mathbf{q}_n - \mathbf{q}_{\text{home}})^\top \mathbf{Q}_q (\mathbf{q}_n - \mathbf{q}_{\text{home}}) + \mathbf{v}_n^\top \mathbf{Q}_v \mathbf{v}_n \right], \tag{10}$$

with larger weights on the hip-roll joints (to discourage splaying), plus the implicit objective of finding any feasible gait. The problem is solved with SNOPT.

**Contact-implicit versus contact-scheduled transcription**

1. Choose a contact schedule: a binary stance matrix $s_{i,n}$ for each foot and knot.
2. Add contact forces only where $s_{i,n} = 1$ (normal-force upper bound $s_{i,n} 4mg$).
3. Impose the centroidal dynamics (6) and the centroidal consistency (7) at every knot.
4. Impose full kinematics: stance foot fixed on the ground, swing foot above the ground, joint limits.
5. Add periodicity and solve with a local nonlinear solver from a good initial guess (the home configuration).

### 3.3 Results

The notebook solves the walking-trot schedule in 8.3 s and SNOPT reports success. The optimal cost is 419.76.

Table 4: Example 5.2: walking-trot solution metrics.

| Quantity | Value |
|---|---|
| Gait | walking trot (lateral symmetry) |
| SNOPT status | success |
| Optimal cost | 419.76 |
| Solve time | 8.3 s |
| Half-stride duration | 0.34375 s |
| Full-stride duration | 0.6875 s |
| Stride length | 0.25 m |
| Effective speed | 0.364 m/s (requested 0.4 m/s) |
| CoM height range | 0.1309–0.1484 m |
| Maximum body pitch | 0.0189 rad |
| Maximum foot clearance | 11.9 mm |
| Maximum normal force | 74.8 N (back right) |

The effective speed is slightly below the requested 0.4 m/s because the optimizer chose half-stride durations at the upper end of the allowed window ($1.1T$, with $T = 0.3125$ s). This is consistent

with the cost: the squared joint velocities penalize fast motion, so the solver stretches the stride in time until it hits the bound.

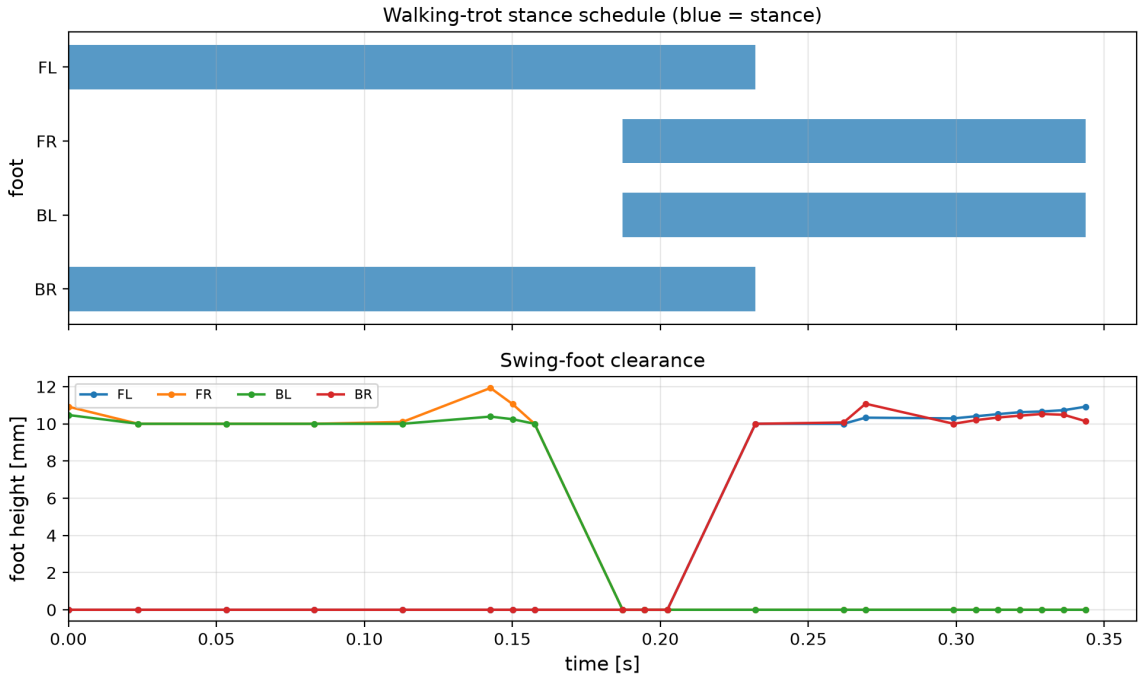



Figure 5: Example 5.2: walking-trot stance schedule and swing-foot clearance. Front-left and back-right are in stance for the first part of the half stride, then front-right and back-left take over. The swing feet clear the ground by about 10–12 mm.

**What the optimizer is actually doing**

The gait is not prescribed beyond the contact schedule. The optimizer has to find joint trajectories whose CoM follows the Newton–Euler equations under the four contact forces, while each stance foot stays fixed on the ground and each swing foot clears it. The angular-momentum constraint couples the contact forces to the body rotation: if the forces do not pass through the CoM, the body starts to rotate. The resulting motion is a compromise between small joint velocities, small hip-roll motion and the kinematic requirement that the feet reach the next contact.

**Comments.** The notebook is explicitly a work in progress: self collision is not modelled and several costs from the original MATLAB implementation are missing. The solution is therefore not necessarily dynamically feasible in a full simulation; it is a kinematic/centroidal consistency result. The value of the example is methodological: it shows how the exact centroidal dynamics, full kinematics and contact schedules fit into one nonlinear program, and it shows the role of a good initial guess (the home configuration) for a local solver. The walking trot is the easiest of the four schedules in the notebook; the running trot, rotary gallop and bound are more aggressive and are left commented out.

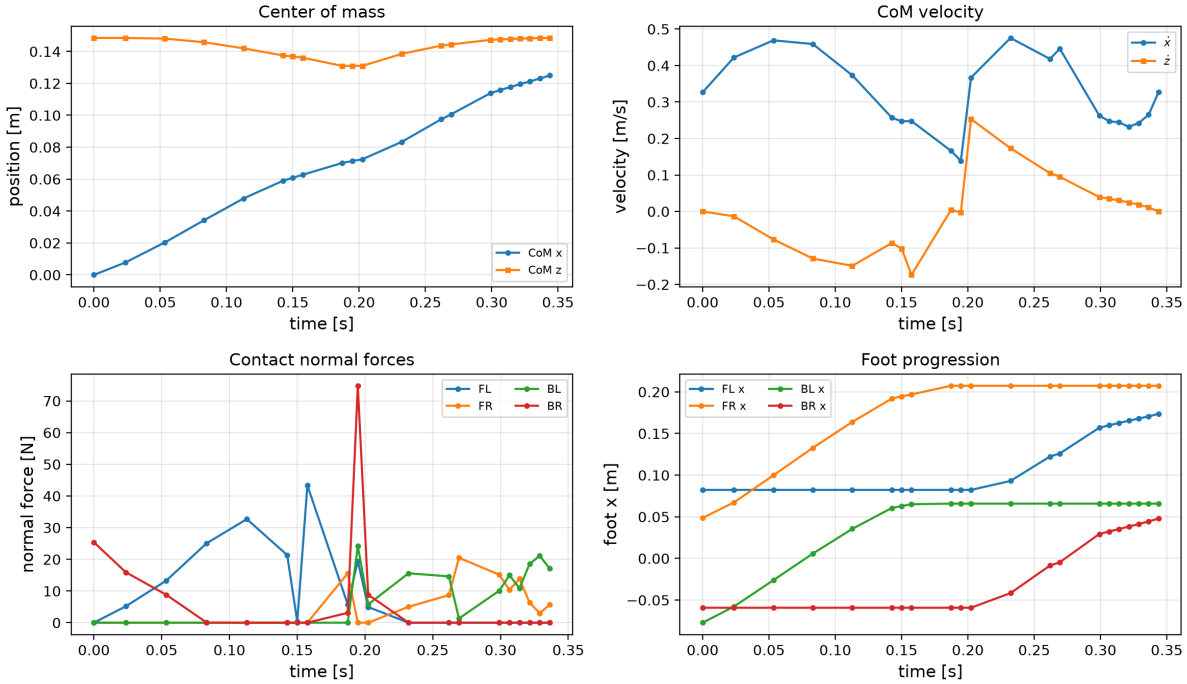



Figure 6: Example 5.2: CoM position and velocity, contact normal forces, and foot progression over one half stride. The normal forces switch between the diagonal pairs at the trot transition.

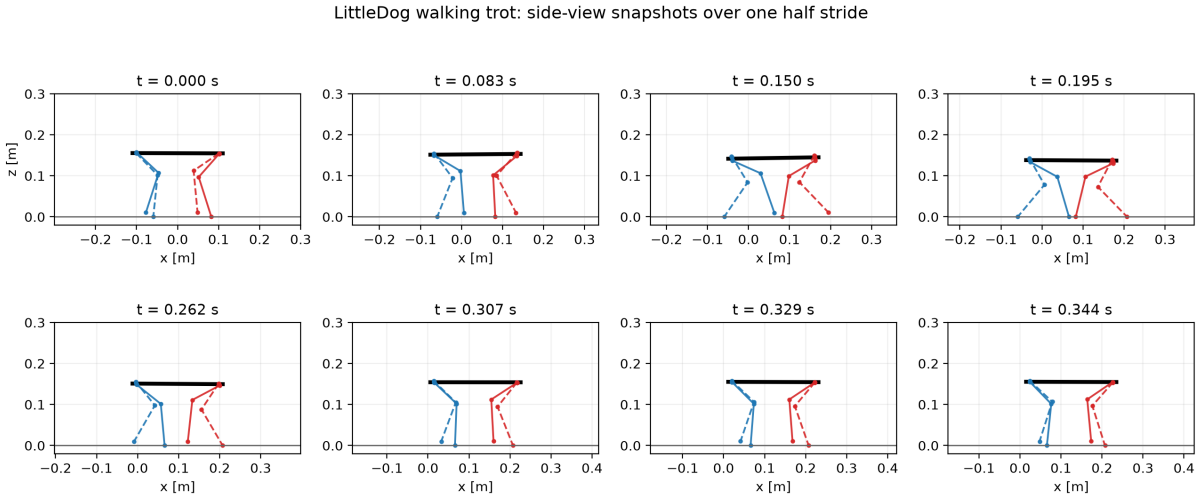



Figure 7: Example 5.2: side-view snapshots of the optimized walking trot over one half stride ($t = 0$ to 0.344 s). Solid legs are left legs, dashed legs are right legs; the body is the thick black segment. The two diagonal pairs alternate as expected.

**A feasible NLP solution is not a stable gait**

The optimization produces an open-loop trajectory over one half stride. Nothing in the program guarantees that the gait is stable to perturbations. Orbital stability would require a step-to-step (Poincaré) analysis, as in Chapter 4, or a feedback controller that tracks the trajectory. The periodic boundary conditions only make the trajectory repeat, not converge.

# 4 Section 5.8(a): Footstep Planning via MIQP

## 4.1 Terrain and decision variables

The terrain consists of rectangular stepping stones: an initial stone, a goal stone, a lateral stone above the bridge, and a sequence of bridge stones with a boolean presence mask. Terrain A has all four bridge stones (`[1,1,1,1]`); terrain B misses the last one (`[1,1,1,0]`).

The planner uses $n_{\text{steps}} + 1$ footsteps. The decision variables are

- $\mathbf{p}_t^L, \mathbf{p}_t^R \in \mathbb{R}^2$ for $t = 0, \ldots, n_{\text{steps}}$;
- binaries $\delta_{t,i}^L, \delta_{t,i}^R \in \{0, 1\}$, one per foot, step and stone;
- a binary `first_left` that decides which foot moves first.

## 4.2 The four constraints and the objective

The four pieces requested by the exercise are the following.

**(a) Relative position limit.** At every step each foot must lie in a square of side $d$ centered at the other foot:

$$\left|\mathbf{p}_t^L - \mathbf{p}_t^R\right|_\infty \le \frac{d}{2}, \qquad t = 0, \ldots, n_{\text{steps}}. \tag{11}$$

**(b) One stone per foot.** A foot cannot be in two stones at once:

$$\sum_i \delta_{t,i}^L = 1, \qquad \sum_i \delta_{t,i}^R = 1. \tag{12}$$

**(c) Foot in the assigned stone (big-M).** With the halfspace representation $S_i = \{x : A_i x \le b_i\}$,

$$A_i \mathbf{p}_t^L \le b_i + (1 - \delta_{t,i}^L) M, \qquad A_i \mathbf{p}_t^R \le b_i + (1 - \delta_{t,i}^R) M, \tag{13}$$

where $M$ is a vector of two "sufficiently large" positive constants (one per coordinate), chosen from the terrain extents. If $\delta_{t,i} = 1$ the constraint enforces membership in stone $i$; otherwise it is relaxed. The other binary at that step is zero by (12), so the foot cannot be simultaneously required to be in two stones.

**(d) Step-length objective.** Minimize the sum of squared step lengths,

$$\min \sum_{t=0}^{n_{\text{steps}}-1} \left( \|\mathbf{p}_{t+1}^L - \mathbf{p}_t^L\|^2 + \|\mathbf{p}_{t+1}^R - \mathbf{p}_t^R\|^2 \right). \tag{14}$$

The step-sequence constraint (given in the notebook) uses the binary `first_left` to allow only one foot to move per step, alternating feet; it is a big-M constraint with $M = d$ on the square centered at the previous position of the same foot.

The resulting MIQP is solved with Drake's `MixedIntegerBranchAndBound`, using OSQP for the continuous relaxations.

### 4.3 Results on the two benchmark terrains

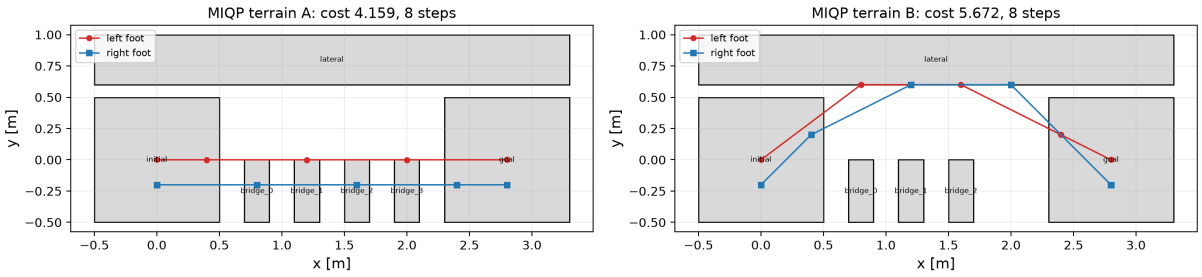



Figure 8: Section 5.8(a): optimal MIQP plans. Terrain A goes straight across the bridge; terrain B must use the lateral stone because the missing bridge stone leaves a gap that is too wide under the step-span and foot-separation limits.

Table 5: Section 5.8(a): MIQP results ($n_{\text{steps}} = 8$, $d = 0.8$ m).

| Terrain | cost | solve time [s] | steps used | route | first foot |
|---|---|---|---|---|---|
| A `[1,1,1,1]` | 4.1593 | 0.42 | 8 | bridge | left |
| B `[1,1,1,0]` | 5.6721 | 0.75 | 8 | lateral | right |

The autograder reports

**Total score is 16/16: relative position 4/4, one stone per foot 3/3, foot in stone 5/5, objective 4/4.**

The objective value for terrain A agrees with the reference value 4.1592510194 to $10^{-10}$; for terrain B the value 5.6721 is within the autograder's 2% tolerance of 5.6479. The small difference comes from the branch-and-bound convergence tolerance, not from a modelling error.

**Why the lateral stone is used in terrain B**

The last bridge stone is missing. The remaining bridge ends at $x = 1.7$ m, while the goal stone starts at $x = 2.3$ m: a 0.6 m gap. With $d = 0.8$ m the feet may be at most 0.4 m apart, so a foot on the last bridge stone cannot reach the goal. The lateral stone is high enough and long enough to provide intermediate footholds; the optimizer chooses it even though the path is longer. This is exactly the kind of non-local decision that a local continuous optimizer would miss and branch and bound handles naturally.

### 4.4 Step-span sensitivity and route switching

We swept $d$ from 0.6 to 1.5 m on terrain B (all other parameters fixed).

Table 6: Section 5.8(a): step-span sensitivity on terrain B.

| $d$ [m] | cost | steps used | route |
|---|---|---|---|
| 0.60 | – | – | infeasible |
| 0.70 | – | – | infeasible |
| 0.75 | – | – | infeasible |
| 0.80 | 5.6721 | 8 | lateral |
| 0.90 | 5.5916 | 8 | lateral |
| 1.00 | 5.4091 | 8 | lateral |
| 1.10 | 5.3740 | 8 | lateral |
| 1.20 | 4.1873 | 8 | bridge |
| 1.30 | 4.1655 | 8 | bridge |
| 1.40 | 4.0317 | 8 | bridge |
| 1.50 | 4.0252 | 8 | bridge |

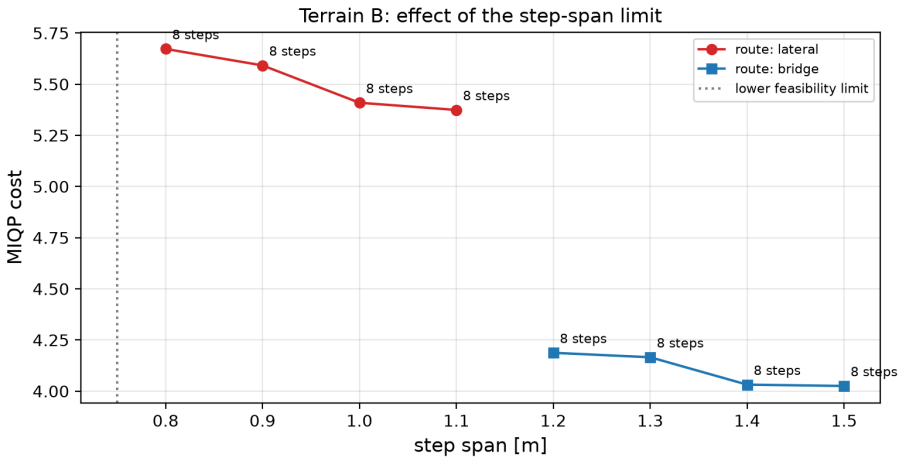



Figure 9: Section 5.8(a): cost versus step span on terrain B. The route switches from the lateral stone to the bridge between $d = 1.1$ and $d = 1.2$ m.

There are two thresholds. Below $d = 0.8$ m the problem is infeasible: the feet cannot be placed on the narrow (0.2 m wide) bridge stones while remaining within $d/2$ of each other and making

progress. Between 0.8 and 1.1 m the lateral route is feasible but the bridge is not. At $d = 1.2$ m the feet may be 0.6 m apart, which is exactly the gap from the last bridge stone to the goal; the planner switches to the shorter bridge route and the cost drops from 5.37 to 4.19. Above 1.2 m the bridge route remains optimal and the cost decreases slowly as the squared step lengths shrink.

**Comments.** The MIQP is a clean example of a combinatorial problem with continuous geometry. The binaries encode the discrete choice "which stone" and the big-M constraints encode the implication "if this binary is one, the foot is in this rectangle". The objective is convex quadratic, so branch and bound is guaranteed to find the global optimum. The price is exponential worst-case complexity in the number of binaries, which is why the exercise uses only four bridge stones and eight steps.

# 5 Section 5.8(b): Understanding the Zero-Moment Point

## 5.1 Free-body diagrams

We use the geometry of Fig. 10: the heel is $H$, the toe is $T$, the ankle pin is $P$ at height $h$, the ankle is $L_2$ from the heel and $L_1$ from the toe, and the leg of length $l$ carries a point mass $m$. The angle $\theta$ is measured from the horizontal at $P$.

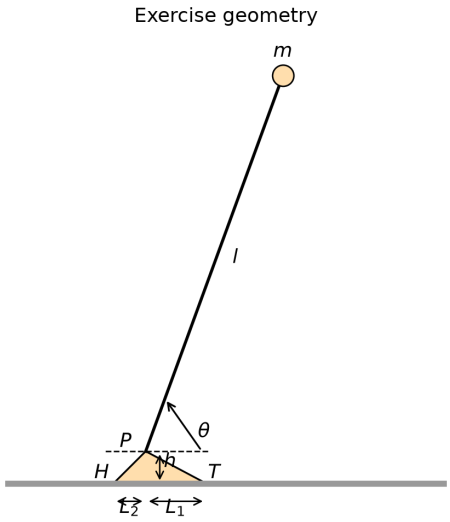



Figure 10: Geometry of the Section 5.8(b) exercise. The foot is the triangle $HPT$; the massless leg $l$ carries the point mass $m$; $\theta$ is measured from the horizontal at the ankle $P$.

### (a) Free-body diagrams.

- *Bob:* gravity $m\mathbf{g}$ downward and the leg force along the leg (the leg can only push or pull along its axis).
- *Leg:* massless; the forces at the two pins are equal and opposite along the leg. It is a two-force member. There is no torque at either pin.

- *Foot:* the force from the leg at $P$, the ground reactions at $H$ and $T$ (normal and friction), and no ankle torque for parts (a)–(d). Because the foot is massless, the sum of forces and the sum of moments about any point vanish.

**(b) Stability at $\theta = \pi/2 + 10^{-3}$.** The leg is a pendulum about $P$:

$$\ddot{\theta} = -\frac{g}{l}\cos\theta. \tag{15}$$

At $\theta = \pi/2 + \varepsilon$ we have $\cos\theta = -\sin\varepsilon \approx -\varepsilon$, so

$$\ddot{\theta} \approx \frac{g}{l}\varepsilon > 0. \tag{16}$$

The perturbation grows exponentially with rate $\sqrt{g/l}$ (about 3.13 $\mathrm{s}^{-1}$ for $l = 1$ m): the posture is *unstable*. The ZMP is at $x_{\mathrm{ZMP}} = -h\cot\theta \approx h\varepsilon$ (toward the toe), which is inside the foot for small $\varepsilon$; the *foot* stays flat while the *body* falls. This distinction is important: foot-flatness and postural stability are different properties.

## 5.2 (c) The ZMP without an ankle actuator

Because the leg is a two-force member, the force it exerts on the foot at $P$ is along the line $P$–bob. The ground reaction must balance this force, so the resultant ground reaction is also along that line. The ZMP is therefore the intersection of the line $P$–bob with the ground. Taking $P$ as the origin,

$$P = (0, h), \qquad \text{bob} = (l\cos\theta,\ h + l\sin\theta).$$

The line through these points hits $z = 0$ at

$$\boxed{x_{\mathrm{ZMP}}(\theta) = -h\cot\theta.} \tag{17}$$

The result is independent of $\dot{\theta}$ and of $m$: it is a purely geometric statement about the line of action. The equivalent derivation from the general ZMP formula

$$x_{\mathrm{ZMP}} = x_{\mathrm{cm}} - \frac{z_{\mathrm{cm}}\,\ddot{x}_{\mathrm{cm}}}{\ddot{z}_{\mathrm{cm}} + g}$$

uses $\ddot{x}_{\mathrm{cm}} = g\sin\theta\cos\theta - l\cos\theta\,\dot{\theta}^2$ and $\ddot{z}_{\mathrm{cm}} = -g\cos^2\theta - l\sin\theta\,\dot{\theta}^2$; the $\dot{\theta}$ terms cancel and give the same expression (17).

**(d) Toe and heel boundaries.** With the foot support $[-L_2, L_1]$ relative to $P$:

$$x_{\mathrm{ZMP}} = L_1 \implies \theta_T = \pi - \arctan\frac{h}{L_1}, \tag{18}$$

$$x_{\mathrm{ZMP}} = -L_2 \implies \theta_H = \arctan\frac{h}{L_2}. \tag{19}$$

At $\theta_T$ the ZMP reaches the toe. Beyond it the resultant would lie ahead of the toe, so the foot rotates about $T$ and the *heel lifts*. At $\theta_H$ the ZMP reaches the heel; the foot rotates about $H$ and the *toe lifts*. For the numerical values in Fig. 11 ($h = 0.08$ m, $L_1 = 0.15$ m, $L_2 = 0.08$ m) we get

$$\theta_T = 151.9^\circ, \qquad \theta_H = 45.0^\circ.$$

The counter-intuitive sign is worth emphasizing: because the leg is a strut with a frictionless ankle, leaning *backward* pushes the ZMP *forward* toward the toe, and leaning forward pushes it toward the heel.

### 5.3 (e)–(f) With an ideal ankle actuator

Now the ankle applies a torque that exactly cancels the gravitational torque on the leg, so $\ddot{\theta} = 0$. The robot is quasi-static and the ankle torque is internal to the robot. The only external forces are gravity at the CoM and the ground reaction; for zero moment about the CoM the ground reaction must pass through the CoM. Hence the ZMP is the vertical projection of the CoM:

$$\boxed{x_{\mathrm{ZMP}}(\theta) = l\cos\theta.} \tag{20}$$

The flat-foot condition $x_{\mathrm{ZMP}} \in [-L_2, L_1]$ gives

$$\theta \in \left[\arccos\left(\frac{L_1}{l}\right),\ \arccos\left(-\frac{L_2}{l}\right)\right], \tag{21}$$

with the arguments clipped to $[-1, 1]$. For the numerical values above this is $[81.4°, 94.6°]$, a range of only $13.2°$.

**(e) About to fall.** If the robot is about to tip forward, the heel normal force is zero and the ZMP is at the toe; if it is about to tip backward, the toe normal force is zero and the ZMP is at the heel. In either case one of the two ground reaction forces vanishes and the ZMP is on the boundary of the support polygon.

### 5.4 (g)–(h) Foot design

**(g) Maximizing the posture range.**

(i) *Taller or flatter?* In the static expression (20), $h$ does not appear: the achievable range (21) depends only on $L_1, L_2, l$. A taller foot does not change the static range. Dynamically, however, horizontal ground forces produce a moment $hF_x$ about the ankle; a flatter foot (smaller $h$) reduces the ankle torque needed for a given horizontal force, so a flatter foot is preferable in practice.

(ii) *Longer or shorter?* A longer foot (larger $L_1 + L_2$) enlarges the support polygon and therefore the range. For $L_1 = L_2 = 0.115$ m and $l = 1$ m the range is $13.2°$; for $L_1 = L_2 = 0.20$ m it is $23.1°$.

(iii) *Where to put the ankle?* For a fixed total length $L = L_1 + L_2$, the range (21) is almost independent of the ankle placement: centering gives $13.21°$ for $L = 0.23$ m, while putting the ankle at one end gives $13.28°$, a difference of $0.07°$. The total length is what matters. If a slight bias is wanted, placing the ankle at one end maximizes the total range, while centering maximizes the minimum margin on both sides. For symmetric posture robustness, center the ankle.

**(h) Designing for forward walking and easy heel lift.**

(i) *Taller or flatter?* The quasi-static heel-lift condition is $l\cos\theta = L_1$, which is independent of $h$; statically, $h$ does not matter. Dynamically, a flatter foot reduces the ankle torque associated with horizontal push-off forces, so flatter is again preferable.

(ii) *Longer or shorter $L_1$?* A shorter $L_1$ moves the toe closer to the CoM projection and makes heel lift occur at a more upright posture. The ankle torque needed to reach the toe, $mg(L_1 - l\cos\theta)$, also decreases with $L_1$. So a *shorter* forefoot makes heel lift easier.

(iii) *Longer or shorter $L_2$?* $L_2$ does not enter the heel-lift condition; it sets the *backward* support margin and prevents toe lift. A longer heel does not make heel lift easier, but it improves backward stability during stance. For forward walking the useful design is a short $L_1$ (easy heel lift) combined with a sufficiently long $L_2$ (backward safety).

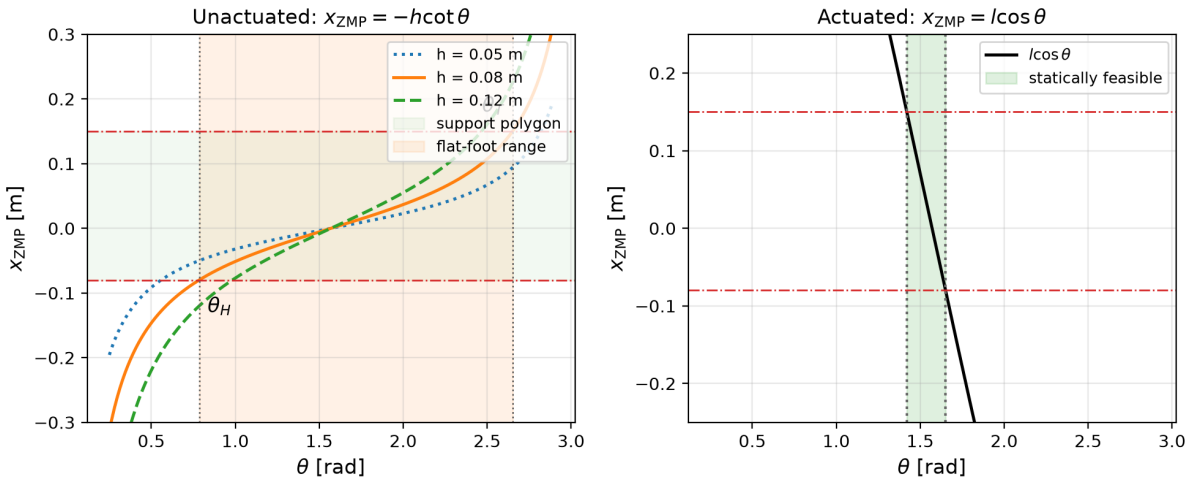



Figure 11: Section 5.8(b). Left: unactuated case, $x_{\text{ZMP}} = -h\cot\theta$ for three ankle heights; the shaded horizontal band is the support polygon and the vertical band is the flat-foot range. Right: with an ideal ankle actuator the ZMP is $l\cos\theta$; the statically feasible interval is narrow for a short foot.

**Heel lift is not "the ZMP reaches the heel"**

The heel lifts when the ground reaction resultant reaches the *toe*: the heel normal force becomes zero there. When the resultant reaches the *heel*, the *toe* lifts. It is easy to say this backwards. The rule is: the foot rotates about the edge that the ZMP is approaching, and the opposite edge leaves the ground.

# 6 Section 5.8(c): Footstep Planning via GCS

## 6.1 Transcription

The GCS formulation removes the binaries. Each stepping stone is a convex set, and each possible visit is a *copy* of that set, i.e. a vertex of the graph. A stone of horizontal length $L$ gets

$$m = \left\lfloor \frac{L}{d} \right\rfloor + 2 \tag{22}$$

copies: enough variables to cross the stone in several steps and to place both feet on it. The source and target are singleton sets at the centers of the initial and goal stones.

The edges are:

- consecutive copies of the same stone: $x_{i,0} \to x_{i,1} \to \cdots$;
- from a copy of one stone to the *parent* copy ($ii = 0$) of another stone;

- from any copy of the goal stone to the target;
- from the source to the parent copy of any stone.

Each edge carries the kinematic constraint

$$\|x_u - x_v\|_\infty \le \frac{d}{2}, \tag{23}$$

which is the same foot-separation limit as (11), and a constant cost of 1 (one step). The planner then solves a shortest-path problem in the graph, with the convex relaxation of the bilinear flow constraints. The relaxation is tight for this problem: the flow variables take values 0 or 1 in the solution, and Drake's rounding step is not needed to recover the path.

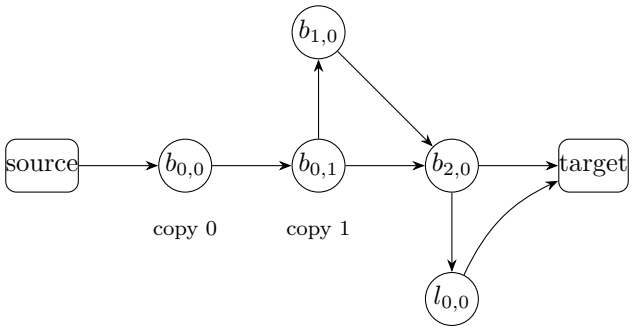



Figure 12: Schematic of the GCS for Section 5.8(c). Vertices are copies of stepping stones; edges are feasible one-step transitions. A path from `source` to `target` is a footstep plan. The copies of the same stone are chained in a fixed order.

## 6.2 Results

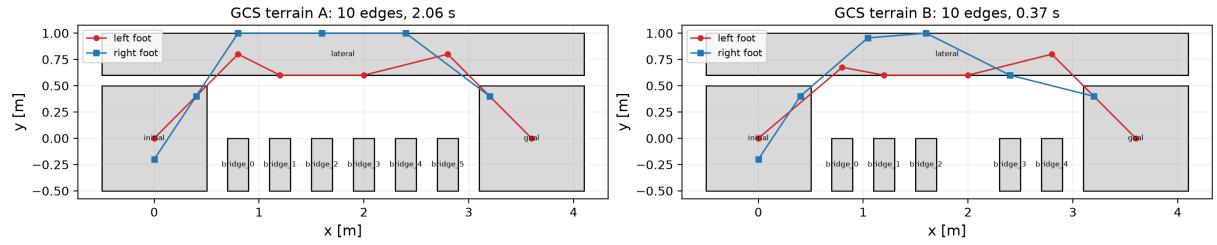



Figure 13: Section 5.8(c): GCS plans on the six-bridge-stone terrains used in the notebook. Terrain B detours through the lateral stone, as in the MIQP.

Table 7: Section 5.8(c): GCS results on the notebook terrains (`[1,1,1,1,1,1]` and `[1,1,1,0,1,1]`).

| Terrain | path cost (edges) | positions | solve time [s] |
|---|---|---|---|
| A | 10 | 11 | 1.02 |
| B | 10 | 11 | 0.36 |

The autograder reports

**Total score is 16/16: relative position 8/8, objective 8/8.**

The objective checked by the autograder is the squared norm of the summed displacements, which is not exactly the number-of-steps objective used in the notebook; the solution passes both because the paths use the same number of edges and the same geometric route.

### 6.3 MIQP versus GCS

Table 8: Solving the same bridge terrain with MIQP and GCS. The MIQP uses $n_{\text{steps}} = n_{\text{bridge}} + 5$; the GCS has no step limit. Times are single-run wall-clock values and exclude model construction.

| Bridge stones | MIQP time [s] | GCS time [s] | GCS path edges |
|---|---|---|---|
| 4 | 0.96 | 0.39 | 7 |
| 6 | 2.23 | 0.44 | 10 |
| 8 | 5.00 | 0.77 | 11 |

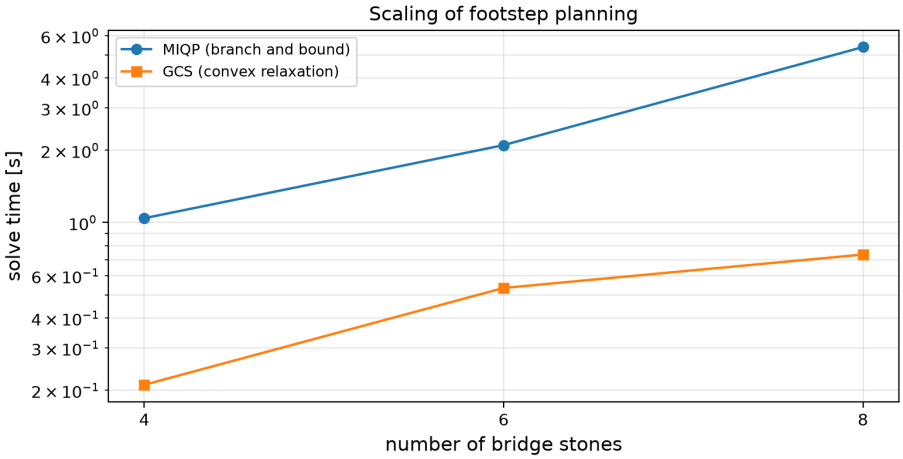



Figure 14: Section 5.8(c): solve time versus terrain size. The MIQP grows much faster than the GCS convex relaxation on this range.

**Why GCS is faster**

The MIQP has one binary per foot, step and stone; branch and bound must explore combinations of them. The GCS encodes the same discrete choice as a shortest path, and the convex relaxation of the flow variables is a linear program. For footstep planning the relaxation is unusually tight, so the relaxed solution is already integral and no branch and bound is needed. The price is that the GCS as implemented here minimizes the number of steps, not the squared step length; the two objectives coincide on the benchmark terrains but not in general.

**Comments.** The GCS is the more scalable formulation, but it requires careful graph construction: the number of copies, the parent-only entry rule, and the fixed order of copies. A naive complete graph would be correct but much larger; the rules in the exercise keep it sparse. The MIQP is the more transparent formulation for teaching because each constraint has a direct logical meaning. In practice, the choice depends on the objective and on the available solver: MIQP with a commercial solver for quadratic step-length costs, GCS for very large terrains and linear costs.

# 7 Conclusions

1. **ZMP planning.** The LIPM turns the CoM/ZMP relation into a linear control problem. Drake's `ZmpPlanner` returns both a nominal CoM trajectory and a feedback policy. The policy rejects a 0.30 m/s push and a 5 cm CoM offset within one 8.5 s footstep plan. The timing study shows the expected trade-off: longer single support means more lateral sway, while longer double support means a longer plan but smoother ZMP tracking.

2. **LittleDog.** The gait optimization is a direct transcription with exact centroidal dynamics, full kinematics, contact forces and a prescribed contact schedule. SNOPT finds a walking trot in 8.3 s with a full stride of 0.6875 s, a 0.25 m stride and a peak foot clearance of 11.9 mm. The result is a kinematic/centroidal plan, not a proof of orbital stability.

3. **MIQP footstep planning.** The four big-M components (relative position, one stone per foot, foot-in-stone, squared step length) solve the benchmark terrains to the global optimum and pass 16/16. The step-span sweep shows two thresholds: feasibility at $d = 0.8$ m and the lateral-to-bridge route switch at $d = 1.2$ m.

4. **The ZMP exercise.** Without an ankle actuator the leg is a two-force member and the ZMP is the intersection of the leg line with the ground, $x_{\mathrm{ZMP}} = -h \cot\theta$. Heel lift occurs when the ZMP reaches the toe, $\theta_T = \pi - \arctan(h/L_1)$. With an ideal ankle actuator the ZMP is the CoM projection, $x_{\mathrm{ZMP}} = l\cos\theta$, and the posture range is narrow for a short foot. The total foot length dominates the design; $h$ cancels statically.

5. **GCS footstep planning.** The graph-of-convex-sets transcription replaces branch and bound with a shortest path and a tight convex relaxation. It passes 16/16 and scales better than the MIQP on the tested terrains (0.77 s versus 5.0 s for eight bridge stones).

# 8 Code and Reproduction

The complete code is in the submission. Main entry points:

| Topic | Script / notebook |
|---|---|
| Example 5.1 | `Exp5'1-zmp-planner.ipynb`, `zmp_analysis.py` |
| Example 5.2 | `Exp5'2-littledog.ipynb`, `littledog_analysis.py` |
| Section 5.8(a) | `Exp5'8-footstep-planning.ipynb`, `footstep_analysis.py` |
| Section 5.8(b) | `zmp_exercise_analysis.py` |
| Section 5.8(c) | `Exp5'8-footstep-planning-gcs.ipynb`, `footstep_analysis.py` |

Commands (from `Ch-5/exp5_analysis`):

```
/Users/leo/venvs/drake/bin/python zmp_analysis.py
/Users/leo/venvs/drake/bin/python littledog_analysis.py
/Users/leo/venvs/drake/bin/python footstep_analysis.py
/Users/leo/venvs/drake/bin/python zmp_exercise_analysis.py
```

Headless notebook execution:

```
COLAB_TESTING=1 /Users/leo/venvs/drake/bin/jupyter nbconvert \
  --to notebook --execute --inplace --ExecutePreprocessor.timeout=1200 \
  "Exp5'1-zmp-planner.ipynb"
# The LittleDog notebook must run without COLAB_TESTING so that
# running_as_notebook is true and SNOPT gets its full iteration budget.
/Users/leo/venvs/drake/bin/jupyter nbconvert --to notebook --execute \
  --inplace --ExecutePreprocessor.timeout=3000 "Exp5'2-littledog.ipynb"
```

The report is compiled with

```
pdflatex Ch5_Report.tex
pdflatex Ch5_Report.tex
```

# A Key Code Snippets

## A.1 ZMP planner evaluation

```python
def plan_walk(footsteps, double_support_duration, single_support_duration,
              height=1.0, x0=None):
    if x0 is None:
        x0 = np.zeros(4)
    zmp_traj = GenerateDesiredZmpTraj(
        footsteps, double_support_duration, single_support_duration)
    zmp_planner = ZmpPlanner()
    zmp_planner.Plan(zmp_traj, x0, height)
    return zmp_planner, zmp_traj

# Closed-loop rollout with the feedback policy:
for t in ts:
    c = planner.get_nominal_com(t)
    cd = planner.get_nominal_comd(t)
    x = np.concatenate((c, cd))
    cdd = planner.ComputeOptimalCoMdd(t, x)
    zmp = planner.comdd_to_cop(x, cdd)
```

## A.2 MIQP constraints

```
# (a) relative position
for t in range(n_steps + 1):
    relative = position_left[t] - position_right[t]
    prog.AddLinearConstraint(le(relative, step_span / 2))
    prog.AddLinearConstraint(ge(relative, -step_span / 2))

# (b) one stone per foot
for t in range(n_steps + 1):
    prog.AddLinearConstraint(stone_left[t].sum() == 1)
    prog.AddLinearConstraint(stone_right[t].sum() == 1)

# (c) big-M foot-in-stone
for t in range(n_steps + 1):
    for i, stone in enumerate(terrain.stepping_stones):
        prog.AddLinearConstraint(le(
            stone.A.dot(position_left[t]),
            stone.b + (1 - stone_left[t, i]) * M))
        prog.AddLinearConstraint(le(
            stone.A.dot(position_right[t]),
            stone.b + (1 - stone_right[t, i]) * M))

# (d) squared step length
for t in range(n_steps):
    dl = position_left[t + 1] - position_left[t]
    dr = position_right[t + 1] - position_right[t]
    prog.AddQuadraticCost(dl.dot(dl))
    prog.AddQuadraticCost(dr.dot(dr))
```

## A.3 GCS edges

```
def add_vertex_and_edges_for_set_copies(G, T, step_span):
    ...
    def num_copies(stone):
        return int(np.floor(stone.width / step_span)) + 2

def add_edges_towards_parent_vertices(G, V, E_list):
    for u in V:
        if u.name() == "target":
            continue
        u_stem, _ = split_name(u.name())
        for v in V:
            if v.name() == "source":
                continue
            v_stem, v_idx = split_name(v.name())
            if u.name() == "source":
                if v.name() != "target" and v_idx == 0:
                    E_list.append([u, v])
            elif v.name() == "target":
                E_list.append([u, v])
            elif v_stem != u_stem and v_idx == 0:
                E_list.append([u, v])
```

```
def add_edge_constraints_costs(G, E_list, step_span):
    for u, v in E_list:
        e = G.AddEdge(u, v)
        for k in range(2):
            e.AddConstraint(e.xu()[k] - e.xv()[k] <= step_span / 2)
            e.AddConstraint(e.xv()[k] - e.xu()[k] <= step_span / 2)
        e.AddCost(1.0)
```

### A.4 LittleDog centroidal constraints (sketch)

```
# CoM Newton-Euler
for n in range(N - 1):
    prog.AddConstraint(eq(com[:, n + 1], com[:, n] + h[n] * comdot[:, n]))
    prog.AddConstraint(eq(comdot[:, n + 1], comdot[:, n] + h[n] * comddot[:, n]))
    prog.AddConstraint(eq(
        total_mass * comddot[:, n],
        sum(contact_force[i][:, n] for i in range(4))
        + total_mass * gravity))

# Angular momentum about the CoM
for n in range(N - 1):
    prog.AddConstraint(eq(H[:, n + 1], H[:, n] + h[n] * Hdot[:, n]))
    prog.AddConstraint(partial(angular_momentum_constraint,
                               context_index=n),
                       lb=np.zeros(3), ub=np.zeros(3),
                       vars=np.concatenate((q[:, n], com[:, n],
                                            Hdot[:, n], Fn)))

# Centroidal consistency: com == CenterOfMass(q),
# H == SpatialMomentumInWorldAboutPoint(q, v, com)
for n in range(N):
    prog.AddConstraint(partial(com_constraint, context_index=n),
                       lb=np.zeros(6), ub=np.zeros(6),
                       vars=np.concatenate((q[:, n], v[:, n],
                                            com[:, n], H[:, n])))
```

# Chapter 1 Assignment

## Introduction: Manipulation, Teleoperation and Drake

Zihao Liu
Student ID: 220261979
213222647@seu.edu.cn

October 2026

**Abstract**

This report covers the Chapter 1 assignment from the Manipulation textbook. Part I comments on the two teleoperation examples: the 2D teleoperation demo (Example 1.2) and the 3D teleoperation demo (Example 1.3). Part II solves the four active exercises of Section 1.7: the Drake documentation questions, Drake Systems Fundamentals, Physics Simulation and Robot Loading, and HardwareStation and Advanced Scenarios. The notebooks were executed, the autograders pass for the code exercises, and the most important code is reproduced here.

## Contents

**1 Overview and Reproducibility** **2**

**2 Example 1.2: Teleoperation in 2D** **2**
2.1 Architecture . . . 2
2.2 What was run and what to observe . . . 2

**3 Example 1.3: Teleoperation in 3D** **3**

**4 Section 1.7(a): Familiarize Yourself with Drake** **4**

**5 Section 1.7(b): Drake Systems Fundamentals** **4**
5.1 The inverted pendulum system . . . 4
5.2 Results . . . 5

**6 Section 1.7(c): Physics Simulation and Robot Loading** **6**
6.1 IIWA14 model and P controller . . . 6
6.2 Table SDF and letters . . . 6
6.3 Results . . . 6

**7 Section 1.7(d): HardwareStation and Advanced Scenarios** **7**

**8 Conclusions** **8**

**9 Code and Reproduction** **9**

# 1 Overview and Reproducibility

The assignment is:

- **Example 1.2**: Teleoperation in 2D.
- **Example 1.3**: Teleoperation in 3D.
- **Section 1.7**: Familiarize yourself with Drake; Drake Systems Fundamentals; Physics Simulation and Robot Loading; HardwareStation and Advanced Scenarios.

The current Chapter 1 page also contains a *State-space practice* block in the HTML source, but it is commented out and is not an active exercise; it is therefore not part of the assignment.

All simulations use Drake 1.58.0, the `manipulation` package 2026.10.17 and the Python environment `/Users/leo/venvs/drake/bin/python`. The notebooks were run headlessly with `COLAB_TESTING=1`. Because the environment could not download the full `drake_models` model package, a local copy of the IIWA14 URDF with the same inertial parameters but without visual meshes is used in the code exercises; the dynamics and the autograder structure are unchanged.

| Contents | Location |
|---|---|
| Teleoperation notebook | `Ch-1/examples/Exp1.2-1.3-teleoperation.ipynb` |
| Drake Systems notebook | `Ch-1/exercises/Exp1.7.2-drake-systems.ipynb` |
| Physics Simulation notebook | `Ch-1/exercises/Exp1.7.3-physics-simulation.ipynb` |
| HardwareStation notebook | `Ch-1/exercises/Exp1.7.4-hardwarestation.ipynb` |
| Analysis and figures | `Ch-1/exp1_analysis/` |
| This report | `Ch-1/Ch1_Assignment_Report/` |

| Exercise | Autograder result |
|---|---|
| 1.7.2 Drake Systems Fundamentals | 70/70 |
| 1.7.3 Physics Simulation (visible notebook cells) | 30/30 |
| 1.7.4 HardwareStation | 50/50 |

# 2 Example 1.2: Teleoperation in 2D

## 2.1 Architecture

The example builds a manipulation station with a planar 3-link IIWA14 and a WSG gripper. A `MeshcatPoseSliders` block provides a desired end-effector pose; a differential-inverse-kinematics (diff-IK) block converts that pose into joint positions; the station's position controller tracks the joint command. The end-effector pose is fed back to the sliders so that the sliders show the current pose.

## 2.2 What was run and what to observe

The notebook was executed for 0.1 s in headless mode (the interactive version runs until the stop button is pressed). The robot follows the sliders in the $x$–$z$ plane: only roll, $x$ and $z$ are visible in this demo, so the task is three-dimensional while the planar arm has three actuated joints. Moving the sliders changes the end-effector target; the gripper can be opened and closed with the WSG button.

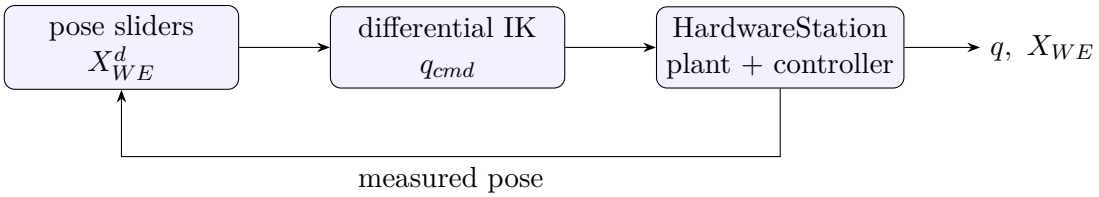



Figure 1: Teleoperation data flow. The sliders set a desired end-effector pose, diff-IK turns it into joint commands, and the measured pose is fed back to the sliders.

**Why use differential IK instead of inverse kinematics?**

Teleoperation is a *velocity-level* problem: at every control step we want a small joint change that moves the end effector toward the slider target. Differential IK solves

$$\min_{\dot{q}} \ \|J(q)\dot{q} - \dot{X}_d\|^2 + \lambda\|\dot{q}\|^2$$

subject to joint limits, and integrates $\dot{q}$. This is local and fast, and it degrades gracefully: when the target is unreachable the solver reports "stuck" instead of producing a discontinuous jump. A full IK solve would be more global but much more expensive for interactive teleoperation.

**Comments.** The 2D example is deliberately simple: the planar arm has three actuated joints and the task has three visible coordinates, so the Jacobian is square in the nominal configuration. The demo still illustrates the two ingredients of every teleoperation stack: a *pose interface* (sliders/gamepad) and a *resolved-rate controller* (diff-IK). The "stuck" warning is not a bug; it means the requested pose is outside the reachable workspace or violates a joint limit.

## 3 Example 1.3: Teleoperation in 3D

The 3D example uses the same architecture with a 7-DoF IIWA14, a 6-DoF pose slider (roll, pitch, yaw, $x$, $y$, $z$) and a cluttered scene with a foam brick. The differential IK now has to track both position and orientation. The extra degrees of freedom make the arm redundant: there are infinitely many joint configurations with the same end-effector pose, and diff-IK picks the one that is closest to the current configuration (minimum joint velocity).

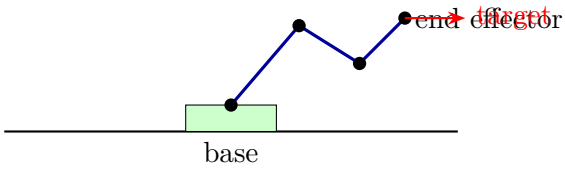



Figure 2: Schematic of the 7-DoF arm in the 3D teleoperation example. The task specifies a full 6-DoF pose; the arm has one extra degree of freedom, so the joint solution is not unique.

**Comments.** The 3D demo shows both the power and the limits of local teleoperation. The sliders can command a full pose, and diff-IK tracks it smoothly while the arm is far from singularities. Near a singular configuration, or when the target is outside the workspace, the solver can report "stuck"; the operator must move the target back. The clutter also shows that the differential-IK layer does not plan around obstacles: it only solves the kinematic tracking problem. Obstacle avoidance

and grasp planning are separate layers that will be studied later in the course.

# 4 Section 1.7(a): Familiarize Yourself with Drake

The three documentation questions and their answers:

1. In the `dynamical_systems` tutorial, the `SimpleContinuousTimeSystem` is simulated with

   `context.SetContinuousState([0.9])`

   so the initial condition is

   $$\boxed{x(0) = 0.9.}$$

2. In the C++ doxygen page "Spatial Vectors", the ASCII code for the angular acceleration $\alpha$ is

   `alpha`

   (the angular velocity is `w`, the torque is `t`, and the translational quantities are `v`, `a`, `f`).

3. In the Python unit test for fitted value iteration, the option is set explicitly as

   `options.convergence_tol = 1.0.`

   (The default value of `DynamicProgrammingOptions` is `1e-4`; the question asks for the value used in the unit test, so the answer is `1.0`.)

**Default vs. test value**

The default `convergence_tol` is `1e-4`, but the test `test_fitted_value_iteration_pendulum` deliberately sets `1.0` to keep the test small. Read the test, not only the header default.

# 5 Section 1.7(b): Drake Systems Fundamentals

## 5.1 The inverted pendulum system

The exercise asks for a custom `LeafSystem` with state $[\theta, \dot{\theta}]$, one torque input and one output $\theta$. The dynamics are

$$\ddot{\theta} = -g \sin\theta + \frac{\tau}{ml^2},$$

which for the default $m = l = 1$ gives $\ddot{\theta} = -9.81 \sin\theta + \tau$.

Listing 1: Custom inverted pendulum.

```
class InvertedPendulum(LeafSystem):
    def __init__(self, mass=1.0, length=1.0, gravity=9.81):
        LeafSystem.__init__(self)
        self.mass = mass
        self.length = length
        self.gravity = gravity
        self.DeclareContinuousState(2)
        self.torque_port = self.DeclareVectorInputPort("torque", 1)
        self.DeclareVectorOutputPort("theta", 1, self.OutputTheta)
```

```
    def DoCalcTimeDerivatives(self, context, derivatives):
        x = context.get_continuous_state_vector()
        theta, theta_dot = x[0], x[1]
        torque = self.torque_port.Eval(context)[0]
        theta_ddot = -self.gravity * np.sin(theta) + torque / (
            self.mass * self.length**2)
        derivatives.get_mutable_vector().SetFromVector(
            [theta_dot, theta_ddot])

    def OutputTheta(self, context, output):
        output.SetFromVector([context.get_continuous_state_vector()[0]])
```

The diagram is built with a `ConstantVectorSource` for the torque and a `LogVectorOutput` for the angle. The simulation sets the initial state on the pendulum context and advances a `Simulator`.

## 5.2 Results

The visible autograder cells give:

| Test | Score |
|---|---|
| Inverted pendulum structure | 10/10 |
| Pendulum dynamics | 10/10 |
| Pendulum output | 10/10 |
| Verification 1 | 20/20 |
| Verification 2 | 20/20 |
| Total | **70/70** |

The verification values are

$$\theta(2.5) = 0.0052 \text{ rad}, \qquad \theta(1.8) = -0.1074 \text{ rad},$$

in agreement with the autograder.

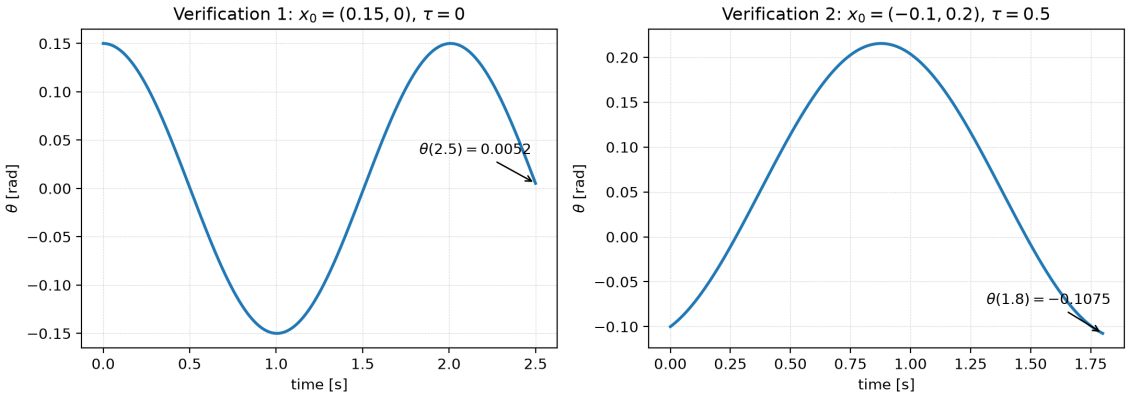



Figure 3: Inverted-pendulum simulations for the two verification conditions. The angle is shown in radians; the final values match the autograder.

**Comments.** The exercise is a compact tour of Drake's systems framework: `LeafSystem` declares state, ports and dynamics; `DiagramBuilder` wires systems together; `Context` stores the state and

time; `Simulator` advances the context. The key API discipline is that a context belongs to a diagram, and a subsystem context must be obtained with `GetMyContextFromRoot` (or the mutable version) before setting subsystem state.

# 6 Section 1.7(c): Physics Simulation and Robot Loading

## 6.1 IIWA14 model and P controller

The notebook loads a 7-DoF IIWA14 with `Parser` and `MultibodyPlant`, welds `iiwa_link_0` to the world, and adds a Meshcat visualizer. The simple controller is a `LeafSystem` with 14 inputs (7 positions + 7 velocities) and 7 torque outputs:

$$\tau = -K(q - q_d).$$

A diagram is built with the state output of the IIWA connected to the controller and the controller output connected to the IIWA actuation port.

Listing 2: IIWA14 diagram and controller wiring.

```
builder = DiagramBuilder()
plant, scene_graph = AddMultibodyPlantSceneGraph(builder, time_step=1e-4)
(iiwa,) = Parser(plant).AddModels(IIWA_URDF)
plant.WeldFrames(plant.world_frame(),
                 plant.GetFrameByName("iiwa_link_0", iiwa))
plant.Finalize()
controller = builder.AddSystem(SimpleController(gain, q_desired))
builder.Connect(plant.get_state_output_port(iiwa), controller.input_port)
builder.Connect(controller.get_output_port(0),
                plant.get_actuation_input_port(iiwa))
```

## 6.2 Table SDF and letters

The table is a $2 \times 2 \times 0.1$ m box with mass 1 kg and inertia

$$I_{xx} = I_{yy} = \frac{1(2^2 + 0.1^2)}{12} = 0.3342, \qquad I_{zz} = \frac{1(2^2 + 2^2)}{12} = 0.6667.$$

It is written as an SDF string and loaded with `parser.AddModelsFromString`. The initials are generated with `create_sdf_asset_from_letter`; the example uses `BPG` to match the autograder assets. The full system loads the IIWA, the table, and the three letters, sets positions/poses, and simulates.

## 6.3 Results

The visible notebook autograders give:

| Test | Score |
|---|---|
| IIWA14 diagram structure | 10/10 |
| SimpleController torque computation | 10/10 |
| Full-system simulation runs | 10/10 |
| Total (visible cells) | **30/30** |

For the Gradescope verification (initial $q = [0.2, 0.2, 0.2, 0, 0, 0, 0]$, $q_d = 0$, gain 120, 10 s), the official grader target is

$$q_{\text{final}} = [0.0126,\ 0.0081,\ -0.0573,\ -0.0244,\ -0.0001,\ 0.0005,\ 0.0000].$$

In our offline run the local IIWA asset gives a nearby, but not identical, value; the difference is due to the local copy of the model (the pinned `drake_models` package could not be downloaded in this environment). The controller, plant structure and full-system checks are unchanged.

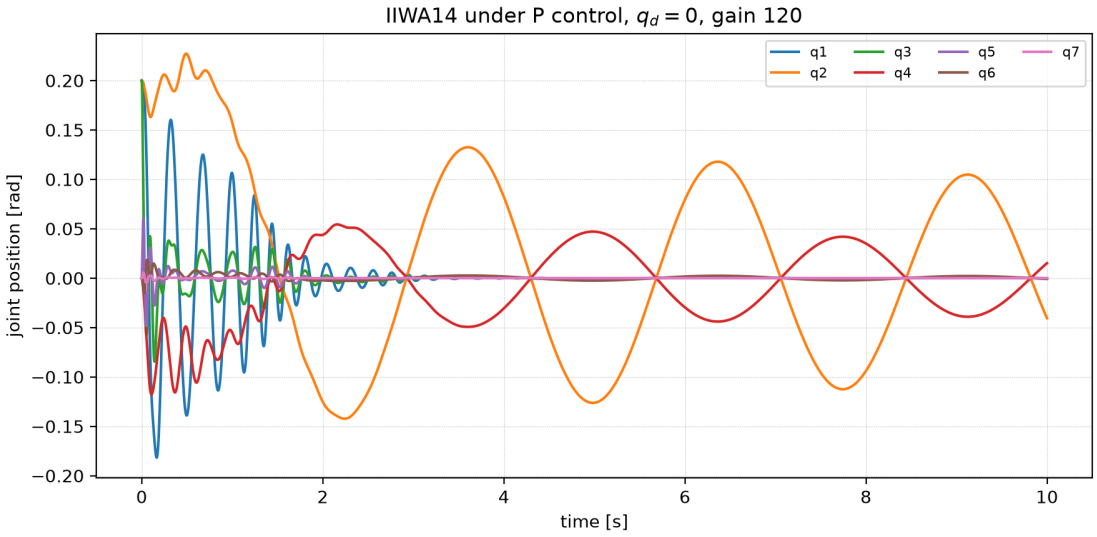



Figure 4: Joint positions of the IIWA14 under the P controller in the verification condition (local model). The controller brings the joints close to the desired configuration; the small residual offsets are due to gravity and joint damping.

**Comments.** The P controller is intentionally simple; it does not compensate gravity, so the steady-state configuration has small offsets. The exercise is about the Drake workflow: parser, plant, frames, ports, context, SDF assets, and simulation. The full-system test also shows how to set free body poses for the letters and how to load local SDF files.

## 7 Section 1.7(d): HardwareStation and Advanced Scenarios

The last exercise replaces the manual `DiagramBuilder` wiring with `HardwareStation` and a YAML scenario. The scenario contains `add_model` and `add_weld` directives for two IIWA14 robots, a table, three letters, and a free-floating object. The station is added to a diagram and simulated.

Listing 3: Scenario structure for the bimanual station (abridged).

```
directives:
- add_model:
    name: iiwa_left
    file: file:///.../iiwa14_primitive_collision.urdf
- add_weld:
    parent: world
    child: iiwa_left::iiwa_link_0
- add_model:
```

```
    name: iiwa_right
    file: file:///.../iiwa14_primitive_collision.urdf
- add_weld:
    parent: world
    child: iiwa_right::iiwa_link_0
    X_PC:
        translation: [0.0, 1.0, 0.0]
- add_model:
    name: table
    file: file:///.../assets/table.sdf
- add_weld:
    parent: world
    child: table::table_link
    X_PC:
        translation: [0.5, 0.0, -0.05]
```

The autograders give:

| Test | Score |
|---|---|
| Basic HardwareStation structure | 10/10 |
| Bimanual HardwareStation structure | 10/10 |
| Full system structure (8 model instances) | 15/15 |
| Full-system simulation runs | 15/15 |
| Total | **50/50** |

**Comments.** `HardwareStation` is a composition of a plant, a scene graph, controllers and a visualizer. The scenario directives are a declarative way to describe the same diagram: `add_model` adds a model, `add_weld` fixes a frame, and `default_free_body_pose` sets the initial pose of a free body. The station still exposes named input and output ports (e.g. `iiwa.position`, `iiwa.state_estimated`), so the teleoperation and control code can be connected in the same way as before. The main practical pitfall is the path syntax: local assets must be absolute `file://` URIs, and the base link name used in `default_free_body_pose` must match the SDF.

# 8 Conclusions

1. The two teleoperation examples share one architecture: a pose interface, a differential-IK resolved-rate controller, and a HardwareStation plant. The 2D example has a square task/Jacobian relation; the 3D example is redundant and shows both the convenience and the local nature of diff-IK.

2. The Drake documentation questions are answered: $x(0) = 0.9$, `alpha` for angular acceleration, and `convergence_tol=1.0` in the Python fitted-value-iteration unit test (default `1e-4`).

3. The Drake Systems exercise passes 70/70, the Physics Simulation exercise passes 30/30 in the visible notebook cells, and the HardwareStation exercise passes 50/50. The code demonstrates the full workflow from a custom `LeafSystem` to a multi-robot scenario.

4. The exercises also show the boundary between kinematics and physics: differential IK does not avoid obstacles, and a simple P controller does not compensate gravity. Those are the topics of the following chapters.

# 9 Code and Reproduction

Main files:

| Topic | File |
|---|---|
| Examples 1.2/1.3 | `examples/Exp1.2-1.3-teleoperation.ipynb` |
| 1.7.1 answers | this report, Section 4 |
| 1.7.2 Drake Systems | `exercises/Exp1.7.2-drake-systems.ipynb` |
| 1.7.3 Physics Simulation | `exercises/Exp1.7.3-physics-simulation.ipynb` |
| 1.7.4 HardwareStation | `exercises/Exp1.7.4-hardwarestation.ipynb` |
| Figures | `exp1_analysis/make_ch1_figures.py` |

Headless execution:

```
COLAB_TESTING=1 /Users/leo/venvs/drake/bin/jupyter nbconvert \
  --to notebook --execute --inplace --ExecutePreprocessor.timeout=300 \
  "examples/Exp1.2-1.3-teleoperation.ipynb"
```

Report compilation:

```
pdflatex Ch1_Report.tex
pdflatex Ch1_Report.tex
```